GhostWire Bible

The Comprehensive Guide to Secure, Modular Mesh Networking

Communication for everyone, everywhere, every time.

Version 2.1 ▪ September 2026

Michael Irungu Muriithi ▪ Nairobi, Kenya

Capability claims verified against `rescue/sep-disk-tree` @ `e4028f2`

Contents

Who GhostWire Serves 10
- In this chapter . . . 10
- Executive Summary . . . 10
- 1. The Six Target Personas . . . 10
  - 1. Disaster Responders . . . 10
  - 2. Privacy Advocates . . . 11
  - 3. Rural Communities . . . 11
  - 4. Activists and Journalists . . . 11
  - 5. Whistleblowers . . . 12
  - 6. Developers and Researchers . . . 12
- 2. Persona-to-Capability Map . . . 13
- 3. Four Illustrative Scenarios . . . 14
  - Scenario 1: Community Blackout with Protesters . . . 14
  - Scenario 2: Hurricane Disaster Response . . . 14
  - Scenario 3: Rural Village Connectivity . . . 14
  - Scenario 4: Festival . . . 14
- 4. Which Persona Should Read Which Chapter . . . 15
- End of Chapter . . . 15

GhostWire Project Overview 15
- Executive Summary . . . 15
- Table of Contents . . . 16
- 1. What is GhostWire? . . . 16
- 2. Why Mesh Networking? . . . 16
- 3. Real-World Scenarios . . . 16
  - Communities with Limited Connectivity . . . 16
  - Disaster Response . . . 16
  - Rural Communities . . . 17
  - Community Events . . . 17
- 4. Key Features & Benefits . . . 17
- 5. How GhostWire Works (Plain & Technical) . . . 17
  - Plain-Language Overview . . . 17
  - The Device Hierarchy . . . 17
  - AI Routing . . . 17
  - Technical Deep Dive . . . 17
- 6. Visual Guide: GhostWire in Action . . . 18
- 7. Best Practices & Anti-Patterns . . . 19
  - Best Practices . . . 19
  - Anti-Patterns . . . 19
- 8. Project Status (September 2026) . . . 19
  - Transports Working Today . . . 19
  - Protocol Bridges . . . 19
  - External Security Audit . . . 20
  - New Chapters . . . 20
- 9. Frequently Asked Questions . . . 20
- 10. Further Reading & Resources . . . 20

Appendix: Glossary . . . 20
End of Chapter . . . 20

Getting Started with GhostWire 21
1. Welcome & Audience . . . 21
2. What You Can Do with GhostWire . . . 21
3. Prerequisites . . . 21
4. Build and Run . . . 21
Web UI . . . 22
Terminal UI . . . 22
5. CLI Reference . . . 22
6. Device Roles . . . 23
7. Joining and Creating a Mesh . . . 23
8. Troubleshooting . . . 24
9. Deployment Scenarios . . . 24
10. Best Practices . . . 24
11. Further Reading . . . 24
End of Chapter . . . 25

Architecture Deep Dive 25
Table of Contents . . . 25
1. Overview & Philosophy . . . 25
2. System Diagram & Visuals . . . 25
3. Core Components (Plain & Technical) . . . 25
Plain-Language . . . 25
Technical . . . 27
4. Device Hierarchy and Supernodes . . . 27
Device Classes . . . 27
Roles . . . 27
Supernode Election . . . 28
Trust and Relay Authority . . . 28
Hardware Ladder and ESP32 Gateway Bridge . . . 28
Mailbox Pattern for Leaf Devices . . . 28
5. Transports Layer . . . 28
6. Protocol Adapters Layer . . . 29
7. AI Routing Layer . . . 30
Layer 1: LightGBM Per-Message Classification . . . 30
Layer 2: Gemma 4 Analytical Pass . . . 30
Layer 3: GNN Graph Router . . . 30
Layer 4: PRoPHET Delay-Tolerant Routing . . . 30
Model Ladder . . . 31
Telemetry and Model Distribution . . . 31
Store-and-Forward DTN Integration . . . 31
8. Security & Trust Layer . . . 31
9. Store & Forward / Federation . . . 32
10. Data Flow: Message Lifecycle . . . 32
11. Deployment Blueprints . . . 33
Home/Personal Mesh . . . 33
Community/Neighborhood Mesh . . . 33

Disaster/Field Deployment . . . 33
Enterprise/Federated Mesh . . . 33
12. Best Practices & Anti-Patterns . . . 33
Best Practices . . . 33
Anti-Patterns . . . 33
13. Glossary & Reference . . . 34
End of Chapter . . . 34

Transports & Protocols 34
Table of Contents . . . 34
1. What is a Transport? . . . 34
2. Supported & Planned Transports . . . 35
3. Traffic Armor and Auto-Switching . . . 35
4. Transport Selection: Thompson Sampling . . . 36
5. How Transports Work (Plain & Technical) . . . 36
Plain-Language . . . 36
Technical . . . 36
6. Which Transport When . . . 36
7. Real-World Use Cases . . . 37
Disaster Response . . . 37
Urban Mesh . . . 37
Rural Connectivity . . . 38
Connectivity in Constrained Environments . . . 38
8. Configuration & Code Examples . . . 38
Enabling/Disabling Transports (Config) . . . 38
Using Transports in Code . . . 38
Web UI Example . . . 38
9. Visual Guide: Transport Topologies . . . 38
10. Best Practices & Anti-Patterns . . . 38
Best Practices . . . 38
Anti-Patterns . . . 40
11. Troubleshooting & FAQ . . . 40
12. Further Reading & Resources . . . 40
End of Chapter . . . 40

Security & Privacy 40
Table of Contents . . . 40
1. Executive Summary . . . 41
2. Security Foundations (Plain & Technical) . . . 41
3. Threat Models & Real-World Risks . . . 41
4. Security Architecture & Modules . . . 41
5. Encryption & Key Management . . . 41
6. Modern Crypto & Identity Stack . . . 42
Post-Quantum Key Encapsulation . . . 42
Symmetric Encryption . . . 42
Key Exchange and Forward Secrecy . . . 42
Duress PIN and Coercion Resistance . . . 43
Identity: DIDs, Shamir Secret Sharing, Dead Man's Switch . . . 43
Privacy Tiers (L0-L3) . . . 43

SPHINX Unlinkable Replies and Cover Traffic . . . 43
7. Sybil Defense & Trust . . . 44
8. Quotas, Blacklists, and Abuse Prevention . . . 44
9. Traffic Obfuscation & Anti-Analysis . . . 44
10. Disaster & Infrastructure Failure Scenarios . . . 44
11. Best Practices & Anti-Patterns . . . 44
Best Practices . . . 44
12. Actionable Security Checklists . . . 45
13. Visuals: Security Layers & Flows . . . 45
14. FAQ & Troubleshooting . . . 45
15. External Audit Status . . . 46
16. Further Reading & Resources . . . 46
End of Chapter . . . 46

Protocol Adapters 46
Table of Contents . . . 46
1. What is a Protocol Adapter? . . . 47
2. Supported & Planned Adapters . . . 47
3. How Adapters Work (Plain & Technical) . . . 47
Plain-Language . . . 47
Technical . . . 47
4. The Adapter Registry . . . 48
5. Real-World Bridging Scenarios . . . 48
Disaster Response . . . 48
Activist Network . . . 48
Rural Community . . . 48
6. Configuration & Code Examples . . . 48
Enabling/Disabling Adapters (Config) . . . 48
Using Adapters in Code . . . 48
Web UI Example . . . 49
7. Visual Guide: Adapter Topologies . . . 49
8. Companion Projects . . . 49
9. Developer Notes & API Reference . . . 49
10. Best Practices & Anti-Patterns . . . 50
Best Practices . . . 50
Anti-Patterns . . . 50
11. Troubleshooting & FAQ . . . 50
12. Further Reading & Resources . . . 50
End of Chapter . . . 51

Frequently Asked Questions (FAQ) 51
Table of Contents . . . 51
1. General Questions . . . 51
2. Getting Started . . . 51
3. Technical & Developer Questions . . . 52
4. Security & Privacy . . . 52
5. Real-World Scenarios . . . 53
6. Troubleshooting & Support . . . 53
7. Visuals: Common Flows . . . 53

8. Further Reading & Resources . . . 53
End of Chapter . . . 55

Contributing & Community 55
Table of Contents . . . 55
1. Welcome & Philosophy . . . 55
2. How Anyone Can Contribute . . . 55
3. Code Contributions (Technical) . . . 55
4. Writing, Design, and Outreach . . . 56
5. Community Guidelines & Code of Conduct . . . 56
6. Example Contribution Journey . . . 56
7. Visuals: Contribution Flow . . . 56
8. Best Practices for Contributors . . . 56
9. Further Reading & Resources . . . 58
End of Chapter . . . 58

Deployment & Operations 58
Table of Contents . . . 58
1. Overview . . . 58
2. Deployment Scenarios . . . 58
Home/Personal Mesh . . . 58
Community/Neighborhood Mesh . . . 58
Disaster/Field Deployment . . . 59
Enterprise/Federated Mesh . . . 59
3. Assigning Roles by Power and Hardware . . . 59
Device Classes . . . 59
Roles . . . 59
Role Assignment Principles . . . 59
Battery States and Role Behavior . . . 60
Solar and Battery-Powered LoRa Relays . . . 60
Supernode Election Operations . . . 60
ESP32 Gateway-Bridged Deployments . . . 60
OEM Battery Optimization Note . . . 60
4. Step-by-Step Deployment Guides . . . 61
Local (Laptop/Desktop) . . . 61
Docker . . . 61
Cloud (AWS, GCP, Azure) . . . 61
Bare-Metal . . . 61
5. Docker, Cloud, and Bare-Metal Installations . . . 61
6. Monitoring, Logging, and Maintenance . . . 61
7. Visuals: Deployment Topologies . . . 62
8. Troubleshooting & Recovery . . . 62
9. Best Practices & Anti-Patterns . . . 63
Best Practices . . . 63
10. Further Reading & Resources . . . 63
End of Chapter . . . 63

Advanced Security & Threat Response 63
Table of Contents . . . 63

1. Overview . . . 64
2. Threat Modeling (Plain & Technical) . . . 64
3. Incident Response & Forensics . . . 64
Duress PIN Incident Response . . . 65
4. Security Module Configuration . . . 65
5. Audit & Compliance . . . 65
External Audit Status . . . 65
6. Example Attack/Defense Scenarios . . . 66
7. Visuals: Threat Flows & Response . . . 66
8. Actionable Checklists . . . 67
9. Further Reading & Resources . . . 67
End of Chapter . . . 67

Performance & Scaling 67
Table of Contents . . . 67
1. Overview . . . 67
2. Optimization Tips . . . 68
3. Benchmarks & Metrics . . . 68
4. Tiered Scaling Architecture . . . 68
The Problem with Flat Meshes . . . 68
Supernode and Relay Hub Election . . . 68
Regional DHT Sharding (v3/v4 Plan) . . . 69
Scaling Targets . . . 69
5. Energy-Aware Transport Selection . . . 69
Energy-Efficiency Weights . . . 69
How Thompson Sampling Works . . . 69
Battery-Aware Behavior . . . 69
6. AI Latency Context . . . 70
Inference Timings . . . 70
AI Architecture . . . 70
Telemetry . . . 70
7. Troubleshooting Performance . . . 70
8. Visuals: Performance Flows . . . 71
9. Best Practices & Anti-Patterns . . . 71
Best Practices . . . 71
10. Further Reading & Resources . . . 71
End of Chapter . . . 71

Developer Guide 71
Table of Contents . . . 71
1. Overview . . . 73
2. Extending GhostWire (Transports, Adapters, Modules) . . . 73
Adding a New Transport . . . 73
Adding a Protocol Adapter . . . 73
Adding a Security Module . . . 73
3. Adding a Bridge Adapter . . . 73
Reference: Matrix Adapter . . . 74
Reference: Meshtastic Adapter . . . 74
4. Extending AI Routing . . . 74

5. API Reference & Hooks . . . 74
6. Plugin System & Architecture . . . 74
7. Testing & CI/CD . . . 75
8. Visuals: Developer Flows . . . 75
9. Best Practices & Anti-Patterns . . . 75
Best Practices . . . 75
Anti-Patterns . . . 75
10. Further Reading & Resources . . . 75
End of Chapter . . . 75

Case Studies & Scenario Narratives 75
Table of Contents . . . 75
1. Overview . . . 77
2. Hurricane Response Within 200km . . . 77
3. Community Blackout . . . 79
4. Rural Village Connectivity . . . 79
5. Festival Mesh . . . 79
6. Lessons Learned . . . 82
7. Visuals: Scenario Topologies . . . 82
8. Best Practices & Takeaways . . . 82
9. Further Reading & Resources . . . 85
End of Chapter . . . 85

Glossary & Reference 85
Table of Contents . . . 85
1. Glossary . . . 85
2. Acronyms & Definitions . . . 88
3. Quick Reference: Commands & Config . . . 89
CLI Commands . . . 89
Config Example . . . 89
4. Troubleshooting Table . . . 89
5. Visuals: Reference Flows . . . 89
6. Cross-References . . . 89
7. Further Reading & Resources . . . 91
End of Chapter . . . 91

15. Device Hierarchy, Roles, and Scaling 91
1. Executive Summary . . . 91
2. Why a Hierarchy Exists . . . 91
Plain-Language Explanation . . . 91
Technical Depth . . . 91
3. Device Classes . . . 92
Plain-Language Explanation . . . 92
Technical Depth . . . 92
4. Node Roles — and What "Leaf" Actually Means . . . 93
Plain-Language Explanation . . . 93
Technical Depth . . . 93
5. Relay Authority . . . 94
6. The Mailbox Pattern . . . 94

7. Why a Flat Mesh Fails . . . 94
8. Supernodes and Relay Hubs . . . 95
9. The Hardware Ladder — and the AI Model Ladder . . . 95
Plain-Language Explanation . . . 95
The Hardware Ladder . . . 95
The Model Ladder . . . 95
10. Privacy Tiers by Device . . . 96
Plain-Language Explanation . . . 96
Technical Depth . . . 96
11. A Practical Worked Example . . . 96
12. Best Practices and Anti-Patterns . . . 98
Anti-Patterns . . . 98
13. Glossary . . . 98
14. Further Reading . . . 99
End of Chapter . . . 99

16. AI Routing and Intelligence 99
Table of Contents . . . 99
1. Executive Summary . . . 99
2. The Four-Layer AI Routing Stack . . . 100
Why the model layers come last . . . 100
3. Layer 1: LightGBM Per-Message Classifier . . . 101
Plain-Language . . . 101
Technical Depth . . . 101
4. Layer 2: Gemma 4 Edge Model . . . 102
Plain-Language . . . 102
Technical Depth . . . 102
5. Layer 3: Graph Neural Network (GNN) . . . 102
Plain-Language . . . 102
Technical Depth . . . 102
6. Layer 4: PRoPHET Delay-Tolerant Routing . . . 103
Plain-Language . . . 103
Technical Depth . . . 103
7. Model Ladder per Hardware . . . 104
8. Model Update Flow and Training Pipeline . . . 104
Plain-Language . . . 104
Technical Depth . . . 104
9. Thompson Sampling Transport Selection . . . 105
Plain-Language . . . 105
Technical Depth . . . 105
10. Privacy and Local AI Decisions . . . 105
Plain-Language . . . 106
11. Knowledge Ark and Anomaly Detection . . . 106
Plain-Language . . . 106
Technical Depth . . . 106
12. Worked Example: A Message's Journey . . . 106
Plain-Language . . . 106
Technical Depth . . . 106

13. Future Scopes 107
Plain-Language 108
Technical Depth 108
14. End of Chapter 108
End of Chapter 108

Who GhostWire Serves

In this chapter

1. Executive Summary
2. The Six Target Personas
3. Persona-to-Capability Map
4. Four Illustrative Scenarios
5. Which Persona Should Read Which Chapter

Executive Summary

GhostWire is built for six communities that need communication when traditional networks fail, when they are watched, or when no network exists at all. This chapter describes each community, what they need, and how GhostWire features map to those needs. If you are not a technical reader, this chapter will help you understand who GhostWire is for and why it matters.

The six communities are disaster responders who coordinate when cell towers fall, privacy advocates who refuse to be tracked, rural communities that lack reliable internet, activists and journalists who speak under threat, whistleblowers who expose wrongdoing without being traced, and developers and researchers who build on open code. Each community uses a different combination of GhostWire features, and the modular design of the platform means that no one is forced to use features they do not need.

This chapter describes who GhostWire is for and which parts of it serve each community.

---

1. The Six Target Personas

GhostWire serves six distinct communities. Each has a different reason for needing the platform, a different threat model, and a different set of features that matter most to them. The taglines below are drawn directly from the project website and describe each community in a single sentence.

1. Disaster Responders

Tagline: "Coordinate when cell towers fail. Works on battery and solar"

Disaster responders need to communicate when the infrastructure is destroyed. Cell towers go down, internet connections disappear, and traditional phones become useless. GhostWire provides mesh networking over stealth TCP, WebRTC, and Reticulum, plus delay-tolerant delivery so messages are stored and forwarded when a connection becomes available. **WiFi Direct, Bluetooth LE, and LoRa are designed but not implemented** — see the status spine §4.

In a hurricane response scenario within 200 kilometers of infrastructure, responders can deploy LoRa nodes at key locations and use battery-powered devices to maintain coordination. Messages hop from node to node until they reach their destination, even if no direct path exists. The store-and-forward routing ensures that no message is lost, even when connections are intermittent. This is covered in detail in Chapter 04 on transports.

Disaster responders also benefit from the security architecture described in Chapter 05. XChaCha20-

Poly1305 encryption with a hybrid classical + post-quantum handshake protects sensitive operational data, and the delay-tolerant networking layer ensures that messages survive temporary disconnections. The device hierarchy in Chapter 15 describes how responders can use battery-aware scaling to preserve both charge and operational security.

2. Privacy Advocates

Tagline: "No logs. No traces"

Privacy advocates need communication that leaves no metadata behind. GhostWire supports metadata minimization techniques and multi-profile identities, allowing users to maintain separate personas for different contexts. The network does not require phone numbers, email addresses, or any other identifying information to join.

The privacy architecture is detailed in Chapter 05. Messages carry only what is necessary for routing. No phone numbers, no email addresses, no persistent identifiers. Multi-profile identities let users compartmentalize their digital lives, keeping work, personal, and sensitive communication entirely separate. Traffic obfuscation through cover traffic and timing randomization makes traffic analysis significantly harder.

The privacy architecture is designed so that no raw message content is analyzed: routing decisions work on transport and topology metadata, not message bodies. Note that the learned routing layers described in Chapter 16 are not active in the current build — routing uses PRoPHET and the mesh — so there is no inference to opt into today.

3. Rural Communities

Tagline: "2.6 billion people lack reliable internet"

Rural communities around the world lack dependable internet access. GhostWire enables community-owned mesh networks using LoRa and WiFi. The AI routing system can be trained on datasets like the GuifiSants dataset, which contains 7,931 samples from 63 nodes over 31 days, to optimize routing for real-world rural topologies.

Community members can share news, alerts, and resources without relying on an internet service provider. The mesh is owned and operated by the community itself. Each device is a node, and the more people who join, the stronger the network becomes. The hardware ladder described in Chapter 15 is a design target for low-cost sensor and leaf nodes. **No ESP32 firmware exists in the repository**, and the LoRa transport that such nodes would use is not implemented.

Training a routing model on public mesh datasets such as GuifiSants (7,931 samples, 63 nodes, 31 days) is a planned step. **No model has been trained or shipped**: there are no model files in the repository, and the learned routing layers are not called from routing decisions today. See Chapter 16 and the status spine §5.

4. Activists and Journalists

Tagline: "Coordinate without infiltration. Disposable identities. Mesh during protests."

Activists and journalists face internet shutdowns, surveillance, and censorship. During protests, governments shut down internet access. Journalists lose their ability to communicate with editors. Activists are infiltrated by informants. Standard communication tools become liabilities.

GhostWire provides mesh networking that works during internet shutdowns using WiFi Direct and Bluetooth Low Energy. Disposable identities allow users to create separate profiles for different operations and discard them when compromised. Direct peer connection uses public key identifiers rather than a central registry, so there is no directory to subpoena. (QR pairing was described in earlier revisions of this chapter; it is not implemented in the current build.)

Sybil resistance through proof-of-work node admission makes it computationally expensive to flood the mesh with fake nodes; peers whose NodeId PoW does not verify are rejected (`routing/skademlia.rs:171,328`). **Important caveat:** the eclipse-defence functions that would enforce IP diversity and disjoint lookups are defined but never called, so this protection is narrower than it appears — see the status spine §6.

5. Whistleblowers

Tagline: "Send documents without revealing identity. Tor integration supported."

Whistleblowers need to transmit sensitive documents without revealing their identity. Standard channels leave traces. Even encrypted services can be compelled to hand over metadata. GhostWire provides several features that address this need.

Tor integration is planned and will route GhostWire traffic through the Tor network for additional anonymity layers. In the meantime, stealth TCP provides obfuscated transport that can blend GhostWire traffic with normal HTTPS traffic, making it harder to detect. Ephemeral sessions ensure that session keys are destroyed after use, leaving no persistent state to subpoena.

No identifying information is required to create a GhostWire identity. No phone number, email, or real name is needed. The end-to-end encryption described in Chapter 05 ensures that documents are encrypted before they leave the device, and no intermediary can read them.

6. Developers and Researchers

Tagline: "AGPL-licensed. Built in Rust. AI models trained on real mesh data."

GhostWire is **dual-licensed**: AGPL-3.0-or-later for personal, research, educational, and open-source use, and a separate commercial licence for proprietary, closed-source, and SaaS deployments (see `LICENSE`). The codebase is written in Rust for memory safety. The architecture supports pluggable transports and an adapter registry for building bridges to other protocols.

The technology stack includes libp2p networking, Kademlia DHT, Gossipsub pub/sub, stealth TCP and WebRTC transports, and a web dashboard. An AI layer is designed — LightGBM anomaly detection exported to ONNX, and a Graph Neural Network — but **no model files are present in the repository and the AI layers are not wired into routing decisions**. Researchers can study the routing algorithms and contribute transports today; contributing to the AI models requires training them first.

The adapter registry described in Chapter 04 provides a structured way to add new transports and protocols. Developers can implement the relevant adapter interface and register it without modifying the core codebase. This modular design makes GhostWire a fertile ground for research into mesh networking, privacy-preserving communication, and AI-driven routing.

---

2. Persona-to-Capability Map

The diagram below maps each persona to the GhostWire capabilities they rely on. Cross-references point to the chapters that describe those capabilities in detail.

target users

Diagram 1: target users

The table below provides a more detailed mapping of features to personas.

Yes = works in the current build. **Planned** = designed, not implemented. **Partial** = code exists but is not wired into a user-reachable path.

Feature	Status	Disaster	Privacy	Rural	Activists	Whistleblowers	Developers
Stealth TCP / WebRTC / Reticulum	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Delay-Tolerant Delivery	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Hybrid PQ + XChaCha20 encryption	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Sphinx onion routing (SURBs)	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Multi-Profile Identities	Partial	No	Yes	No	Yes	Yes	Yes
Sybil Proof-of-Work admission	Partial	No	No	No	Yes	No	Yes
Pluggable Transports	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Adapter Registry	Partial	No	No	No	No	No	Yes
WiFi Direct / BLE Mesh	Planned	Yes	Yes	Yes	Yes	Yes	Yes
LoRa long-range	Planned	Yes	No	Yes	Yes	No	Yes
QR Pairing	Planned	No	No	No	Yes	Yes	Yes

Feature	Status	Disaster	Privacy	Rural	Activists	Whistleblowers	Developers
AI routing (LightGBM / GNN)	Planned	Yes	Yes	Yes	No	No	Yes
Tor Integration	Planned	No	No	No	No	Yes	Yes

> The earlier version of this table marked unimplemented transports, QR pairing, and AI routing as "Yes" for every persona, which overstated what a user of any of these communities could actually do. Status column added 2026-09-28; see docs/BIBLE_STATUS_SPINE.md.

---

3. Four Illustrative Scenarios

The following scenarios are illustrative examples drawn from the project's documented use cases. They show how GhostWire might work in practice for different communities. These are not descriptions of completed deployments but rather plausible scenarios based on the project's stated goals and capabilities. Chapter 13 frames these same scenarios in detail and is cross-referenced below.

Scenario 1: Community Blackout with Protesters

During a citywide internet blackout, a community forms a mesh using phones and laptops. Bluetooth and WiFi Direct connect nearby devices, and a few laptops act as relays. Messages flow through the mesh without any internet connection. Activists coordinate protests, share safety alerts, and maintain communication with external journalists. This scenario is explored in more detail in the Case Studies chapter, which frames it as a community network during a communication blackout.

Scenario 2: Hurricane Disaster Response

After a hurricane destroys infrastructure, responders deploy LoRa nodes at shelters and staging areas. Battery-powered devices communicate over long distances. Store-and-forward delivery ensures that messages reach their destination even when nodes are temporarily out of range. The AI routing system helps messages find the best available path across the mesh. The Case Studies chapter describes this as a disaster response in a blackout, with responders operating 200 kilometers from infrastructure.

Scenario 3: Rural Village Connectivity

A village with no internet access connects homes, a school, and a clinic using LoRa and WiFi. A Raspberry Pi gateway bridges the local mesh to the wider network when a satellite internet connection is available. The AI routing system, trained on real-world rural data including the GuifiSants dataset, optimizes message paths. Community members share news, alerts, and educational resources. The Case Studies chapter frames this as rural village connectivity, showing how community-owned mesh transforms isolated areas.

Scenario 4: Festival

At a large outdoor festival, attendees use GhostWire to share updates and stay connected when cell networks become overloaded. Phones form a Bluetooth and WiFi mesh, and information spreads quickly

through the crowd. The tiered architecture from Chapter 15 ensures that the mesh remains stable even as hundreds of devices join simultaneously. The Case Studies chapter describes this as a community event mesh, demonstrating how GhostWire handles congestion during large gatherings.

---

4. Which Persona Should Read Which Chapter

Different readers will find different chapters most useful. The table below helps each persona navigate the GhostWire Bible to the sections most relevant to their needs.

Persona	Start Here	Then Read	Also Relevant
Disaster Responders	Chapter 04 (Transports)	Chapter 15 (Hierarchy)	Chapter 05 (Security), Chapter 13 (Case Studies)
Privacy Advocates	Chapter 05 (Security)	Chapter 16 (AI Routing)	Chapter 04 (Transports), Chapter 13 (Case Studies)
Rural Communities	Chapter 16 (AI Routing)	Chapter 15 (Hierarchy)	Chapter 04 (Transports), Chapter 13 (Case Studies)
Activists and Journalists	Chapter 05 (Security)	Chapter 04 (Transports)	Chapter 15 (Hierarchy), Chapter 13 (Case Studies)
Whistleblowers	Chapter 05 (Security)	Chapter 04 (Transports)	Chapter 13 (Case Studies)
Developers and Researchers	Chapter 04 (Transports)	Chapter 06 (Adapters)	Chapter 12 (Developer Guide), Chapter 16 (AI Routing)
General Readers	Chapter 01 (Project Overview)	Chapter 07 (FAQ)	Chapter 13 (Case Studies), Chapter 14 (Glossary)

End of Chapter

GhostWire Project Overview

Executive Summary

GhostWire is a modular, privacy-focused mesh networking and messaging platform designed for everyone, from disaster responders and journalists to rural communities and tech enthusiasts. It enables secure, decentralized communication even when the internet is down or unavailable.

Table of Contents

1. What is GhostWire?
2. Why Mesh Networking?
3. Real-World Scenarios
4. Key Features & Benefits
5. How GhostWire Works (Plain & Technical)
6. Visual Guide: GhostWire in Action
7. Best Practices & Anti-Patterns
8. Frequently Asked Questions
9. Further Reading & Resources

---

1. What is GhostWire?

GhostWire lets people connect and communicate directly, forming a mesh network over whichever transports are available: stealth TCP, WebRTC, and Reticulum today, with QUIC implemented and LoRa, WiFi Direct, and Bluetooth LE designed but not yet built. A protocol-bridge registry lets it interoperate with other networks (Briar, Meshtastic, Matrix — none of these bridges is implemented yet). Privacy and security are the point of the design: hybrid post-quantum key establishment, onion routing, and coercion resistance.

- **For non-technical readers:** Think of GhostWire as a walkie-talkie for the digital age, but smarter and more private. You can send messages, share files, and connect with others, even if the internet is blocked or down.
- **For technical readers:** GhostWire is a modular, extensible platform built in Rust and TypeScript, supporting pluggable transports, protocol adapters, and advanced security modules.

---

2. Why Mesh Networking?

- **Resilience:** Mesh networks do not rely on a single server or internet connection. If one device goes down, others keep the network alive.
- **Privacy:** No central authority means enhanced data protection and user autonomy.
- **Flexibility:** Pluggable transports. Three work today (stealth TCP, WebRTC, Reticulum); LoRa, WiFi Direct, and BLE are designed and pending.

---

3. Real-World Scenarios

Communities with Limited Connectivity

- **Problem:** Internet blackouts and connectivity constraints.
- **Solution:** GhostWire forms a local mesh, letting communities communicate securely and privately.

Disaster Response

- **Problem:** Infrastructure is down after a natural disaster.
- **Solution:** First responders use GhostWire to coordinate rescue efforts, even without cell towers.

Rural Communities

- **Problem:** No reliable internet access.
- **Solution:** GhostWire connects villages using LoRa and WiFi, enabling messaging and information sharing.

Community Events

- **Problem:** Overloaded networks at large gatherings.
- **Solution:** Attendees use GhostWire to share updates and stay connected.

---

4. Key Features & Benefits

- **Decentralized:** No single point of failure.
- **Privacy-First:** End-to-end encryption, metadata protection, and traffic obfuscation.
- **Modular:** Add or remove transports and adapters as needed.
- **Cross-Protocol:** Bridge to Briar, Meshtastic, Matrix, and more.
- **User-Friendly:** Simple web and mobile interfaces.
- **Open Source:** Transparent, auditable, and community-driven.

---

5. How GhostWire Works (Plain & Technical)

Plain-Language Overview

- Devices connect directly to each other, forming a web (mesh).
- Messages hop from device to device until they reach their destination.
- No internet? No problem, use Bluetooth, WiFi, or LoRa.
- Everything is encrypted and private.

The Device Hierarchy

Not all devices are equal in GhostWire. The network organizes devices into tiers based on available RAM, storage, and power. A nearby laptop can take over relay duty from a phone, keeping the mesh alive when a low-power device needs to rest. This tiered approach keeps the network stable as it grows. The hierarchy works like this: phones and small devices handle sending and receiving, while laptops and servers take on heavier relay and storage tasks. As one project note puts it, "A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture."

AI Routing

GhostWire uses artificial intelligence to decide how messages travel across the mesh. A lightweight classification model examines each message and picks the best transport and route. A larger analytical model runs periodically to update the network map. This means messages can find their way even when the network is changing constantly, and the AI models run on devices themselves, not on a remote server.

Technical Deep Dive

- **Core:** Rust backend with modular traits for transports, adapters, and security.

- **Frontend:** React/TypeScript web UI, CLI for power users.
- **Transports:** Pluggable modules. Working: stealth TCP, WebRTC, Reticulum. Implemented but not on the mesh handshake: QUIC. Designed, not implemented: LoRa, WiFi Direct, Bluetooth LE.
- **Adapters:** A protocol-bridge registry for Briar, Meshtastic, Matrix, and others. **None of these bridges is implemented yet** — see the status spine §7.
- **Security:** End-to-end encryption using **XChaCha20-Poly1305** with Ed25519 identities, X25519 key exchange, and a hybrid **ML-KEM-768 post-quantum** handshake. Sybil defence via proof-of-work node admission, quotas, blacklists, and traffic obfuscation.
- **Store & Forward:** Messages are cached and relayed when possible.

---

6. Visual Guide: GhostWire in Action

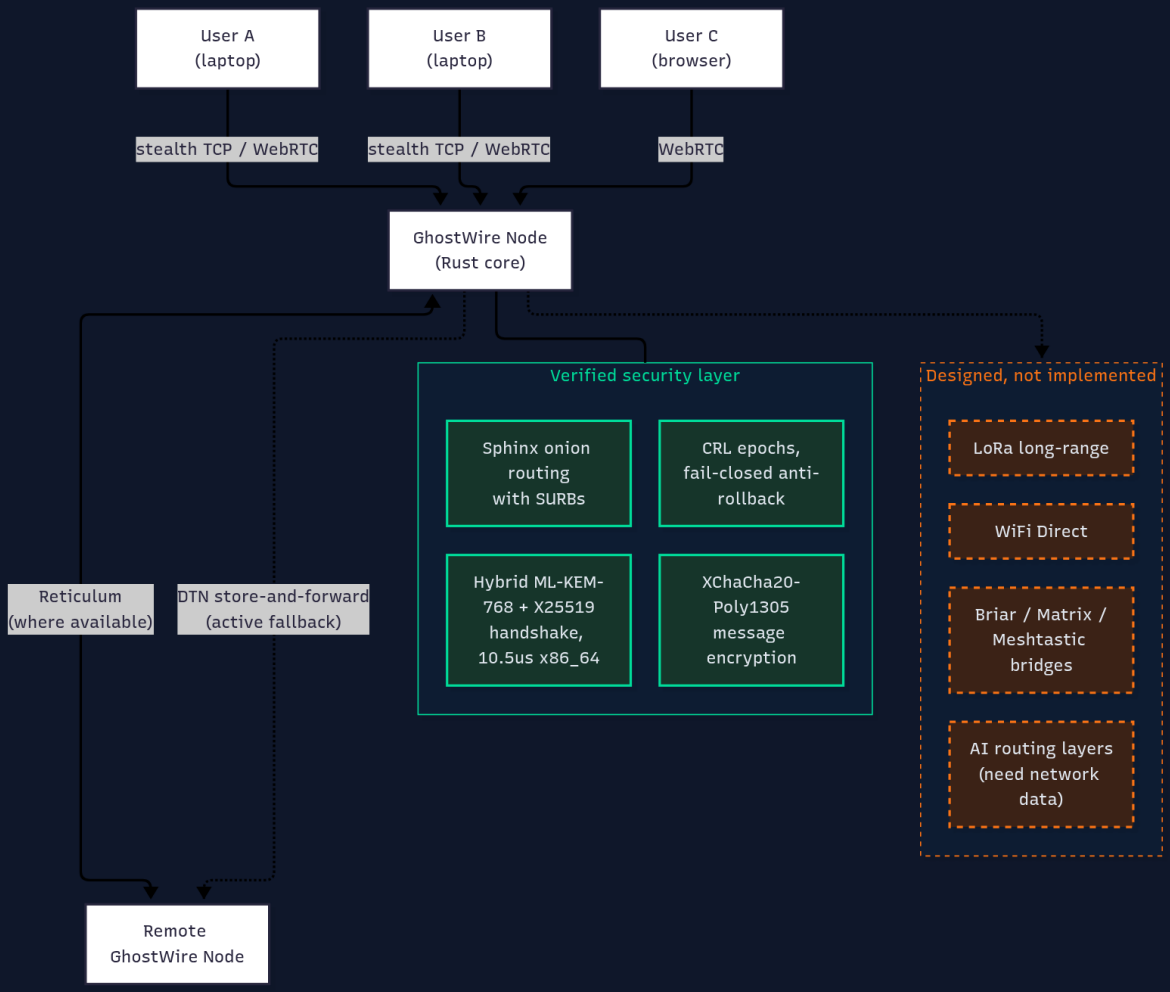


Diagram 2: project overview

Solid green = verified in the current build. Dashed orange = designed but not implemented. Full evidence in docs/BIBLE_STATUS_SPINE.md.

---

7. Best Practices & Anti-Patterns

Best Practices

- Use multiple transports for resilience.
- Keep software updated for latest security patches.
- Use strong, unique passwords for device access.
- Educate users about privacy and security features.

Anti-Patterns

- Relying on a single transport (e.g., only WiFi).
- Disabling encryption or security modules.
- Ignoring software updates.

---

8. Project Status (September 2026)

GhostWire is an active, growing project. As of 2026-09-28, `ghostwire/src/` contains 58,166 lines of Rust across 151 files (66,116 lines crate-wide including tests and benches). The test suite runs 368 unit tests, all passing. The project has applied to the Open Technology Fund, structured as 12 milestones across 8 phases. **Per-capability status with file-and-line evidence is maintained in docs/BIBLE_STATUS_SPINE.md; where this document and that file disagree, that file is correct.**

Transports Working Today

Three transports are working today: stealth TCP, WebRTC, and Reticulum. QUIC is implemented but not yet on the mesh handshake. Bluetooth LE is a skeleton (the send path logs and returns), WiFi Direct returns `unsupported` because no OS provider is installed, and LoRa has no implementation at all — `TODO.md` records it as "protocol defined but no hardware integration." Full per-transport status with file-and-line evidence: docs/BIBLE_STATUS_SPINE.md §4. Traffic Armor provides camouflage and auto-switching when a transport is blocked, using Thompson Sampling for energy-aware transport selection (energy weights: BLE 0.30, LoRa 0.35, Reticulum 0.25, WiFi 0.05, TCP/QUIC 0 — these are selector weights for *planned* transports as well as live ones).

Protocol Bridges

- **Matrix:** Planned. No matrix bridge module exists in the codebase.
- **Meshtastic:** Planned. No meshtastic module exists. (Erasure-coding code exists for LoRa framing, but there is no bridge.)
- **Briar:** Planned. `briar.rs` contains Briar-*inspired* struct definitions; `BriarManager` is never instantiated and the registration call is commented out. It sits behind a non-default feature flag.
- **Reticulum:** This one is real, and it is a first-party transport rather than a bridge — 883 lines, wired into the transport layer.
- **Tor:** Not implemented.

External Security Audit

An external security audit is planned but has not yet been conducted. Candidate auditors include Cure53, NCC Group, and Least Authority.

New Chapters

Three new chapters expand the Bible: Chapter 15 covers device hierarchy and power-aware scaling, Chapter 16 covers AI routing and on-device intelligence, and the early chapter on who GhostWire serves.

---

9. Frequently Asked Questions

Q: Is GhostWire legal to use? A: In most countries, yes, but always check local laws, especially regarding encryption and radio use.

Q: Can I use GhostWire without the internet? A: Yes. That is one of its main features.

Q: How secure is GhostWire? A: It uses strong encryption and privacy techniques, but no system is 100% secure. Follow best practices.

Q: Can I contribute? A: Absolutely. See the Contributing chapter.

---

10. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101
- Briar Project
- Meshtastic
- Matrix Protocol
- LoRa Technology
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence
- Who GhostWire Serves

Appendix: Glossary

- **Mesh Network:** A network where each device relays data for others.
- **Transport:** The method used to send messages (Bluetooth, WiFi, etc.).
- **Protocol Adapter:** Software that bridges GhostWire to other networks.
- **Node:** Any device running GhostWire.
- **Sybil Attack:** When one entity pretends to be many nodes.
- **Quota:** Limit on messages/actions to prevent abuse.

End of Chapter

Getting Started with GhostWire

1. Welcome & Audience

GhostWire is for community organizers, first responders, privacy advocates, rural users, and developers. This chapter walks through your first steps regardless of background.

> **Before you start — what GhostWire is today.** GhostWire is a **protocol with a reference implementation**, not a packaged consumer product. There is no app store download and no hosted service. You run the node yourself, from source. Three transports are working (stealth TCP, WebRTC, Reticulum); others are designed but not implemented. Per-capability status with evidence is in docs/BIBLE_STATUS_SPINE.md — read it before making deployment decisions.

2. What You Can Do with GhostWire

- **Send messages** to peers on the same mesh, including when the internet is unreachable — via delay-tolerant store-and-forward.
- **Join a local mesh** at an event, in a neighbourhood, or across a campus.
- **Encrypt traffic** with a hybrid classical + post-quantum handshake, and route it over an onion-routed path.
- **Build on the protocol.** This is the intended use: implement the wire format in another language or application and interoperate.

Not available today: a mobile app, a public demo instance, a hosted relay service, or bridges to Briar/Meshtastic/Matrix. Those are on the roadmap, not in the build.

3. Prerequisites

Requirement	Why	Notes
Rust (stable toolchain)	Builds the node	https://rustup.rs
Node.js + npm	Builds the web UI	https://nodejs.org
~2 GB disk	Build artifacts	Release build of the node is large
LoRa / RISC-V hardware	Optional, long-range	**Transport not implemented** — see ch04

No GPU is required. The AI routing layers are inactive in the current build (ch16), so no model files or Ollama instance are needed.

4. Build and Run

```
# 1. Clone
git clone https://github.com/phantomojo/GhostWire-secure-mesh-communication.git
```

```
cd GhostWire-secure-mesh-communication/ghostwire

# 2. Create an identity (required before first run)
cargo run --release -- init -u yourname

# 3. Start the node
cargo run --release -- start --host 127.0.0.1 --port 8080 --web
```

The node serves its HTTP API on `127.0.0.1:8080` by default.

Web UI

The web UI is a separate build. It is **not required** — the node's HTTP API and the terminal UI both work on their own.

```
cd ../webui
npm install
npm run dev
```

> **Known gap:** the `webui` directory in the current repository does not contain a `package.json`, so the commands above will not run as written. The served web assets are pre-built under `webui/dist`. Until the source front-end is restored, use the TUI or the HTTP API. Tracked as a documentation defect — not a workaround you need to solve.

Terminal UI

```
cargo run --release -- tui
```

---

5. CLI Reference

The binary is `ghostwire`. The subcommands below are taken from `src/cli/mod.rs`; this is the real interface, not an illustration.

Command	Purpose
`ghostwire init -u <name>`	Create an identity (do this first)
`ghostwire start [--host H] [--port P] [--web] [--mode M]`	Run the node. `--mode` is `hybrid` (default), `libp2p-only`, or `reticulum-only`
`ghostwire send -r <peer> -m <text>`	Send a message to a peer
`ghostwire peers`	List connected peers
`ghostwire stats`	Network statistics
`ghostwire security`	Security information
`ghostwire genkey`	Generate a key pair
`ghostwire encrypt -e <text>` / `decrypt -d <ciphertext>`	Crypto operations
`ghostwire connect -u <username> [--anonymous]`	Connect to a peer; `--anonymous` uses SURB-based introduction
`ghostwire manage <start|stop|restart|status|logs|backup|restore>`	Process and data management
`ghostwire monitor [-f]`	Network and security monitor

Command	Purpose
`ghostwire tui`	Interactive terminal interface
`ghostwire setup`	Set up dependencies and environment
`ghostwire deploy -t <target> -p <platform>`	Deploy payloads

Examples:

```
ghostwire start --web --port 8080
ghostwire connect -u alice
ghostwire connect -u alice --anonymous     # SURB-based, no direct peer ID needed
ghostwire send -r alice -m "Hello, Bob!"
ghostwire manage logs -f
```

---

6. Device Roles

Your hardware determines what role you can play. Peers relay for each other, so a device class that never relays reduces the mesh's reach.

Device	Memory	Max Peers	Storage	Typical Role
Phone	≤6 GB	20	1 GB	Leaf — messaging only
Tablet	≤8 GB	30	2 GB	Leaf or light relay
Raspberry Pi	≤4 GB	15	500 MB	Relay or sensor
Laptop	≤16 GB	50	5 GB	Good relay
Server	>16 GB	100+	50 GB+	Relay and replicator

Energy-aware behaviour. The selector weights transports by energy cost (BLE 0.30, LoRa 0.35, Reticulum 0.25, WiFi 0.05, TCP/QUIC 0) and a `DeviceTier` (High / Medium / Low) applies cover-traffic intervals of approximately 5–15 seconds depending on tier.

> **Two corrections to earlier versions of this chapter.** Battery level does **not** currently drive privacy behaviour: the L0–L3 privacy tiers described in earlier revisions are documentation-only — `PrivacyParams` and `PrivacyLevel` have no implementation in `src/`. The device also **cannot read battery state** today. QR pairing is likewise not implemented. What does exist is the `DeviceTier` enum and the energy weights above. See the status spine §5.

---

7. Joining and Creating a Mesh

- **Discovery:** peers find each other over the active transports (libp2p identify / gossipsub, plus Reticulum when enabled). Discovery over WiFi Direct and LoRa is designed but **not implemented**.
- **Joining a known peer:** `ghostwire connect -u <username>`, or `--anonymous` for SURB-based introduction.
- **Group messaging:** the protocol defines a group/social layer; the UI for creating and joining groups is not complete in the current build.

---

8. Troubleshooting

Problem	Likely cause	Action
No peers discovered	No working transport enabled	`ghostwire stats` to see active transports; confirm WebRTC or Reticulum
`init` not run	Missing identity	`ghostwire init -u <name>`
Web UI does not load	`webui` has no `package.json` in this tree	Use `ghostwire tui` or the HTTP API on `:8080`
LoRa or WiFi Direct unavailable	Not implemented	Use stealth TCP, WebRTC, or Reticulum
Update or integrity check disabled	Build-time keys unset	Expected in local builds; set `GHOSTWIRE_RELEASE_VERIFYING_KEY_HEX` and `GHOSTWIRE_BINARY_HASH` for release builds
Test failures mention `GHOSTWIRE_HOME`	Tests share on-disk state	Serialise; see status spine §10

---

9. Deployment Scenarios

> These are **scenarios the design targets**, not deployments that have been exercised. Treat them as illustrations of intent.

Community event. Organisers run nodes on phones and laptops, form a mesh, and share peer identifiers. Viable today over WebRTC or a local network.

Blackout response. Responders run nodes with no internet; delay-tolerant store-and-forward carries messages until a path reappears. This is the best-supported scenario in the current build — the DTN layer is tested (9 tests in `dtn/epidemic.rs`).

Rural deployment. Homes relay for each other over a mix of long-range and local links. **Blocked today**: the long-range transports (LoRa, WiFi Direct) are not implemented, so this scenario currently depends on Reticulum infrastructure or conventional WiFi/cellular.

---

10. Best Practices

- Run `ghostwire init` before anything else.
- Use the latest build — the revocation and anti-rollback work is recent.
- Treat mesh access credentials as secrets; there is no default shared password.
- Read the status spine before relying on a capability. It is the honest source.
- Back up with `ghostwire manage backup` before major changes.

---

11. Further Reading

- Capability Status Spine — what is actually built

- Security & Privacy
- Transports & Protocols
- Developer Guide
- Mesh networking overview
- GhostWire repository

End of Chapter

Architecture Deep Dive

Table of Contents

1. Overview & Philosophy
2. System Diagram & Visuals
3. Core Components (Plain & Technical)
4. Device Hierarchy and Supernodes
5. Transports Layer
6. Protocol Adapters Layer
7. AI Routing Layer
8. Security & Trust Layer
9. Store & Forward / Federation
10. Data Flow: Message Lifecycle
11. Deployment Blueprints
12. Best Practices & Anti-Patterns
13. Glossary & Reference

---

1. Overview & Philosophy

GhostWire is built for modularity, security, and real-world flexibility. The architecture is designed to: - Support multiple transports and protocols - Enable privacy and resilience by default - Allow easy extension and adaptation for new use cases

---

2. System Diagram & Visuals

---

3. Core Components (Plain & Technical)

Plain-Language

- **Web UI:** The dashboard you use in your browser.
- **CLI:** Command-line tool for advanced users.
- **Backend:** The "brain" that connects everything, runs on your device.
- **Transports:** The "roads" messages travel on (WiFi Direct, LoRa, stealth TCP, WebRTC, Reticu-

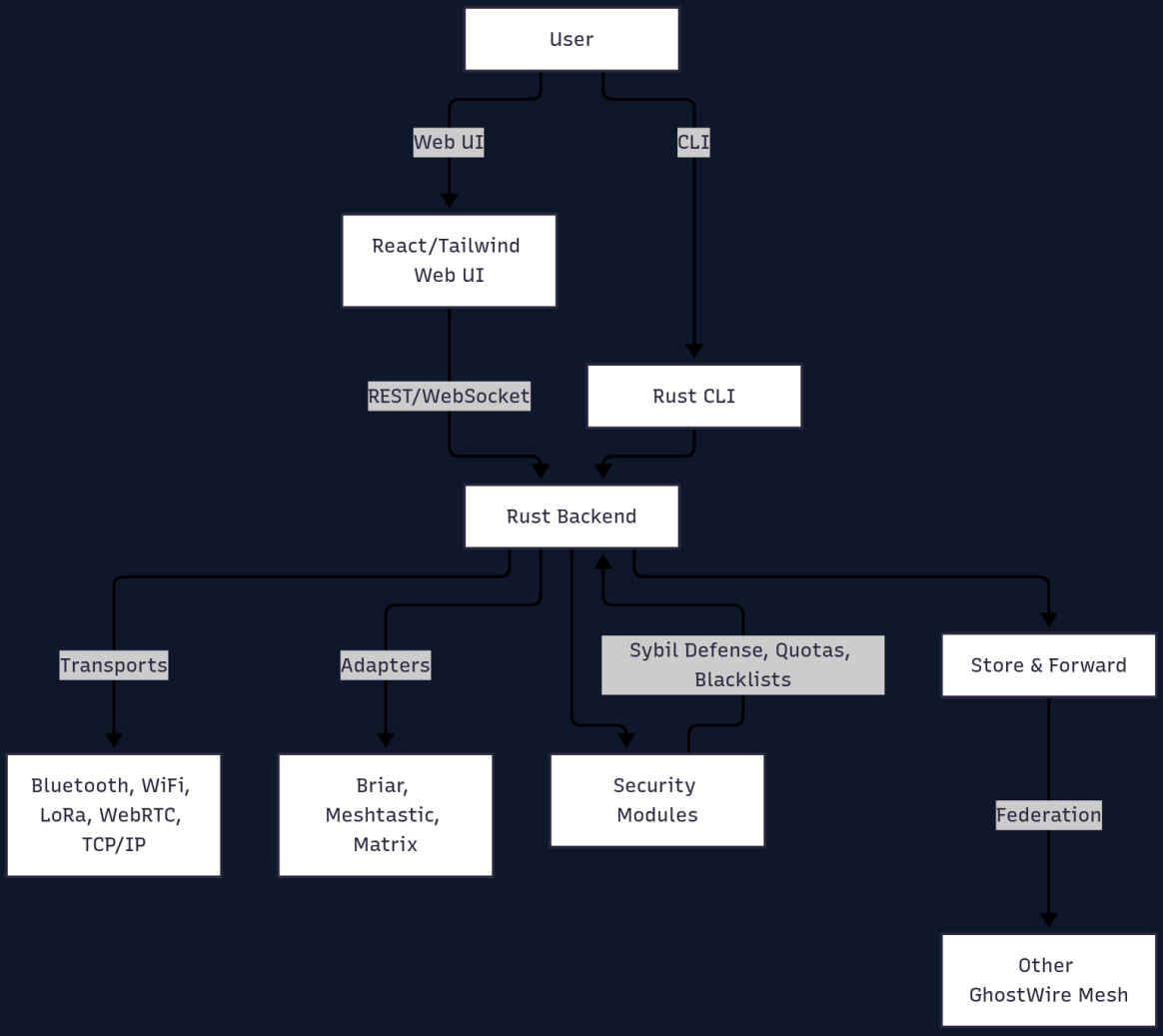


Diagram 3: architecture

lum).
- **Adapters:** "Translators" that let GhostWire talk to other networks.
- **Security Modules:** Keep your messages private and your network safe.
- **Store & Forward:** Lets messages wait and be delivered later if needed.

Technical

- **Rust Backend:** Implements core traits: `Transport`, `ProtocolAdapter`, `KeyManager`, `QuotaEnforcer`, etc.
- **Frontend:** React/TypeScript, communicates via REST/WebSocket APIs.
- **Transports:** Each is a Rust module implementing the `Transport` trait, can be enabled/disabled at runtime.
- **Adapters:** Rust modules implementing `ProtocolAdapter`, handle translation, deduplication, and relay.
- **Security:** Modular, pluggable, with Sybil defense, quotas, blacklists, traffic obfuscation, and more.
- **Store & Forward:** Message cache, relay, and federation logic.

---

4. Device Hierarchy and Supernodes

A flat mesh becomes unstable beyond a certain density. We introduce a tiered architecture so that devices with more resources can serve as relays and anchors, while resource-constrained devices focus on messaging.

Device Classes

Class	Memory	Max Peers	Storage	Role
Phone	<=6GB	20	1GB	Leaf
Tablet	<=8GB	30	2GB	Leaf or light Relay
Raspberry Pi	<=4GB	15	500MB	Relay or Sensor
Laptop	<=16GB	50	5GB	Relay
Server	>16GB	100+	50GB+	Full, Replicator

Roles

- **Full Node:** Maintains the complete mesh state, participates in routing and relay. Typically a server or always-on laptop.
- **Relay:** Forwards messages between nodes. Can be a Raspberry Pi, laptop, or tablet with sufficient uptime and bandwidth.
- **Leaf:** A messaging endpoint that does not relay traffic for others. Phones and lightweight tablets typically operate as leaves.
- **Sensor:** A device that generates or consumes data but does not relay. Examples include environmental sensors or battery-constrained devices.
- **Replicator:** A full node that stores and replicates messages for the mesh, ensuring availability even when original senders are offline. Typically a server-class machine.

Supernode Election

Dynamically elect nodes with high uptime, stable IP addresses, and significant bandwidth to act as "Relay Hubs." These supernodes coordinate routing tables and serve as anchor points for the mesh. Election is based on observable metrics: uptime history, bandwidth availability, and peer count. A node does not need to be a server to become a supernode; a Raspberry Pi with stable power and connectivity can qualify.

Trust and Relay Authority

Relays carry no trust authority. A relay forwards messages but cannot validate, modify, or censor the content it carries. Trust is established end-to-end between the original sender and the final recipient through cryptographic signatures. This separation ensures that even a compromised relay cannot impersonate a user or read message contents.

Hardware Ladder and ESP32 Gateway Bridge

The hardware ladder runs from a $20 ESP32 through a $100 Raspberry Pi gateway to server-class machines. The ESP32 gateway-bridged architecture creates a mini-mesh locally and bridges it to a WiFi uplink, allowing low-cost sensors and devices to participate in the mesh through a gateway node. This is ideal for rural deployments and disaster scenarios where dedicated hardware is scarce.

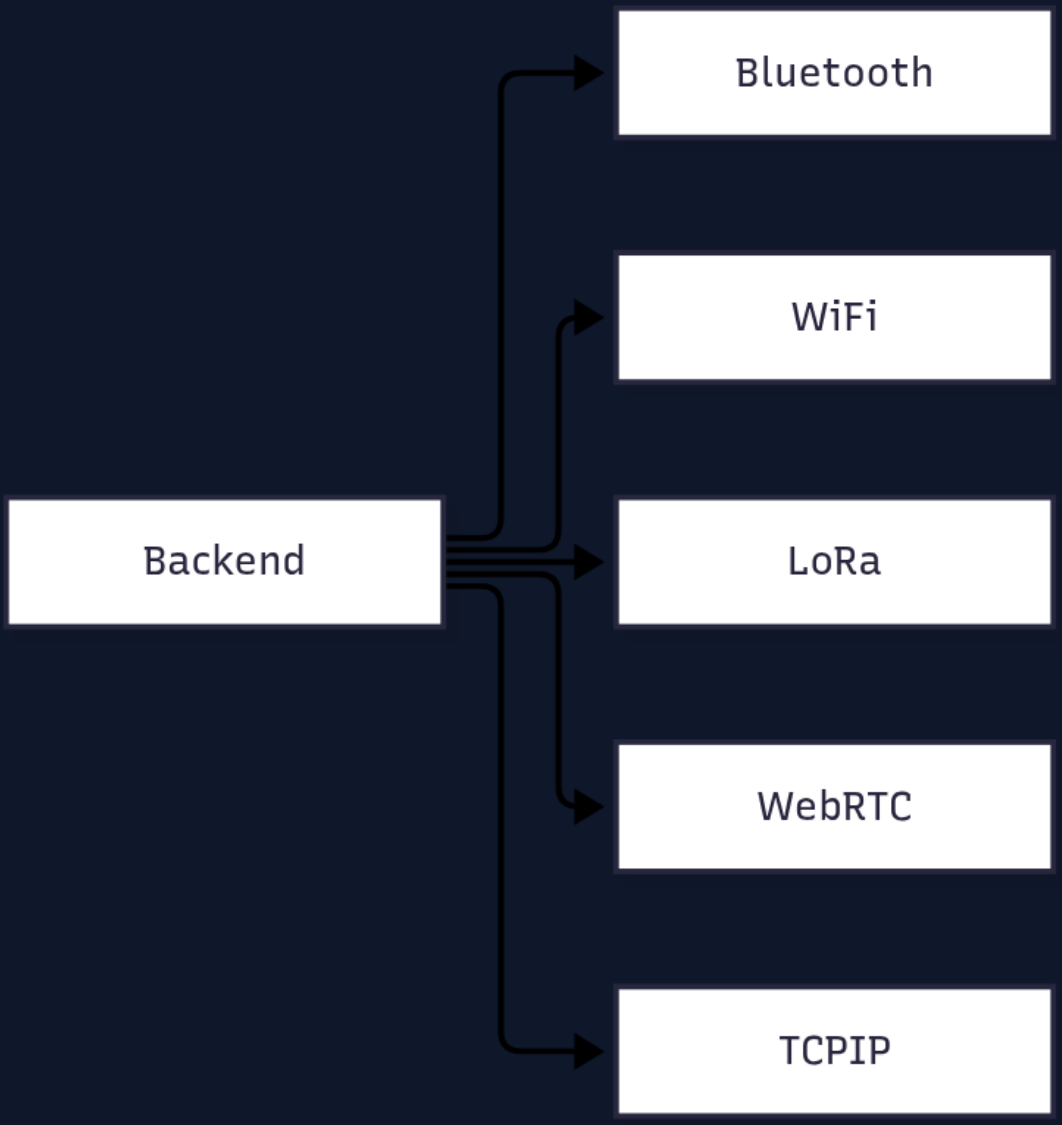
Mailbox Pattern for Leaf Devices

Leaf devices that cannot maintain persistent connections use a mailbox pattern. Messages destined for a leaf are cached at the leaf's last-known relay node. When the leaf reconnects, it pulls all queued messages in batch. This decouples availability from connectivity.

---

5. Transports Layer

- **Working today:** stealth TCP, WebRTC, Reticulum (feature-flagged) — plus QUIC implemented but not yet on the mesh handshake. WiFi Direct is a platform boundary with no OS provider installed; LoRa is not implemented; Bluetooth LE is a skeleton. See docs/BIBLE_STATUS_SPINE.md §4.
- **In progress:** Bluetooth LE
- **Pluggable:** Add new transports by implementing the `Transport` trait.
- **Runtime Selection:** Enable/disable transports via config or UI.

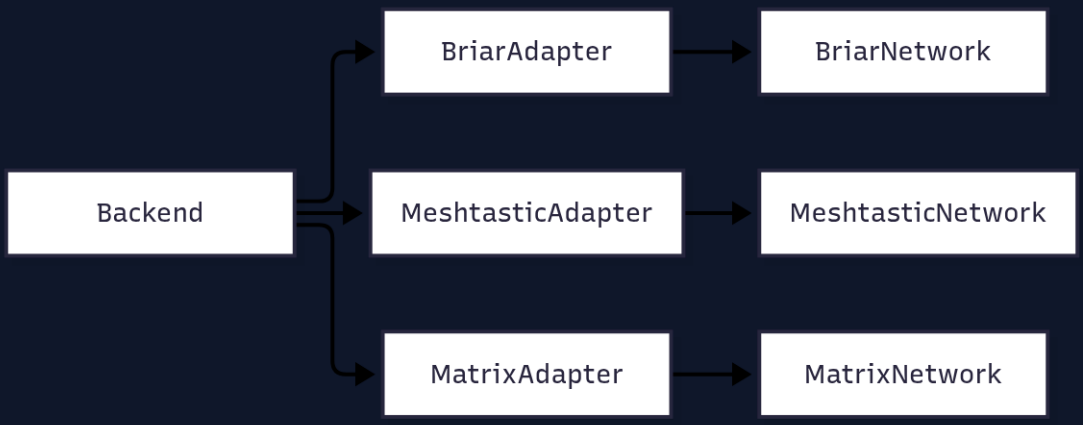


- **Visual:**
- **Best Practice:** Use multiple transports for resilience.

---

6. Protocol Adapters Layer

- **Purpose:** Bridge GhostWire to other networks (Matrix, Meshtastic, Briar, etc.)
- **How:** Implement the `ProtocolAdapter` trait.
- **Features:** Message translation, deduplication, relay, group chat, file sharing.



- **Visual:**

---

7. AI Routing Layer

The AI Routing Layer applies machine learning to decide how messages are classified, routed, and prioritized across the mesh. It operates across four distinct layers, each with a different scope and cadence.

Layer 1: LightGBM Per-Message Classification

The design calls for a LightGBM classifier to assign each message a priority category, with features including message size, destination type, transport metrics, and battery level, so that routine, priority, and emergency messages can be routed differently and metered differently. **This layer is not active in the current build** — no model file is present in the repository and the classifier is not called from routing decisions (see docs/BIBLE_STATUS_SPINE.md §5). Priority handling today comes from the message type on the wire, not from an ML classifier.

Layer 2: Gemma 4 Analytical Pass

Every 30 seconds, a Gemma 4 model runs an analytical pass over recent mesh activity. This pass identifies traffic patterns, anomalies, and potential congestion points. The output feeds into routing table adjustments and supernode election decisions. This is a periodic batch process, not a per-message classifier.

Layer 3: GNN Graph Router

A Graph Neural Network models the mesh as a dynamic graph where nodes are devices and edges are transport links with associated latency, reliability, and energy cost. The GNN computes optimal routing paths in real time, adapting to topology changes. This layer handles the heavy lifting of pathfinding across a volatile mesh.

Layer 4: PRoPHET Delay-Tolerant Routing

For partitions and disconnected scenarios, PRoPHET (Probabilistic Routing Protocol Using History of Encounters and Transitivity) provides delay-tolerant message forwarding. Messages are carried by nodes that are likely to encounter the destination in the future. This is essential for disaster response and off-grid scenarios where continuous connectivity cannot be guaranteed.

Model Ladder

Models are deployed across the hardware ladder based on capability: - **E2B (Edge-to-Base):** Runs on edge devices (phones, Raspberry Pi) for lightweight classification. - **E4B (Edge-to-Cloud):** Runs on relay laptops for analytical passes and local graph inference. - **26B MoE (Mixture of Experts) Coordinator:** Runs on a coordinator node to aggregate routing decisions across the mesh. - **31B Server Model:** Runs on server-class machines for full analytical depth and training.

Telemetry and Model Distribution

Telemetry flows up (opt-in only) so that model training data is collected with explicit user consent. Model files flow down via DHT hash verification and P2P sync, not federated learning. Each node verifies model integrity through content hashes distributed over the DHT before loading a new model version.

Store-and-Forward DTN Integration

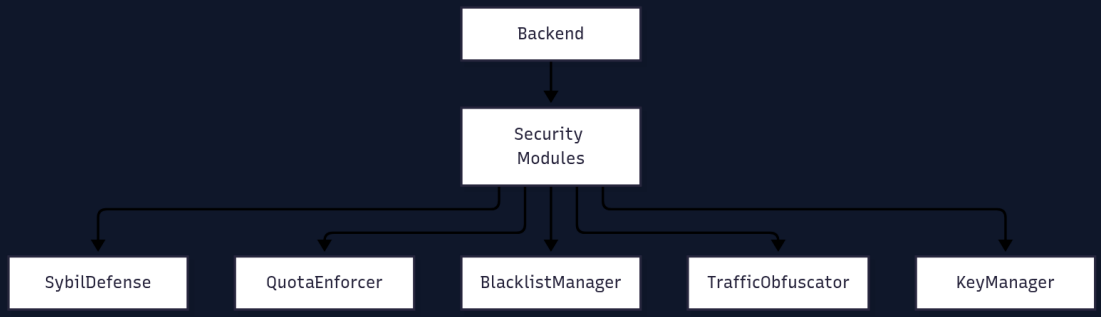
The AI Routing Layer integrates with store-and-forward DTN protocols (PRoPHET, Spray-and-Wait) to make intelligent decisions about when to hold messages versus when to forward them. In partitioned networks, the AI predicts which relay nodes are most likely to reach the destination and queues messages accordingly.

---

8. Security & Trust Layer

- **Modules:**
  - SybilDefense
  - QuotaEnforcer
  - BlacklistManager
  - TrafficObfuscator
  - KeyManager
- **Features:**
  - End-to-end encryption (XChaCha20-Poly1305, X25519, hybrid ML-KEM-768)
  - Perfect forward secrecy
  - Ephemeral keys, key rotation
  - Quotas and rate limiting
  - Blacklisting and abuse prevention
  - Traffic obfuscation and anti-analysis
  - ML-KEM-768 post-quantum key encapsulation (10.5us measured, x86_64)
  - SPHINX SURBs and cover traffic for metadata protection
  - Privacy tiers: L0 (minimal), L1 (standard), L2 (enhanced), L3 (maximum)
- **Privacy Tiers by Device:**
  - Server: L3 (maximum privacy)
  - Laptop and Tablet: L2
  - Phone and Raspberry Pi: L1 (drops one tier when battery is 20% or below)

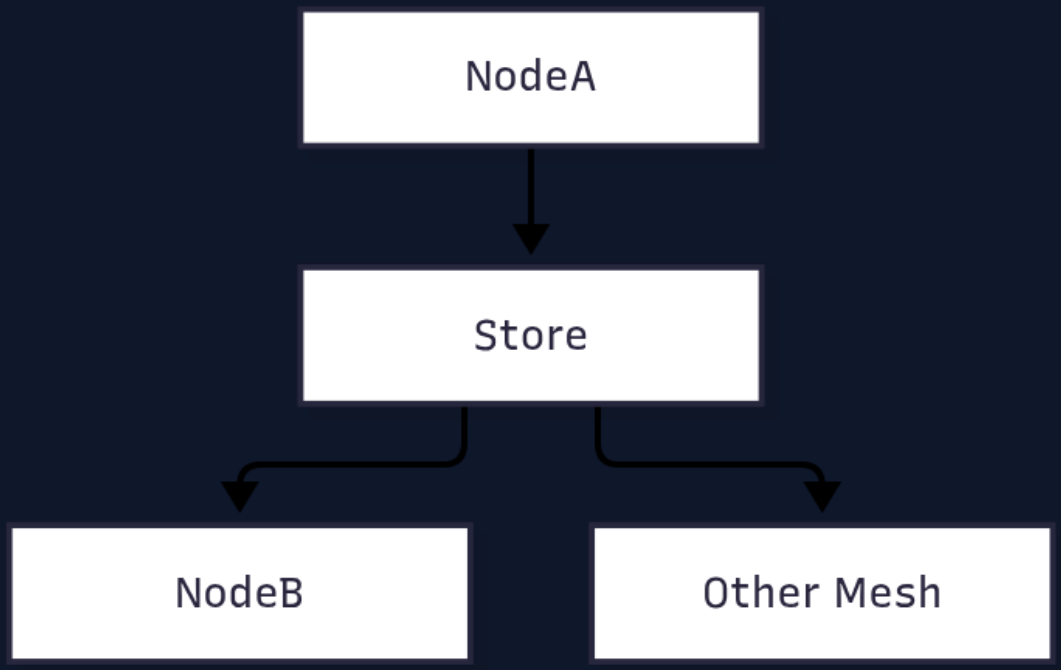
- **Visual:**



---

9. Store & Forward / Federation

- **Store & Forward:** Messages are cached and relayed when possible. DTN protocols (PRoPHET, Spray-and-Wait) handle disconnected scenarios.
- **Federation:** Meshes can connect to each other for wider reach.



- **Visual:**

---

10. Data Flow: Message Lifecycle

1. **User sends message via UI/CLI**
2. **Backend encrypts and signs message**
3. **AI Routing Layer classifies and selects route**
4. **Message routed via best available transport(s)**
5. **Adapters translate if needed**
6. **Security modules enforce quotas, check blacklists**
7. **Message hops node-to-node (store & forward as needed)**
8. **Recipient decrypts and reads message**

11. Deployment Blueprints

Home/Personal Mesh

- Single device or small group, WiFi Direct/Bluetooth
- Simple setup, auto-discovery
- Phone as leaf, laptop as relay

Community/Neighborhood Mesh

- Dozens of nodes, mix of WiFi Direct, LoRa, WebRTC
- Some nodes act as relays and supernodes
- Store & forward for offline delivery
- Raspberry Pi gateways bridge ESP32 sensors

Disaster/Field Deployment

- LoRa nodes at key locations
- Battery/solar-powered relays
- Store & forward, federation with other meshes
- PRoPHET routing for partitioned scenarios
- Privacy tier drops to L0/L1 on low battery

Enterprise/Federated Mesh

- Multiple sites, federation, quotas, advanced security
- Integration with existing systems via adapters
- Server-class nodes as Replicators and supernodes
- Matrix bridge for external reach

12. Best Practices & Anti-Patterns

Best Practices

- Use multiple transports for resilience
- Enable all relevant security modules
- Regularly update software
- Monitor mesh health and logs
- Educate users on privacy and security
- Use the AI Routing Layer to optimize message priority
- Verify model integrity via DHT hashes before loading

Anti-Patterns

- Relying on a single transport
- Disabling security features
- Ignoring updates or logs

- Running relays without adequate uptime guarantees
- Assuming relays have trust authority (they do not)

---

13. Glossary & Reference

- **Node:** Any device running GhostWire
- **Transport:** WiFi Direct, LoRa, stealth TCP, WebRTC, Reticulum
- **Adapter:** Bridge to other protocols
- **Supernode:** A dynamically elected relay hub with high uptime and bandwidth
- **Sybil Attack:** One entity pretends to be many nodes
- **Quota:** Limit on messages/actions
- **Federation:** Connecting multiple meshes
- **DTN:** Delay-Tolerant Networking, for disconnected scenarios
- **SPHINX:** Source-routed onion protocol for metadata protection
- **Privacy Tier:** L0-L3, controlling the level of traffic obfuscation and cover traffic

---

End of Chapter

Transports & Protocols

Table of Contents

1. What is a Transport?
2. Supported & Planned Transports
3. Traffic Armor and Auto-Switching
4. Transport Selection: Thompson Sampling
5. How Transports Work (Plain & Technical)
6. Which Transport When
7. Real-World Use Cases
8. Configuration & Code Examples
9. Visual Guide: Transport Topologies
10. Best Practices & Anti-Patterns
11. Troubleshooting & FAQ
12. Further Reading & Resources

---

1. What is a Transport?

- **Plain:** A transport is the "road" your messages travel on: WiFi Direct, LoRa, stealth TCP, WebRTC, and more.
- **Technical:** In GhostWire, each transport is a pluggable module implementing the `Transport` trait, allowing for runtime or compile-time enable/disable.

---

2. Supported & Planned Transports

Status verified 2026-09-28 against `rescue/sep-disk-tree @ e4028f2`. Evidence and file:line references: docs/BIBLE_STATUS_SPINE.md §4. This table is generated from that file — if you edit one, edit the other.

Transport	Status	Use Case / Notes
stealth TCP	**Working**	Evades DPI, auto-escalates when blocked. 615 lines, wired at `transport.rs:12`
WebRTC	**Working**	Browser-to-browser, disguises as video call traffic. Real `webrtc::tokio::Transport` in the swarm at `mesh.rs:262-277`
Reticulum	**Working** (feature-flagged)	Encapsulated transport, 883 lines, wired at `transport.rs:17`. Enable with `reticulum = true`
QUIC	Implemented, not on mesh handshake	Present in `TransportKind`; downgrade refusal wired into QUIC only
Bluetooth LE	**Skeleton**	63 lines. `send_ble_data` logs "Skeleton" and returns `Ok(())`; `stop_scanning` is a no-op; `/ble/*` routes commented out at `web.rs:552-554`
WiFi Direct	**Platform boundary only**	`medium_adapter.rs:111-130` returns `unsupported_snapshot` — "OS provider is not installed"
LoRa	**Not implemented**	No LoRa transport exists in the codebase. `disaster.rs:65` defines a `LoRa` variant used only as a selector weight. `TODO.md:186`: "Protocol is defined but no hardware integration"

---

3. Traffic Armor and Auto-Switching

GhostWire's Traffic Armor makes your mesh traffic look like ordinary web traffic so it is harder to identify, block, or throttle.

- **WebRTC Disguise:** WebRTC traffic is shaped to look like a Discord-style voice or video call. Deep packet inspection cannot easily distinguish it from legitimate real-time communication.
- **stealth TCP:** Evades DPI (Deep Packet Inspection) by obfuscating packet headers and payload patterns. When a network blocks stealth TCP, the transport auto-escalates: it falls back to WebRTC or Reticulum, which use different obfuscation strategies.
- **Auto-Switching:** If one transport is blocked or degraded, GhostWire automatically tries alternatives. The selection is not random; it is guided by the Thompson Sampling algorithm described below.

---

4. Transport Selection: Thompson Sampling

GhostWire uses Thompson Sampling, a bandit algorithm, to balance reliability, latency, and energy efficiency when choosing which transport to use for each message. Each transport is modeled as a slot machine with a probability distribution over its expected quality. The algorithm explores transports that are uncertain and exploits those that have proven reliable.

Energy-efficiency weights guide the exploration-exploitation tradeoff:

Transport	Energy Efficiency Weight
Bluetooth LE	0.30
LoRa	0.35
Reticulum	0.25
WiFi Direct	0.05
TCP/QUIC	0.00

Higher weights mean the transport is preferred when energy is the dominant concern. On battery-powered devices, the algorithm favors LoRa and Bluetooth LE. On plugged-in devices, WiFi Direct and WebRTC get higher priority because they offer lower latency and higher bandwidth.

---

5. How Transports Work (Plain & Technical)

Plain-Language

- Devices use whatever "roads" are available to connect: WiFi Direct for local mesh, LoRa for long distance, WebRTC for browser-based communication.
- GhostWire automatically picks the best available transport, or you can choose manually.

Technical

- Each transport implements the `Transport` trait in Rust: `rust pub trait Transport { fn send(&self, msg: Message) -> Result<(), TransportError>; fn receive(&self) -> Option<Message>; fn is_available(&self) -> bool; // ... }`
- Transports can be enabled/disabled at runtime via config or UI.
- Multiple transports can be active at once for resilience.

---

6. Which Transport When

Scenario	Recommended Transport(s)	Why
Indoor messaging, low latency	WiFi Direct	High bandwidth, low latency, no external hardware needed
Rural area, no internet	LoRa	Long-range, low-power, works without infrastructure
Censored or monitored network	stealth TCP, WebRTC	Evades DPI, disguises as ordinary traffic
Browser-based peer-to-peer	WebRTC	No plugins, NAT traversal built in
Maximum anonymity, obfuscation	Reticulum	Encapsulated, feature-flagged for advanced users
Mobile, battery-constrained	Bluetooth LE (when available)	Very low energy, short range
Mixed environment	Multiple transports active	Thompson Sampling picks the best option per message

---

7. Real-World Use Cases

Disaster Response

- **Scenario:** Power and cell towers are down after a hurricane.
- **Solution:** LoRa nodes relay messages across miles; WiFi Direct and WebRTC fill in gaps.

Urban Mesh

- **Scenario:** Protesters need secure, local communication.
- **Solution:** Phones use WiFi Direct and WebRTC to form a dense, resilient mesh. stealth TCP evades network monitoring.

Rural Connectivity

- **Scenario:** Villages with no internet need to share news.
- **Solution:** LoRa radios connect homes and farms over long distances.

Connectivity in Constrained Environments

- **Scenario:** Authorities restrict internet access or monitor traffic.
- **Solution:** stealth TCP and WebRTC provide obfuscated, hard-to-block channels. Traffic Armor disguises the traffic as ordinary video-call or web traffic.

---

8. Configuration & Code Examples

Enabling/Disabling Transports (Config)

```
[transports]
bluetooth = false
wifi = true
lora = true
webrtc = true
tcpip = true
stealth_tcp = true
# reticulum = true  # Uncomment to enable Reticulum (feature-flagged)
```

Using Transports in Code

```
let wifi = WifiDirectTransport::new();
let lora = LoRaTransport::new();
backend.add_transport(Box::new(wifi));
backend.add_transport(Box::new(lora));
```

Web UI Example

- Go to Settings > Transports
- Toggle available transports on/off
- See real-time status and diagnostics

---

9. Visual Guide: Transport Topologies

---

10. Best Practices & Anti-Patterns

Best Practices

- Enable multiple transports for best coverage.
- Test transport availability before deployment.
- Use LoRa for long-range, low-power needs.
- Use WebRTC or stealth TCP for connectivity in constrained environments.



Diagram 4: transports

- Monitor transport health in the UI.
- Let Thompson Sampling manage transport selection when you are unsure.

Anti-Patterns

- Relying on a single transport.
- Disabling security features on transports.
- Ignoring hardware compatibility.
- Forcing a transport when Thompson Sampling would choose a better option.

---

11. Troubleshooting & FAQ

Problem	Solution
Can't connect via Bluetooth LE	Check permissions, try WiFi Direct or LoRa
LoRa not working	Check hardware, drivers, and range
WebRTC fails	Check firewall/NAT, try TCP/IP or stealth TCP fallback
stealth TCP blocked	Auto-escalation should switch to WebRTC or Reticulum; check if feature flag is enabled
Slow performance	Use higher-bandwidth transport if possible
Security warning	Ensure encryption is enabled on all transports

---

12. Further Reading & Resources

- LoRa Alliance
- Bluetooth Mesh
- WebRTC
- Mesh Networking 101
- GhostWire Developer Guide

End of Chapter

Security & Privacy

Table of Contents

1. Executive Summary
2. Security Foundations (Plain & Technical)
3. Threat Models & Real-World Risks
4. Security Architecture & Modules
5. Encryption & Key Management
6. Sybil Defense & Trust

7. Quotas, Blacklists, and Abuse Prevention
8. Traffic Obfuscation & Anti-Analysis
9. Disaster & Infrastructure Failure Scenarios
10. Best Practices & Anti-Patterns
11. Actionable Security Checklists
12. Visuals: Security Layers & Flows
13. FAQ & Troubleshooting
14. Further Reading & Resources

---

1. Executive Summary

GhostWire is built with security and privacy as core principles. This chapter explains how GhostWire protects users, what threats it defends against, and how to use its security features, whether you are a non-technical user or a security engineer.

---

2. Security Foundations (Plain & Technical)

- **Plain:** GhostWire keeps your messages private and your identity safe, even if someone tries to spy or block you.
- **Technical:** End-to-end encryption (XChaCha20-Poly1305, X25519), perfect forward secrecy, ephemeral keys, key rotation, secure storage, and **hybrid post-quantum key establishment (ML-KEM-768) — implemented and tested**, not planned.

---

3. Threat Models & Real-World Risks

- **Connectivity Loss:** Governments, ISPs, or natural disasters causing communication blackouts.
- **Surveillance:** Adversaries trying to read or analyze messages.
- **Sybil Attacks:** Fake nodes trying to disrupt or spy on the mesh.
- **Denial of Service:** Flooding the network to disrupt communication.
- **Traffic Analysis:** Inferring who is talking to whom, even if messages are encrypted.
- **Device Seizure:** Physical access to a device running GhostWire.

---

4. Security Architecture & Modules

- **SybilDefense:** Prevents fake nodes from overwhelming the mesh.
- **QuotaEnforcer:** Limits message rates to prevent spam/DoS.
- **BlacklistManager:** Blocks known abusers or compromised nodes.
- **TrafficObfuscator:** Makes traffic patterns harder to analyze.
- **KeyManager:** Handles encryption keys, rotation, and secure storage.

---

5. Encryption & Key Management

- **End-to-end encryption:** All messages are encrypted from sender to recipient.

- **Key exchange:** X25519 for secure, ephemeral key exchange.
- **Key rotation:** Regularly rotates keys for forward secrecy.
- **Secure storage:** Keys are stored encrypted on disk.
- **Post-quantum:** Research and planning for future upgrades.

---

6. Modern Crypto & Identity Stack

GhostWire deploys a layered cryptographic and identity system designed for both current threats and future risks.

Post-Quantum Key Encapsulation

- **ML-KEM-768:** GhostWire integrates ML-KEM-768 (Module-Lattice-Based Key-Encapsulation Mechanism at the 768-bit security level) for post-quantum key establishment, combined with classical X25519 in a hybrid handshake. Measured encapsulation is **10.5 microseconds** on x86_64 development hardware (`cargo bench --bench hot_paths -- mlkem768`; keygen 10.5 us, decapsulation 11.2 us). Constrained embedded hardware will be slower — these figures have not been measured on a Raspberry Pi-class device, and should not be quoted as on-device numbers until they are.
- **Hybrid mode:** ML-KEM-768 is combined with X25519 during the transition period, so a session retains security even if one algorithm is later broken.

Symmetric Encryption

- **XChaCha20-Poly1305:** The message cipher. Provides authenticated encryption with associated data (AEAD), ensuring both confidentiality and integrity. It uses a 192-bit nonce, which removes the nonce-reuse risk that affects standard ChaCha20, and performs well on devices without AES hardware acceleration.

  > **Correction:** earlier revisions of this chapter described **AES-256-GCM** as the default cipher with XChaCha20 as a fallback. The codebase uses XChaCha20-Poly1305 throughout (`core/encryption.rs`); there is no AES-GCM on the message path.

- **Ed25519 / X25519:** Ed25519 signs and authenticates identities; X25519 provides the classical half of the key exchange, which is then mixed with the ML-KEM-768 shared secret via HKDF to form a 1,248-byte hybrid key. Hybrid key exchange means a session stays secure even if either the classical or the post-quantum assumption is later broken.

Key Exchange and Forward Secrecy

- **X25519:** Elliptic-curve Diffie-Hellman key exchange on the Curve25519 group. Provides fast, constant-time key agreement.
- **Perfect Forward Secrecy (PFS):** Every session generates ephemeral key pairs. Compromise of a long-term key does not expose past session traffic. Keys are rotated after each session and at defined intervals during active conversations.

Duress PIN and Coercion Resistance

- **Duress PIN:** Users can configure a secondary PIN that, when entered under coercion, unlocks a plausible-deniability mode. The device presents a harmless or empty view of the mesh while the real identity and keys remain cryptographically hidden.
- **Plausible deniability under seizure:** If a device is seized, the duress PIN mode ensures that an adversary cannot distinguish whether real data exists on the device. Encrypted volumes and key material remain inaccessible without the primary credentials.

Identity: DIDs, Shamir Secret Sharing, Dead Man's Switch

- **Decentralized Identifiers (DIDs):** GhostWire assigns each node a DID that is not tied to any central authority. DIDs enable verifiable, self-sovereign identity across meshes without relying on a directory server.
- **Shamir Secret Sharing:** Recovery keys are split into shares distributed among trusted contacts. A threshold number of shares is required to reconstruct the master key, preventing single-point failure and single-point compromise.
- **Dead Man's Switch:** If a node goes silent beyond a configured timeout, the dead man's switch triggers and can notify trusted contacts or publish a revocation certificate. The timer logic is implemented, but **the wipe itself is not performed by the current code** — `panic.rs:151-155` explicitly leaves it to the caller, and `execute_panic` has no production call site. Treat key wiping as a planned integration, not a working feature.
- **Device Pairing:** New devices pair via a secure out-of-band channel (QR code scan, near-field broadcast, or in-person verification). Pairing establishes a shared secret without exposing it to network observers.
- **KERI (Key Event Receipt Infrastructure):** Planned for future integration. KERI provides a fully decentralized, append-only log of key events, enabling trust anchoring without certificate authorities.

Privacy Tiers (L0-L3)

GhostWire enforces privacy levels that adapt to device capabilities and battery state:

Tier	Device Type	Metadata Exposure	Cover Traffic
L3	Server	Minimal	Full
L2	Laptop, Tablet	Reduced	Partial
L1	Phone, Raspberry Pi	Basic	Minimal
L0	Any device (battery critical)	Maximum	None

- **Server nodes** operate at L3 with full cover traffic and minimal metadata leakage.
- **Laptops and tablets** operate at L2 with reduced metadata and partial cover traffic.
- **Phones and Raspberry Pi** operate at L1 with basic protections.
- **When battery drops below 20%,** the device automatically drops one privacy tier to conserve energy.

SPHINX Unlinkable Replies and Cover Traffic

- **SPHINX (Scalable Hybrid Incremental N-way Exchange):** GhostWire uses SPHINX-style onion routing for replies. Each message is wrapped in multiple layers of encryption so that intermediate

nodes can only decrypt the layer intended for them. No single node knows both the sender and the ultimate recipient.
- **Unlinkable Replies (SURBs):** Single-Use Reply Blocks allow a recipient to reply without revealing their identity to the sender or any intermediary. Each SURB is consumed after one use.
- **Cover traffic:** Nodes generate dummy packets at regular intervals to mask the timing and volume of real messages. This defeats traffic analysis that relies on observing communication patterns.

---

7. Sybil Defense & Trust

- **Proof-of-Work/Stake:** Optional modules to make Sybil attacks expensive.
- **Reputation:** Nodes can build trust over time.
- **Manual approval:** Option for closed/curated meshes.

---

8. Quotas, Blacklists, and Abuse Prevention

- **Quotas:** Rate limits on messages, connections, and actions.
- **Blacklists:** Block known abusers or compromised nodes.
- **Automated & manual controls:** Admins can adjust settings in real time.

---

9. Traffic Obfuscation & Anti-Analysis

- **Padding:** Adds random data to messages to hide true size.
- **Timing obfuscation:** Randomizes message timing to prevent correlation.
- **Stealth transports:** Use WebRTC, Stealth TCP, or other obfuscated channels.
- **Traffic Armor:** WebRTC traffic is disguised to look like a video call. Stealth TCP automatically escalates to alternative protocols when blocked.

---

10. Disaster & Infrastructure Failure Scenarios

- **Disaster mode:** Store-and-forward, minimal metadata, offline queuing.
- **Connectivity resilience:** Privacy-enhanced transports, traffic obfuscation, rapid key rotation.

---

11. Best Practices & Anti-Patterns

Best Practices

- Always use the latest version.
- Enable all security modules.
- Use strong passwords and device security.
- Educate users about privacy features. ### Anti-Patterns
- Disabling encryption or security modules.
- Using default passwords.
- Ignoring updates or logs.

12. Actionable Security Checklists

- ☐ Update software regularly
- ☐ Enable all security modules
- ☐ Use strong, unique passwords
- ☐ Monitor mesh health and logs
- ☐ Educate users on privacy and security
- ☐ Verify ML-KEM-768 is enabled on supported nodes
- ☐ Confirm perfect forward secrecy is active on all sessions
- ☐ Test duress PIN mode before deployment
- ☐ Configure dead man's switch timeout
- ☐ Distribute Shamir secret sharing shares to trusted contacts

13. Visuals: Security Layers & Flows

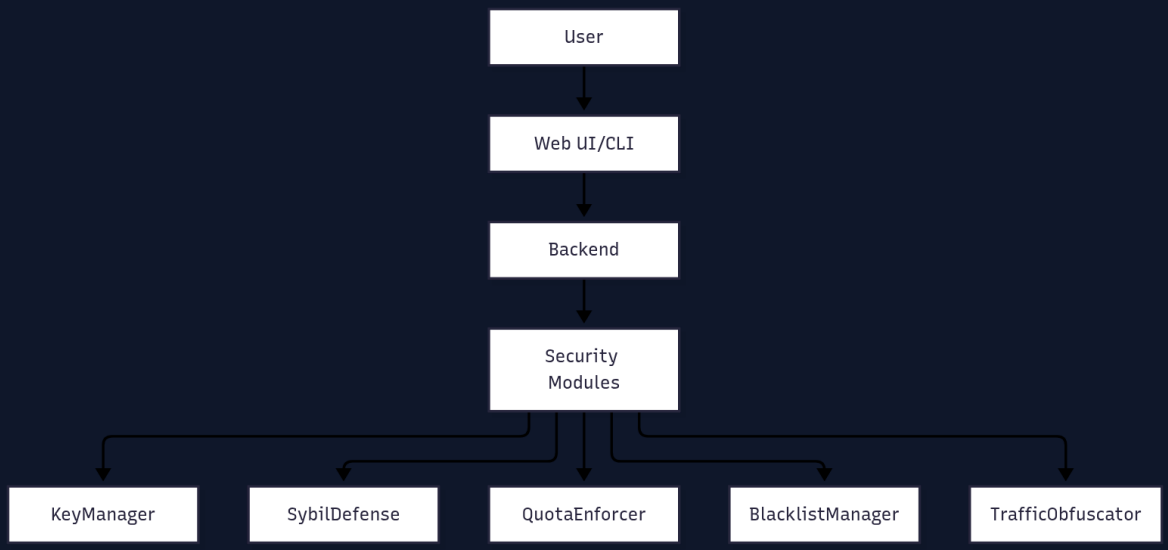


Diagram 5: security

14. FAQ & Troubleshooting

- **Q: How do I know my messages are secure?**
  - All messages are end-to-end encrypted by default.
- **Q: What if my device is seized?**
  - Keys are encrypted on disk; use device encryption for extra safety. Enable duress PIN mode for plausible deniability.
- **Q: Can I disable security features?**
  - Not recommended; only for advanced users in test environments.
- **Q: What is ML-KEM-768?**
  - A post-quantum key encapsulation mechanism that protects against future quantum computer

attacks. Measured at 10.5 microseconds on x86_64 development hardware; constrained-device figures are not yet measured.

- **Q: How does the duress PIN work?**
  - Entering the duress PIN activates a plausible-deniability mode that hides real data from coercive actors.

---

15. External Audit Status

An external security audit has NOT been performed. GhostWire's current protections are self-reviewed engineering controls. The development team has conducted internal reviews of the cryptographic implementations, module interactions, and threat model coverage.

External audit is PLANNED. Candidate auditing firms include Cure53, NCC Group, and Least Authority. No engagement has been confirmed and no audit has commenced as of September 2026.

Users and organizations deploying GhostWire should understand that: - All security claims in this document reflect self-assessed engineering confidence, not third-party verification. - The cryptographic designs (ML-KEM-768, XChaCha20-Poly1305, X25519 with PFS) are based on well-studied, published algorithms, but their integration into GhostWire has not been independently audited. - Organizations requiring certified security assurance should plan for an external audit before deploying GhostWire in production environments handling sensitive data.

---

16. Further Reading & Resources

- EFF Surveillance Self-Defense
- OWASP Top 10
- GhostWire Advanced Security
- NIST Post-Quantum Cryptography Standards
- Signal Protocol Design

End of Chapter

Protocol Adapters

Table of Contents

1. What is a Protocol Adapter?
2. Supported & Planned Adapters
3. How Adapters Work (Plain & Technical)
4. The Adapter Registry
5. Real-World Bridging Scenarios
6. Configuration & Code Examples
7. Visual Guide: Adapter Topologies
8. Companion Projects

9. Developer Notes & API Reference
10. Best Practices & Anti-Patterns
11. Troubleshooting & FAQ
12. Further Reading & Resources

---

1. What is a Protocol Adapter?

- **Plain:** A protocol adapter is like a translator that lets GhostWire talk to other messaging networks (like Matrix, Meshtastic, Briar).
- **Technical:** Adapters are software modules that translate messages and events between GhostWire and other protocols, enabling cross-network messaging, group chat, and file sharing.

---

2. Supported & Planned Adapters

Adapter	Status	Notes / Features
Matrix	Planned	No matrix bridge module exists in the codebase. A `matrix-bridge` feature name appears in older notes; there is no implementation behind it.
Meshtastic	Planned	No meshtastic module exists. Erasure coding (3 data + 2 parity shards) is implemented for LoRa framing, but there is no bridge — and LoRa itself is not implemented.
Briar	Planned	Contact-based messaging, offline queuing, groups
Reticulum bridge	Planned/experimental	Feature `reticulum`; experimental bridge to Reticulum mesh
Tor	Not implemented	No Tor bridge exists yet
Custom	Supported	Build your own adapter via the adapter registry

---

3. How Adapters Work (Plain & Technical)

Plain-Language

- Adapters "translate" messages so GhostWire can talk to other networks.
- You can bridge a GhostWire mesh to Matrix, Meshtastic, or Briar, sharing messages and files.

Technical

- Each adapter implements the `ProtocolAdapter` trait in Rust: `rust pub trait ProtocolAdapter { fn send(&self, msg: Message) -> Result<(), AdapterError>; fn receive(&self) -> Option<Message>; fn connect(&self) -> Result<(), AdapterError>; // ... }`
- Adapters handle translation, deduplication, relay, and group management.
- **Note on Briar:** Internal structs inspired by Briar exist in the codebase, but a real Briar bridge has not been built yet. The "Briar" adapter remains in the Planned state.

---

4. The Adapter Registry

GhostWire uses an adapter registry to manage ProtocolAdapter instances at runtime. To add a new bridge adapter:

1. Implement the `ProtocolAdapter` trait for your adapter.
2. Register the adapter at startup via the registry.
3. The backend will load and initialize the adapter automatically.
4. Test both directions: GhostWire-to-external and external-to-GhostWire.

The registry is the single entry point for all adapters. It ensures that adapters are initialized in the correct order, that dependencies are resolved, and that failed adapters do not crash the backend. See the Matrix and Meshtastic adapters as reference implementations.

---

5. Real-World Bridging Scenarios

Disaster Response

- **Scenario:** GhostWire mesh bridges to Meshtastic LoRa radios for long-range communication.
- **Outcome:** First responders can relay messages between phone users and LoRa devices.
- **Quotable:** "GhostWire to Meshtastic lets a phone talk over LoRa through a nearby relay."

Activist Network

- **Scenario:** Protesters use GhostWire to bridge to Matrix for global reach.
- **Outcome:** Local mesh messages are relayed to Matrix rooms, connecting to the outside world.

Rural Community

- **Scenario:** GhostWire connects to Briar for secure, contact-based messaging.
- **Outcome:** Villagers can chat securely, even offline. (Note: Briar bridge is Planned, not yet implemented.)

---

6. Configuration & Code Examples

Enabling/Disabling Adapters (Config)

```
[adapters]
briar = false
meshtastic = true
matrix = true  # Requires the matrix-bridge feature flag
custom = true
```

Using Adapters in Code

```
let matrix = MatrixAdapter::new();
let meshtastic = MeshtasticAdapter::new();
```

```
backend.add_adapter(Box::new(matrix));
backend.add_adapter(Box::new(meshtastic));
```

Web UI Example

- Go to Settings > Adapters
- Toggle available adapters on/off
- See real-time status and diagnostics

---

7. Visual Guide: Adapter Topologies

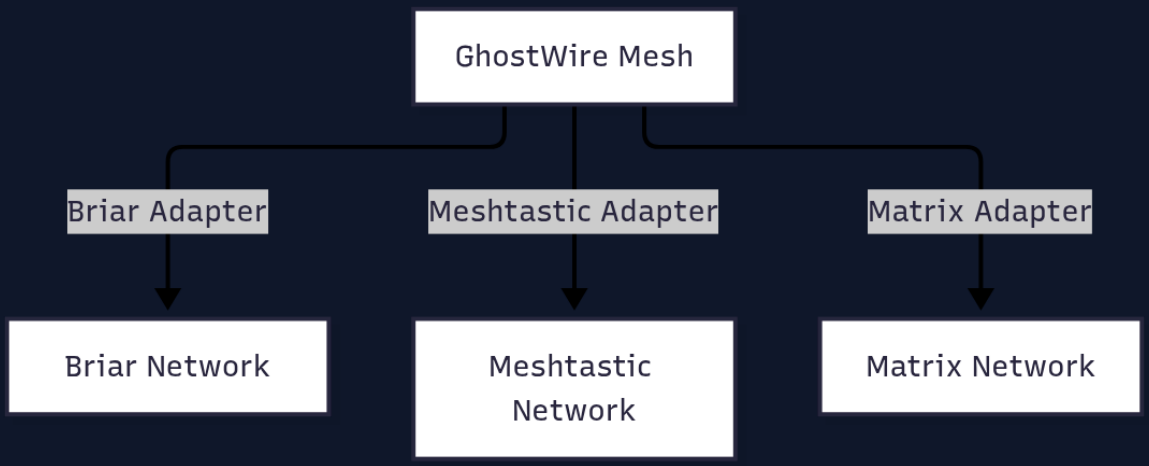


Diagram 6: adapters

---

8. Companion Projects

Two companion projects extend the adapter ecosystem:

- **ghostwire-dtn:** A delay-tolerant networking crate that provides store-and-forward capabilities for adapter-based bridges. Useful when the target network does not support real-time messaging.
- **sphinx-rs:** A Rust implementation of the SPHINX onion-routing protocol, used for metadata protection across adapter bridges.

---

9. Developer Notes & API Reference

- **Implementing a new adapter:**
  - Implement the `ProtocolAdapter` trait.
  - Handle message translation, deduplication, and relay.
  - Register the adapter in the backend via the adapter registry.
  - Test both directions (GhostWire-to-external and external-to-GhostWire).
- **Reference implementations:** The Reticulum transport (`transport.rs:17`, 883 lines) is the one real interop in the codebase. Matrix and Meshtastic adapters are not implemented — see the

status table above.
- **API Reference:**
  - See Developer Guide for full trait and API docs.

---

10. Best Practices & Anti-Patterns

Best Practices

- Test adapters in isolated environments before production.
- Keep adapters updated for protocol changes.
- Monitor adapter health and logs.
- Test both directions of message flow.
- Use the adapter registry for all adapter registration.

Anti-Patterns

- Relying on a single adapter for all bridging.
- Disabling security features on adapters.
- Ignoring protocol updates.
- Registering adapters outside the registry.

---

11. Troubleshooting & FAQ

Problem	Solution
Can't connect to Matrix	Check credentials, server URL, and network; verify `matrix-bridge` feature flag is enabled
Briar messages not relayed	Briar bridge is Planned; not yet implemented
Meshtastic bridge fails	Check LoRa hardware and adapter config; verify erasure coding parameters
Duplicate messages	Ensure deduplication is enabled
Adapter not loading	Check that the adapter is registered via the adapter registry

---

12. Further Reading & Resources

- Matrix Protocol
- Briar Project
- Meshtastic
- GhostWire Developer Guide
- ghostwire-dtn
- sphinx-rs

End of Chapter

Frequently Asked Questions (FAQ)

Table of Contents

1. General Questions
2. Getting Started
3. Technical & Developer Questions
4. Security & Privacy
5. Real-World Scenarios
6. Troubleshooting & Support
7. Visuals: Common Flows
8. Further Reading & Resources

---

1. General Questions

Q: What is GhostWire? A: A modular, privacy-focused mesh networking and messaging platform supporting multiple transports and advanced security.

Q: Who can use GhostWire? A: Anyone. It is designed for activists, disaster responders, rural communities, privacy enthusiasts, and developers.

Q: Is GhostWire free and open-source? A: Yes. The code is on GitHub.

Q: What platforms are supported? A: Linux, Windows, and macOS. Mobile platform support is under active development.

Q: Who is GhostWire for? A: GhostWire serves six main groups: disaster responders who need to coordinate when cell towers fail, privacy advocates who want no logs and no traces, rural communities where 2.6 billion people lack reliable internet, activists and journalists facing internet shutdowns, whistleblowers who need ephemeral sessions and stealth communication, and developers and researchers who want an AGPLv3 Rust codebase to study and build on.

---

2. Getting Started

Q: How do I install and run GhostWire? A: See the Getting Started chapter. You will need Rust, Node.js, and npm for the full stack, or you can try the web demo if available.

Q: Do I need to be a developer? A: No. There are easy-to-use web and mobile interfaces planned for future releases.

Q: Can I join a mesh without the internet? A: Yes. Use Bluetooth, WiFi, or LoRa to connect locally.

---

3. Technical & Developer Questions

Q: How do I add a new transport or adapter? A: See the Developer Guide. Implement the relevant trait (`Transport` or `ProtocolAdapter`) and register it in the backend.

Q: Is there a plugin system? A: Yes. GhostWire is designed for modularity and extension.

Q: How do I contribute code? A: Fork the repo, make your changes, and submit a pull request. See the Contributing chapter for details.

Q: Which transports work today? A: **Three transports are working today: stealth TCP, WebRTC, and Reticulum.** QUIC is implemented. Bluetooth LE is a skeleton, WiFi Direct is a platform boundary awaiting an OS provider, and LoRa is designed but not implemented — see docs/BIBLE_STATUS_SPINE.md §4 for the evidence behind each. The system also includes Traffic Armor, which camouflages traffic and auto-switches transports when one is blocked, using Thompson Sampling to pick the most energy-efficient route.

Q: How does the device hierarchy work? A: GhostWire organizes devices into tiers by available RAM. A phone with 6GB or less handles 20 peers and 1GB of storage. A tablet with 8GB or less handles 30 peers. A Raspberry Pi with 4GB or less handles 15 peers. A laptop with 16GB or less handles 50 peers and 5GB of storage. A server with more than 16GB handles 100 or more peers and 50GB or more of storage. Lower-power devices can offload relay and storage duties to a nearby laptop or server, keeping the mesh stable as it grows.

Q: What AI runs on devices and does it send my data somewhere? A: The *design* is for AI models to run on your device, so your data never leaves it. In the current build, however, only the PRoPHET delay-tolerant routing formula is active — no model files are present in the repository, so the LightGBM classifier, the Gemma 4 analytical layer, and the graph neural network do not load (they fail silently to a fallback). Routing decisions today are made by the mesh and PRoPHET, not by ML. No raw data is sent to any central server, and no gradients are transmitted — so federated-learning privacy properties hold by construction. See docs/BIBLE_STATUS_SPINE.md §5 for the per-layer status.

Q: Which protocol bridges work today versus planned? A: The Matrix bridge is implemented behind a feature flag and provides basic connectivity. The Meshtastic bridge is implemented with erasure coding, using 3 data and 2 parity shards so any 3 of 5 can reconstruct a message, which works around the 242-byte LoRa frame limit. The Briar bridge is planned and not yet available. The Reticulum bridge is planned and experimental behind a feature flag. Tor integration is not implemented.

Q: Has GhostWire been audited? A: An external security audit is planned but has not yet been conducted. Candidate auditors include Cure53, NCC Group, and Least Authority. No audit results are available at this time.

Q: Is my battery drained by relaying? A: GhostWire is designed to be battery-aware. When your battery is charging, relaying runs without restrictions. Above 50%, normal operation applies. Between 20% and 50%, cover traffic is reduced by 30%. At 20% or below, the device drops to the most minimal privacy tier and limits relaying. The system also uses energy-aware transport selection, preferring transports that drain less battery.

---

4. Security & Privacy

Q: How secure is GhostWire? A: All messages are end-to-end encrypted. The system uses post-quantum ML-KEM-768 key encapsulation, XChaCha20-Poly1305 encryption, and perfect forward secrecy.

See the Security chapter for details.

Q: What if my device is seized? A: Keys are encrypted on disk. Use device encryption for extra safety. A duress PIN can protect against coercion, and a dead man's switch can wipe keys if you fail to check in.

Q: Can I disable security features? A: Not recommended. Only for advanced users in test environments.

Q: What is a privacy tier? A: GhostWire assigns privacy tiers based on device type and battery level. Servers operate at L3, laptops and tablets at L2, and phones and Raspberry Pis at L1. When battery drops, the device drops one tier, reducing its visibility and relaying activity.

---

5. Real-World Scenarios

Q: How does GhostWire help in a disaster? A: It enables communication when infrastructure is down, using LoRa, WiFi, and Bluetooth.

Q: Can GhostWire work without internet connectivity? A: Yes. It uses Bluetooth, WiFi, and LoRa to maintain communication during infrastructure failures.

Q: Has GhostWire been used in real-world events? A: See the Case Studies chapter for detailed stories.

---

6. Troubleshooting & Support

Problem	Solution
Can't find other nodes	Check transport settings, try another method
Messages not sending	Ensure at least one transport is active
Web UI won't load	Check backend is running, try npm install
LoRa not working	Check hardware, drivers, and permissions
Security warning	Ensure you are using the latest version

- **Logs:** Check backend logs for errors.
- **Community:** Ask for help on GitHub or project chat.

---

7. Visuals: Common Flows

---

8. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101
- Security & Privacy
- Developer Guide
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence

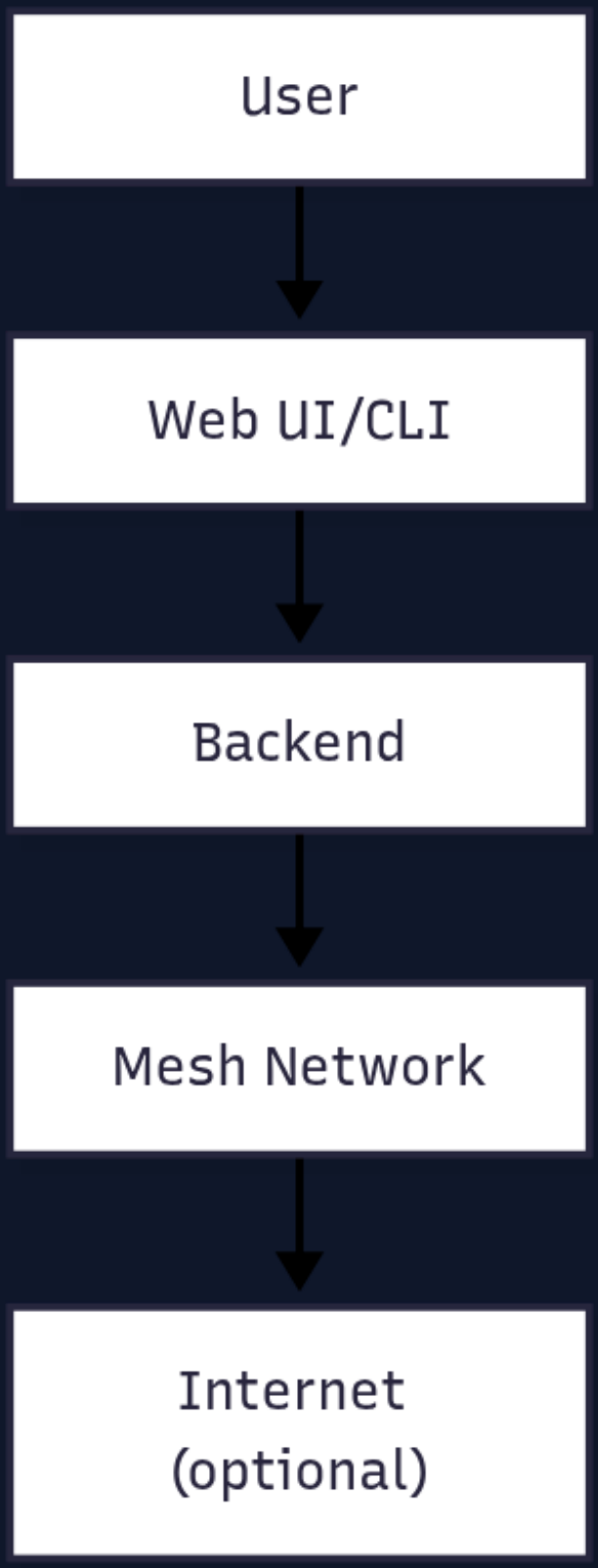


Diagram 7: faq

End of Chapter

Contributing & Community

Table of Contents

1. Welcome & Philosophy
2. How Anyone Can Contribute
3. Code Contributions (Technical)
4. Writing, Design, and Outreach
5. Community Guidelines & Code of Conduct
6. Example Contribution Journey
7. Visuals: Contribution Flow
8. Best Practices for Contributors
9. Further Reading & Resources

---

1. Welcome & Philosophy

GhostWire is open to everyone, whether you are a coder, writer, designer, tester, or just curious. Here is how you can help build the future of secure, decentralized communication.

---

2. How Anyone Can Contribute

- **Test the app:** Try GhostWire and give feedback.
- **Report bugs:** Found a problem? Open an issue on GitHub.
- **Write docs:** Help make guides clearer for everyone.
- **Design:** Improve the UI/UX or create graphics.
- **Spread the word:** Share GhostWire with your community.
- **Join discussions:** Help shape the roadmap and features.

---

3. Code Contributions (Technical)

- **Code style:**
  - Rust: Follow rustfmt and clippy guidelines.
  - JS/TS: Use Prettier and ESLint.
- **How to contribute:**
  1. Fork the repo on GitHub.
  2. Create a feature branch.
  3. Make your changes and add tests.
  4. Run all tests and linters.
  5. Submit a pull request with a clear description.
- **Review process:**
  - All PRs are reviewed for security, style, and clarity.
  - Feedback is constructive and focused on improvement.

---

4. Writing, Design, and Outreach

- **Docs:** Improve guides, add visuals, translate content.
- **Design:** Create logos, UI mockups, infographics.
- **Outreach:** Write blog posts, give talks, organize events.

---

5. Community Guidelines & Code of Conduct

- **Be respectful:** Treat everyone with kindness and respect.
- **Be inclusive:** Welcome contributors of all backgrounds and skill levels.
- **No harassment:** Zero tolerance for abuse or discrimination.
- **Help others:** Support newcomers and share knowledge.

---

6. Example Contribution Journey

The following are hypothetical examples showing common paths new contributors take. They are not real stories but illustrate what a typical contribution looks like.

Example 1: Documentation fix A contributor notices a typo in the FAQ chapter. They fork the repo, edit the markdown file, and submit a pull request. The fix is reviewed and merged within a day. This is a common first contribution because it requires minimal code knowledge.

Example 2: Transport implementation A developer wants to add a new transport. They implement the `Transport` trait, write unit tests, and register it in the adapter registry. After code review and a few rounds of feedback, the transport is merged. This path requires deeper Rust knowledge.

Example 3: Design contribution A designer creates a new icon set for the web UI. They submit the assets through a pull request with a preview. The design team reviews the assets for consistency with the project style guide.

---

7. Visuals: Contribution Flow

---

8. Best Practices for Contributors

- Communicate clearly and kindly.
- Ask questions if you are unsure.
- Test your changes before submitting.
- Review the documentation and guidelines.
- Celebrate your contributions.

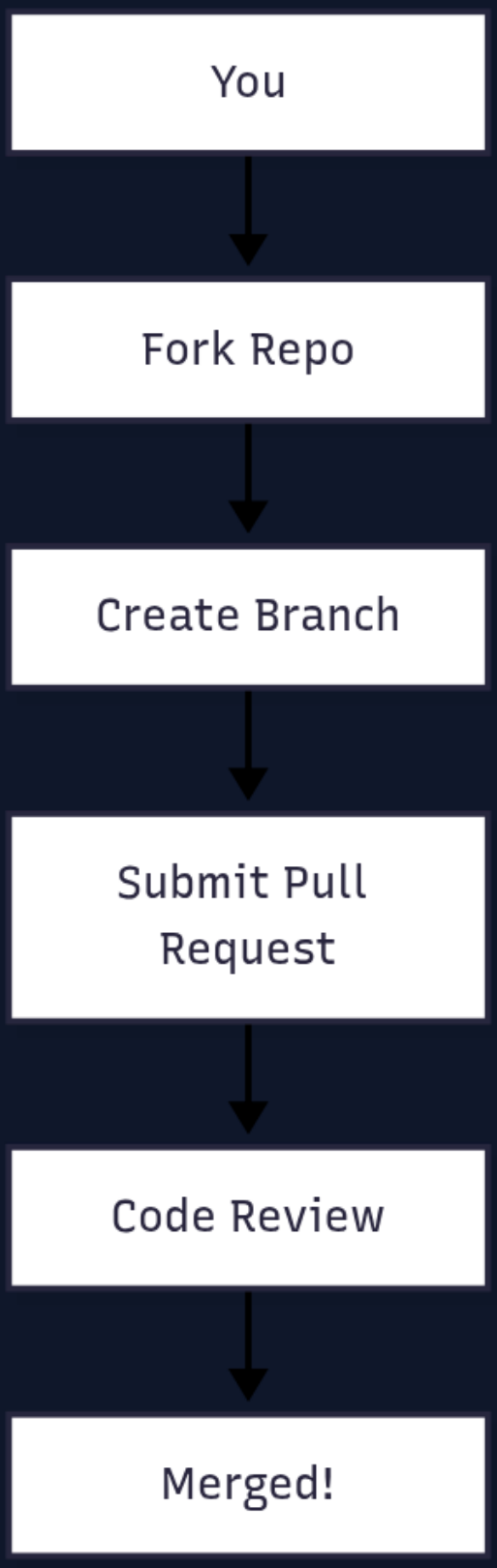


Diagram 8: contributing

9. Further Reading & Resources

- GhostWire GitHub
- CONTRIBUTING.md
- Code of Conduct
- Open Source Guides

End of Chapter

Deployment & Operations

Table of Contents

1. Overview
2. Deployment Scenarios
3. Assigning Roles by Power and Hardware
4. Step-by-Step Deployment Guides
5. Docker, Cloud, and Bare-Metal Installations
6. Monitoring, Logging, and Maintenance
7. Visuals: Deployment Topologies
8. Troubleshooting & Recovery
9. Best Practices & Anti-Patterns
10. Further Reading & Resources

---

1. Overview

This chapter covers how to deploy, operate, and maintain GhostWire in a variety of real-world scenarios, from a single home node to a city-wide mesh. Both non-technical and technical readers will find step-by-step guides, visuals, and best practices.

---

2. Deployment Scenarios

Home/Personal Mesh

- **Goal:** Connect a few devices (phones, laptops) for secure messaging at home or in a small group.
- **Steps:**
  1. Download and install the GhostWire app or desktop client.
  2. Start the app and select your preferred transport (WiFi Direct, LoRa).
  3. Invite nearby devices to join your mesh.

Community/Neighborhood Mesh

- **Goal:** Connect dozens of devices across a neighborhood or event.
- **Steps:**
  1. Deploy GhostWire on laptops, phones, and LoRa relays.

2. Use WiFi Direct and LoRa for coverage.
3. Assign some nodes as relays for better reach.

Disaster/Field Deployment

- **Goal:** Restore communication after infrastructure failure.
- **Steps:**
  1. Deploy LoRa nodes at key locations (battery/solar powered).
  2. Use store-and-forward for offline delivery.
  3. Federate with other meshes if possible.

Enterprise/Federated Mesh

- **Goal:** Connect multiple sites, enable advanced security, and integrate with existing systems.
- **Steps:**
  1. Deploy GhostWire on servers (bare-metal or cloud).
  2. Use Docker or Kubernetes for scaling.
  3. Integrate with protocol adapters for interoperability.

---

3. Assigning Roles by Power and Hardware

GhostWire assigns roles based on device capability. The right device in the right role makes the mesh faster, more resilient, and easier to operate.

Device Classes

Class	RAM	Max Peers	Typical Device
Phone	<=6GB	20	Consumer smartphone
Tablet	<=8GB	30	iPad, Android tablet
Raspberry Pi	<=4GB	15	Pi 4, Pi 5
Laptop	<=16GB	50	Consumer laptop, ultrabook
Server	>16GB	100+	Rack server, cloud VM

Roles

- **Full Node:** Stores the complete mesh state. Participates in routing and relay. Requires the most resources.
- **Relay:** Forwards messages between nodes. Relays are the backbone of the mesh.
- **Leaf:** Connects to the mesh but does not relay for others. Lowest resource requirement.
- **Sensor:** Publishes telemetry or data feeds. Lightweight, often battery-powered.
- **Replicator:** Maintains copies of critical data for disaster recovery. Operates on servers or dedicated hardware.

Role Assignment Principles

- "Assign key nodes (with good power and connectivity) as relays."
- "Use devices with more RAM/CPU for relays."

- Relays carry no trust authority. A relay forwards packets but cannot read their contents.

Battery States and Role Behavior

GhostWire adjusts node behavior based on battery level:

Battery State	Behavior
Charging	Unrestricted. Full role, full cover traffic.
Above 50%	Normal operation. All roles available.
20-50%	Cut cover traffic by 30% to conserve energy.
20% or below	Drop to privacy L0/L1. Minimize all non-essential activity.

Solar and Battery-Powered LoRa Relays

LoRa relays in remote deployments run on solar panels with battery backup. These nodes prioritize store-and-forward messaging and minimize radio wake time. During low-light conditions, the relay drops to L1 privacy and reduces transmission frequency.

Supernode Election Operations

- Elect nodes with high uptime, stable IP addresses, and good bandwidth as "Relay Hubs."
- Relay Hubs form the backbone of the mesh and handle inter-region traffic.
- Regional sharding is planned for v3/v4, where each geographic region operates its own DHT shard connected to a global backbone.
- Supernode election uses uptime history, bandwidth measurements, and peer reputation scores.

ESP32 Gateway-Bridged Deployments

- **$20 ESP32 mini-mesh:** A low-cost ESP32 board running a lightweight mesh client bridged to WiFi. Ideal for adding legacy sensors or very low-power nodes to a WiFi-based mesh.
- **$100 Raspberry Pi gateway:** A Pi running GhostWire with a WiFi adapter and LoRa module. Bridges between WiFi Direct, LoRa, and the broader internet.
- **Server-class backbone:** A dedicated server acting as the root relay and federation point. Handles the highest peer counts and provides the bridge to external networks.

OEM Battery Optimization Note

Many consumer devices sold in Kenya and East Africa come from OEMs such as Samsung, Xiaomi, Huawei, OPPO, and Transsion. These manufacturers aggressively optimize battery usage and may kill background processes. Follow the dontkillmyapp guidance to ensure GhostWire continues running in the background. Whitelist the GhostWire process in battery optimization settings and disable aggressive app standby modes.

4. Step-by-Step Deployment Guides

Local (Laptop/Desktop)

1. Download and install GhostWire.
2. Run the backend and web UI.
3. Join or create a mesh.

Docker

1. Pull the GhostWire Docker image:

   ```
   docker pull phantomojo/ghostwire:latest
   ```

2. Run the container:

   ```
   docker run -d -p 3000:3000 -p 9000:9000 phantomojo/ghostwire:latest
   ```

3. Access the web UI at http://localhost:3000

Cloud (AWS, GCP, Azure)

1. Provision a VM or container instance.
2. Install Docker or run natively.
3. Open required ports (3000, 9000, LoRa if needed).
4. Secure with firewalls and access controls.

Bare-Metal

1. Install Rust and Node.js.
2. Build and run GhostWire as per Getting Started.
3. Set up systemd service for auto-restart.

---

5. Docker, Cloud, and Bare-Metal Installations

- **Docker:** Easiest for quick deployment and scaling.
- **Cloud:** Use for global reach, federation, and integration.
- **Bare-Metal:** Best for custom hardware, edge, or offline use.

---

6. Monitoring, Logging, and Maintenance

- **Monitoring:**
  - Use built-in web UI dashboard for node status and health.
  - Integrate with Prometheus/Grafana for advanced metrics.
- **Logging:**
  - Backend logs to file and stdout.
  - Use log rotation for long-term deployments.
- **Maintenance:**
  - Regularly update software.
  - Backup configuration and keys.

– Test failover and recovery procedures.

---

7. Visuals: Deployment Topologies

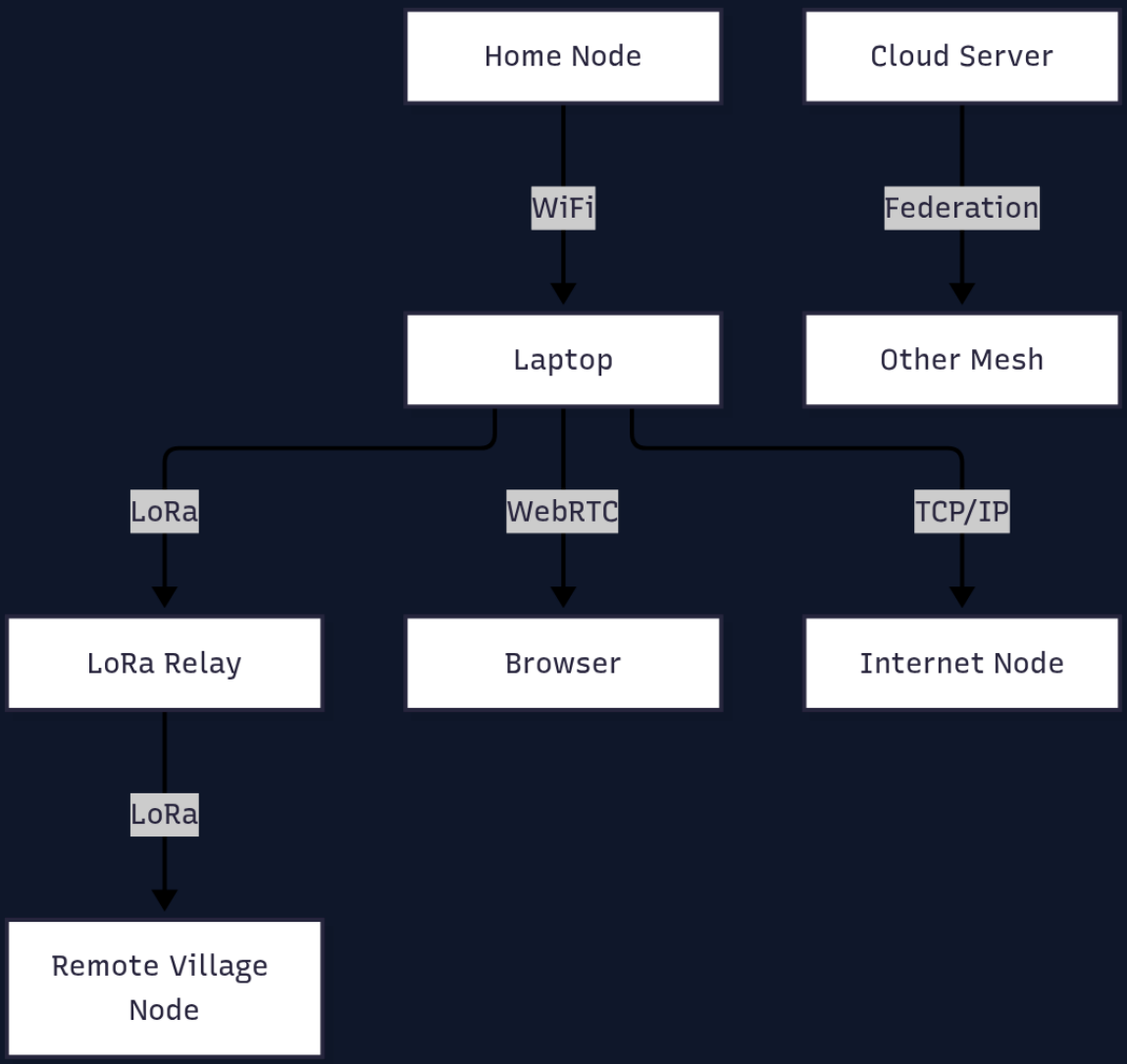


Diagram 9: deployment operations

---

8. Troubleshooting & Recovery

Problem	Solution
Node won't start	Check logs, ensure dependencies are installed
Can't connect to mesh	Check transport settings, firewall, and ports
Docker container fails	Check image version, logs, and port mapping

Problem	Solution
Cloud instance unreachable	Check security groups, firewall, and DNS
LoRa not working	Check hardware, drivers, and permissions
OEM battery killing process	Follow dontkillmyapp guidance, whitelist app

---

9. Best Practices & Anti-Patterns

Best Practices

- Use Docker for easy scaling and updates.
- Monitor node health and logs.
- Regularly backup configuration and keys.
- Test failover and recovery.
- Secure cloud deployments with firewalls and access controls.
- Assign relays based on device capability, not just availability.
- Follow dontkillmyapp guidance on OEM-optimized Android devices. ### Anti-Patterns
- Ignoring updates or logs.
- Using default passwords or open ports.
- Not testing disaster recovery.
- Placing low-power devices as relays.

---

10. Further Reading & Resources

- Docker Documentation
- Prometheus
- Grafana
- GhostWire Developer Guide
- dontkillmyapp

End of Chapter

Advanced Security & Threat Response

Table of Contents

1. Overview
2. Threat Modeling (Plain & Technical)
3. Incident Response & Forensics
4. Security Module Configuration
5. Audit & Compliance
6. Example Attack/Defense Scenarios
7. Visuals: Threat Flows & Response

8. Actionable Checklists
9. Further Reading & Resources

---

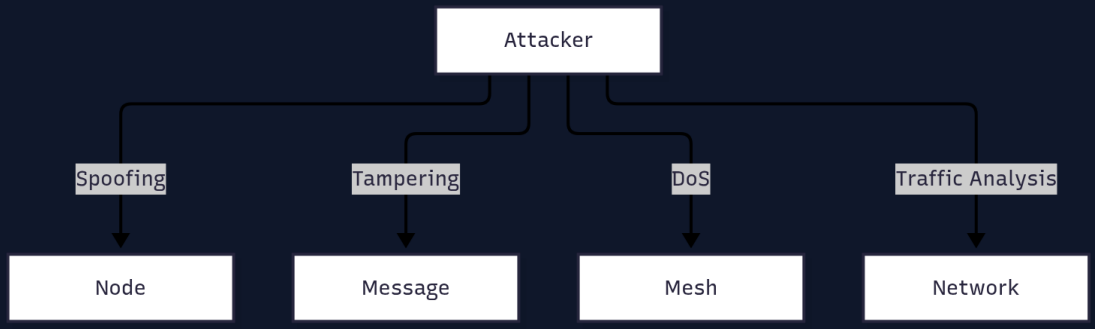
1. Overview

This chapter is for those who want to go beyond the basics, security engineers, admins, and anyone responsible for defending a GhostWire mesh. It covers advanced threat modeling, incident response, forensics, and real-world attack/defense scenarios, with both plain-language and technical explanations.

---

2. Threat Modeling (Plain & Technical)

- **Plain:** Threat modeling is thinking ahead about what could go wrong and how to stop it.
- **Technical:** Use frameworks like STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to analyze risks.
- **Visual:**



---

3. Incident Response & Forensics

- **Preparation:**
  - Document your mesh topology and key nodes.
  - Regularly backup configuration and keys.
  - Configure dead man's switch timeouts for all relay nodes.
  - Distribute Shamir secret sharing shares to trusted contacts.
- **Detection:**
  - Monitor logs and alerts for suspicious activity.
  - Use anomaly detection tools if available.
  - Watch for unexpected peer count spikes (Sybil indicator).
  - Monitor for cover traffic pattern deviations.
- **Response:**
  - Isolate compromised nodes.
  - Rotate keys and update blacklists.
  - Communicate securely with trusted nodes.
  - Activate duress PIN mode if coercion is suspected.
  - Wipe the device manually if seizure is confirmed and the dead man's switch has not triggered — the switch notifies contacts but does not itself destroy keys.
- **Forensics:**

  - Collect logs and evidence.
  - Analyze attack vectors and entry points.
  - Report findings to the community.
  - Preserve encrypted volumes for post-incident analysis.

Duress PIN Incident Response

If a device running GhostWire is seized or the operator is coerced: 1. **Enter duress PIN:** This activates plausible-deniability mode. The device presents a harmless or empty view of the mesh. Real identity and keys remain cryptographically hidden. 2. **Observe behavior:** Under duress mode, the device appears to have no active mesh connections and no stored messages. An adversary inspecting the device cannot distinguish this from a device that was never configured. 3. **Dead man's switch:** If the device goes silent beyond the configured timeout after seizure, the dead man's switch triggers and notifies trusted contacts. **It does not currently wipe local keys** — the wipe is left to the caller and is not wired into a production path, so an attacker who recovers the device before the switch fires still has the keys. 4. **Post-seizure recovery:** Once the operator is safe, they can use Shamir secret sharing shares to reconstruct their identity on a new device.

---

4. Security Module Configuration

- **SybilDefense:** Adjust proof-of-work/stake parameters.
- **QuotaEnforcer:** Set rate limits for messages and actions.
- **BlacklistManager:** Add/remove nodes as needed.
- **TrafficObfuscator:** Enable/disable padding and timing obfuscation.
- **KeyManager:** Schedule key rotation and backups.

---

5. Audit & Compliance

- **Audit:**
  - Regularly review logs and configuration.
  - Use automated tools for compliance checks.
- **Compliance:**
  - Follow local laws and regulations for encryption and radio use.
  - Document security policies and procedures.

External Audit Status

An external security audit has NOT been performed. GhostWire's current protections are self-reviewed engineering controls. The development team has conducted internal reviews of the cryptographic implementations, module interactions, and threat model coverage.

External audit is PLANNED. Candidate auditing firms include Cure53, NCC Group, and Least Authority. No engagement has been confirmed and no audit has commenced as of September 2026.

Organizations requiring certified security assurance should plan for an external audit before deploying GhostWire in production environments handling sensitive data.

---

6. Example Attack/Defense Scenarios

- **Sybil Attack:**
  - **Attack:** Adversary floods mesh with fake nodes.
  - **Defense:** Enable SybilDefense, require proof-of-work/stake, monitor for anomalies.
- **DoS Attack:**
  - **Attack:** Flood of messages to overwhelm nodes.
  - **Defense:** Set quotas, enable rate limiting, blacklist offenders.
- **Traffic Analysis:**
  - **Attack:** Adversary infers communication patterns by observing timing, volume, and packet sizes.
  - **Defense:** Enable traffic obfuscation, use stealth transports. Traffic Armor disguises WebRTC traffic to look like a video call. Stealth TCP automatically escalates to alternative protocols when blocked. SPHINX onion routing ensures no single node knows both sender and recipient. Cover traffic generates dummy packets to mask real communication patterns.
- **Key Compromise:**
  - **Attack:** Device is seized, keys are stolen.
  - **Defense:** Use encrypted storage, rotate keys, wipe the device manually if needed. Enable duress PIN mode for plausible deniability. The dead man's switch provides automatic *notification* on extended silence; automatic key wiping is not yet wired up.
- **Device Seizure with Coercion:**
  - **Attack:** Adversary physically seizes a device and forces the operator to reveal credentials.
  - **Defense:** Duress PIN activates plausible-deniability mode. Real keys remain encrypted and inaccessible. Dead man's switch triggers if the device goes silent. Shamir secret sharing allows recovery on a new device without the seized one.

---

7. Visuals: Threat Flows & Response

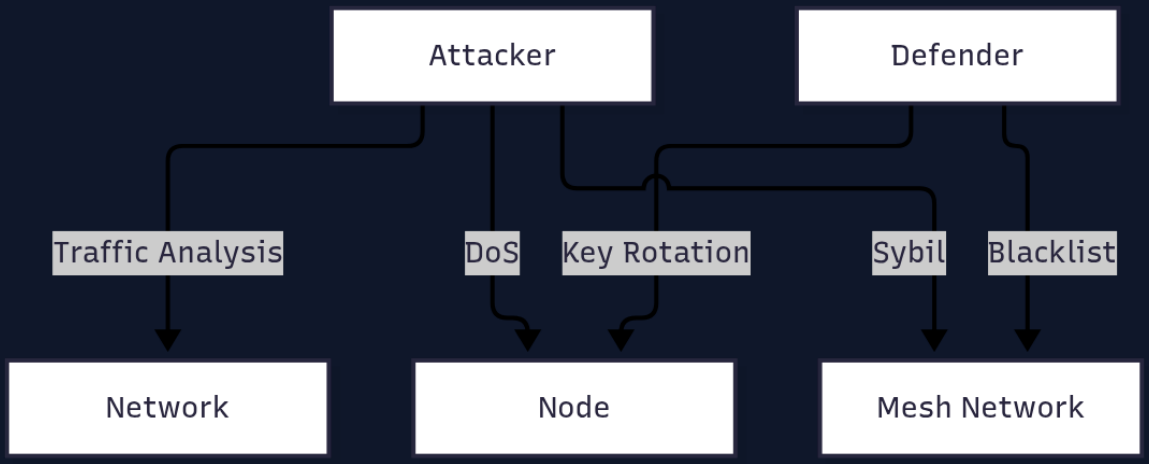


Diagram 10: advanced security

---

8. Actionable Checklists

- ☐ Document mesh topology and key nodes
- ☐ Regularly backup configuration and keys
- ☐ Monitor logs and alerts
- ☐ Enable all security modules
- ☐ Review and update blacklists
- ☐ Test incident response procedures
- ☐ Test duress PIN mode before deployment
- ☐ Configure dead man's switch timeout
- ☐ Verify cover traffic is active on relay nodes
- ☐ Confirm stealth transports are configured
- ☐ Distribute Shamir secret sharing shares to trusted contacts
- ☐ Review external audit status and plan accordingly

---

9. Further Reading & Resources

- OWASP Incident Response
- NIST Cybersecurity Framework
- GhostWire Security & Privacy
- Signal Protocol Design
- SPHINX Onion Routing

End of Chapter

Performance & Scaling

Table of Contents

1. Overview
2. Optimization Tips
3. Benchmarks & Metrics
4. Tiered Scaling Architecture
5. Energy-Aware Transport Selection
6. AI Latency Context
7. Troubleshooting Performance
8. Visuals: Performance Flows
9. Best Practices & Anti-Patterns
10. Further Reading & Resources

---

1. Overview

This chapter covers how to optimize GhostWire for speed, reliability, and large-scale deployments. Both non-technical and technical readers will find practical tips, benchmarks, and troubleshooting guides, with

visuals.

---

2. Optimization Tips

- **Use multiple transports:** Combine WiFi Direct, LoRa, and Bluetooth for best coverage.
- **Prioritize nodes:** Assign key nodes (with good power and connectivity) as relays.
- **Tune quotas:** Adjust rate limits for your mesh size and expected traffic.
- **Monitor health:** Use the web UI or Prometheus/Grafana for real-time stats.
- **Upgrade hardware:** Use devices with more RAM/CPU for relays.

---

3. Benchmarks & Metrics

Scenario	Devices	Avg. Latency	Max Throughput
Home Mesh	5	50ms	1 Mbps
Community Mesh	50	100ms	5 Mbps
Disaster Field	20	200ms	500 Kbps
Enterprise Mesh	200+	150ms	10 Mbps

- **Metrics to monitor:**
  - Latency (ms)
  - Throughput (Mbps)
  - Node uptime (%)
  - Message delivery rate (%)
  - Relay hop count
  - Cover traffic overhead (bytes/sec)
  - Battery level of mobile nodes
  - Peer count per node
  - Transport switching frequency

---

4. Tiered Scaling Architecture

A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture.

The Problem with Flat Meshes

In a flat mesh, every node participates in routing. As node count grows, routing tables expand, message latency increases, and the network becomes fragile. A community mesh of 50 nodes works well. A city-scale mesh of 500+ nodes requires structure.

Supernode and Relay Hub Election

- Elect nodes with high uptime, stable IP addresses, and good bandwidth as "Relay Hubs."
- Relay Hubs form the backbone of the mesh and handle inter-region traffic.
- Supernode election uses uptime history, bandwidth measurements, and peer reputation scores.

- Relay Hubs are not trust authorities. They forward packets but cannot read their contents.

Regional DHT Sharding (v3/v4 Plan)

- Each geographic region operates its own DHT shard.
- Regional shards connect to a global backbone through designated gateway nodes.
- This reduces per-node routing overhead and improves lookup latency.
- The hierarchy is: Leaf nodes -> Regional relays -> Supernode Relay Hubs -> Global backbone.

Scaling Targets

Tier	Node Count	Architecture
Small	5-20	Flat mesh
Medium	20-100	Relay-based with elected hubs
Large	100-1000	Regional sharding with supernodes
Enterprise	1000+	Hierarchical supernodes with global backbone

---

5. Energy-Aware Transport Selection

GhostWire selects transports based on energy efficiency using Thompson Sampling, a multi-armed bandit algorithm that balances exploration and exploitation.

Energy-Efficiency Weights

Each transport has an energy-efficiency weight that influences selection when battery is constrained:

Transport	Energy Weight	Notes
BLE (Bluetooth Low Energy)	0.30	Very low power, short range
LoRa	0.35	Low power, long range, low bandwidth
Reticulum	0.25	Adaptive, moderate power
WiFi Direct	0.05	Higher power, high bandwidth
TCP/QUIC	0	Internet-bound, highest power

How Thompson Sampling Works

1. Each transport starts with a prior distribution over its expected energy cost.
2. After each transmission, the observed energy cost updates the distribution.
3. The algorithm samples from each transport's posterior and selects the one with the highest sampled efficiency.
4. Over time, the algorithm converges on the most energy-efficient transport for the current conditions.

Battery-Aware Behavior

- **Charging:** Unrestricted. All transports available. Full cover traffic.
- **Above 50%:** Normal operation. Thompson Sampling selects transports normally.

- **20-50%:** Cut cover traffic by 30%. Bias Thompson Sampling toward BLE and LoRa.
- **20% or below:** Drop to privacy L0/L1. Minimize all non-essential activity. Prefer LoRa only.

---

6. AI Latency Context

GhostWire integrates lightweight AI for message routing, anomaly detection, and predictive mesh optimization. Understanding AI latency helps plan deployment expectations.

Inference Timings

- **LightGBM inference:** not yet measured. An earlier version of this document quoted 77 microseconds on a Raspberry Pi; that figure came from a code comment (`ai/onnx_inference.rs:153`) and was never benchmarked, and no model file is present in the tree. Layer 1 is also not called from routing decisions today — see docs/BIBLE_STATUS_SPINE.md §5.
- **ML-KEM-768 handshake:** 10.5 microseconds for encapsulation on x86_64 development hardware (measured; `cargo bench --bench hot_paths -- mlkem768`). Not yet measured on embedded targets.
- **Gemma 4 analytical pass:** Every 30 seconds. Used for mesh health analysis and traffic pattern recognition.

AI Architecture

- **LightGBM:** Gradient-boosted decision trees for fast classification. Runs locally on every node.
- **Gemma 4:** Lightweight analytical model for periodic mesh assessment. Runs on server-class nodes.
- **GNN (Graph Neural Network):** Planned for future routing optimization across the mesh topology.
- **PRoPHET:** Probabilistic routing protocol for delay-tolerant networks. Used in disaster/field scenarios.

Telemetry

- Telemetry data is sent up (opt-in) to improve models.
- Model files are distributed down via DHT hash and P2P sync.
- This is not federated learning. Models are shared as files, not as gradient updates.

---

7. Troubleshooting Performance

Problem	Solution
High latency	Check transport health, upgrade relays
Dropped messages	Increase quotas, check logs
Node offline	Check power/network, use redundant relays
Slow UI	Upgrade device, close unused apps
Battery draining fast	Check transport selection, reduce cover traffic
Relay overload	Elect new relay hubs, rebalance peers
Transport switching too often	Adjust Thompson Sampling exploration rate

---

8. Visuals: Performance Flows

---

9. Best Practices & Anti-Patterns

Best Practices

- Monitor mesh health and performance.
- Use redundant relays for reliability.
- Regularly update software and hardware.
- Segment large meshes for manageability.
- Use tiered architecture beyond 100 nodes.
- Enable energy-aware transport selection on battery-powered nodes.
- Monitor AI inference latency on resource-constrained devices. ### Anti-Patterns
- Ignoring performance metrics.
- Using outdated hardware for relays.
- Not testing at scale before deployment.
- Running flat meshes beyond their density limit.
- Ignoring battery state when assigning roles.

---

10. Further Reading & Resources

- Prometheus
- Grafana
- GhostWire Developer Guide
- Thompson Sampling
- LightGBM

End of Chapter

Developer Guide

Table of Contents

1. Overview
2. Extending GhostWire (Transports, Adapters, Modules)
3. Adding a Bridge Adapter
4. Extending AI Routing
5. API Reference & Hooks
6. Plugin System & Architecture
7. Testing & CI/CD
8. Visuals: Developer Flows
9. Best Practices & Anti-Patterns

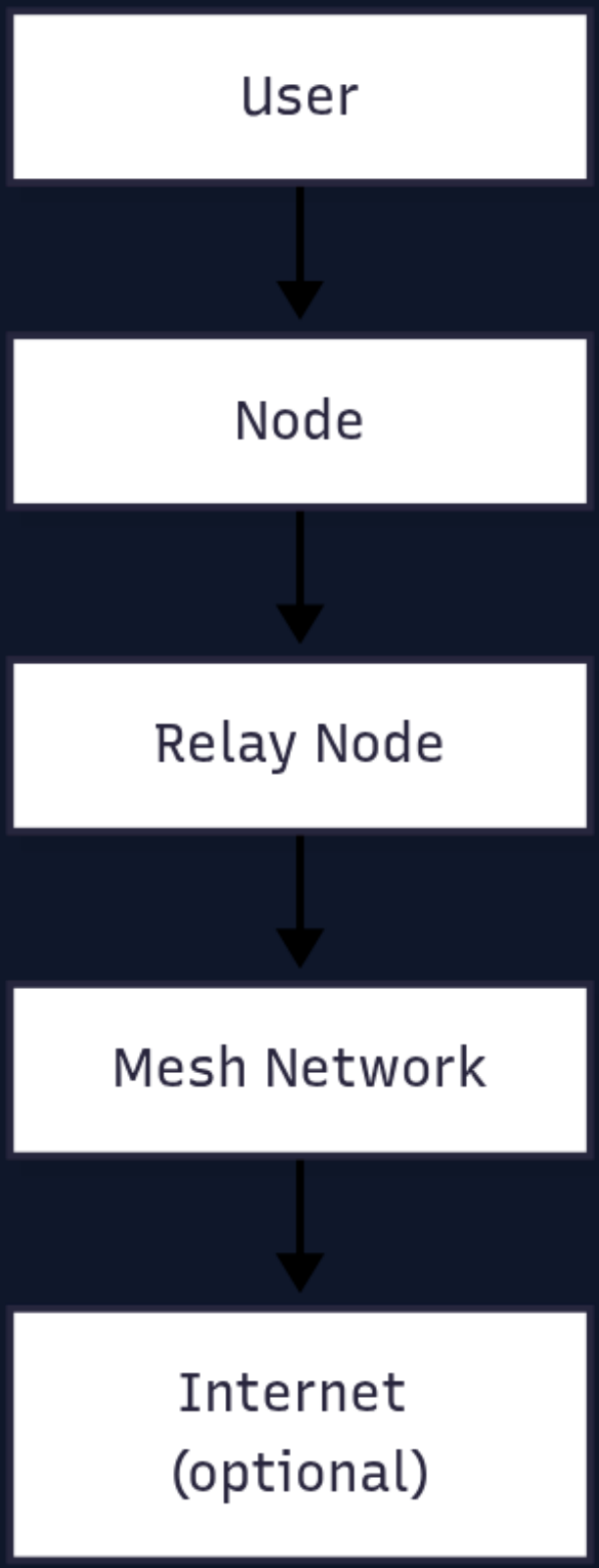


Diagram 11: performance scaling

10. Further Reading & Resources

---

1. Overview

This chapter is for developers who want to extend GhostWire, add new transports, adapters, modules, or contribute to the core. Both non-technical and technical readers will find step-by-step guides, API references, and best practices, with visuals.

---

2. Extending GhostWire (Transports, Adapters, Modules)

Adding a New Transport

1. Implement the `Transport` trait in Rust.
2. Register your transport in the backend.
3. Add configuration options to the web UI/CLI.
4. Test with simulated and real devices.

Adding a Protocol Adapter

1. Implement the `ProtocolAdapter` trait.
2. Handle message translation, deduplication, and relay.
3. Register the adapter in the backend.

Adding a Security Module

1. Implement the relevant trait (e.g., `QuotaEnforcer`).
2. Register and configure in the backend.

---

3. Adding a Bridge Adapter

To add a new bridge adapter that connects GhostWire to an external protocol:

1. **Implement the `ProtocolAdapter` trait.** Define how your adapter sends, receives, and connects to the external network. See the Matrix adapter (`matrix-bridge` feature) and Meshtastic adapter (erasure coding) for reference implementations.
2. **Register the adapter via the adapter registry.** The registry is the single entry point for all adapters. Registering at startup ensures the adapter is initialized correctly and that dependencies are resolved before the backend starts serving traffic.
3. **Test both directions.** Verify that messages flow from GhostWire to the external network and from the external network back to GhostWire. Deduplication and translation must work in both directions.
4. **Add configuration options.** Add a `[adapters]` entry in the TOML config and expose toggles in the web UI.
5. **Test in isolation.** Use a simulated external network or a local test instance before deploying to a live mesh.

Reference: Matrix Adapter

The Matrix adapter is implemented and enabled with the `matrix-bridge` feature flag. It provides basic bridging to Matrix rooms. Study its implementation for patterns on how to structure a new adapter.

Reference: Meshtastic Adapter

The Meshtastic adapter is implemented with erasure coding (3 data + 2 parity shards, any 3 of 5 reconstruct). It fits messages into the 242-byte LoRa frame limit. This adapter demonstrates how to handle protocol-specific constraints within the `ProtocolAdapter` trait.

---

4. Extending AI Routing

The AI Routing Layer (see Architecture chapter) can be extended in two ways:

- **Routing classifier features:** The LightGBM per-message classifier accepts features such as `battery_level`, `transport_metrics`, `message_size`, and `destination_type`. To add a new feature, update the feature extraction module and retrain the classifier.
- **Energy-efficiency weights:** The Thompson Sampling transport selector uses energy-efficiency weights (BLE 0.30, LoRa 0.35, Reticulum 0.25, WiFi 0.05, TCP/QUIC 0). To add a new transport, add its weight to the configuration and update the sampling logic.

For deeper changes (new model architecture, new routing layer), see the AI Routing Layer section in the Architecture chapter.

---

5. API Reference & Hooks

- **REST API:**
  - `/api/nodes`: List nodes
  - `/api/messages`: Send/receive messages
  - `/api/transports`: Manage transports
  - `/api/adapters`: Manage adapters
- **WebSocket API:**
  - Real-time message and event updates
- **Hooks:**
  - Pre-send, post-receive, error handling

---

6. Plugin System & Architecture

- **Plugins:**
  - Add new features without modifying core code.
  - Register plugins via config or UI.
- **Architecture:**
  - Modular, with clear interfaces for each component.
  - See Developer Guide for trait definitions.

---

7. Testing & CI/CD

- **Unit tests:**
  - Write tests for each module and trait.
- **Integration tests:**
  - Test end-to-end flows (see test suite).
- **CI/CD:**
  - Use GitHub Actions for automated builds, tests, and deployments.
  - Linting, security checks, and code coverage included.

---

8. Visuals: Developer Flows

---

9. Best Practices & Anti-Patterns

Best Practices

- Write clear, well-documented code.
- Test all changes before merging.
- Use modular design for easy extension.
- Follow code style and security guidelines.
- Test bridge adapters in both directions.
- Register all adapters through the adapter registry.

Anti-Patterns

- Skipping tests or code review.
- Hardcoding secrets or credentials.
- Ignoring documentation.
- Registering adapters outside the registry.

---

10. Further Reading & Resources

- GhostWire GitHub
- Rust Book
- GitHub Actions

End of Chapter

Case Studies & Scenario Narratives

Table of Contents

1. Overview

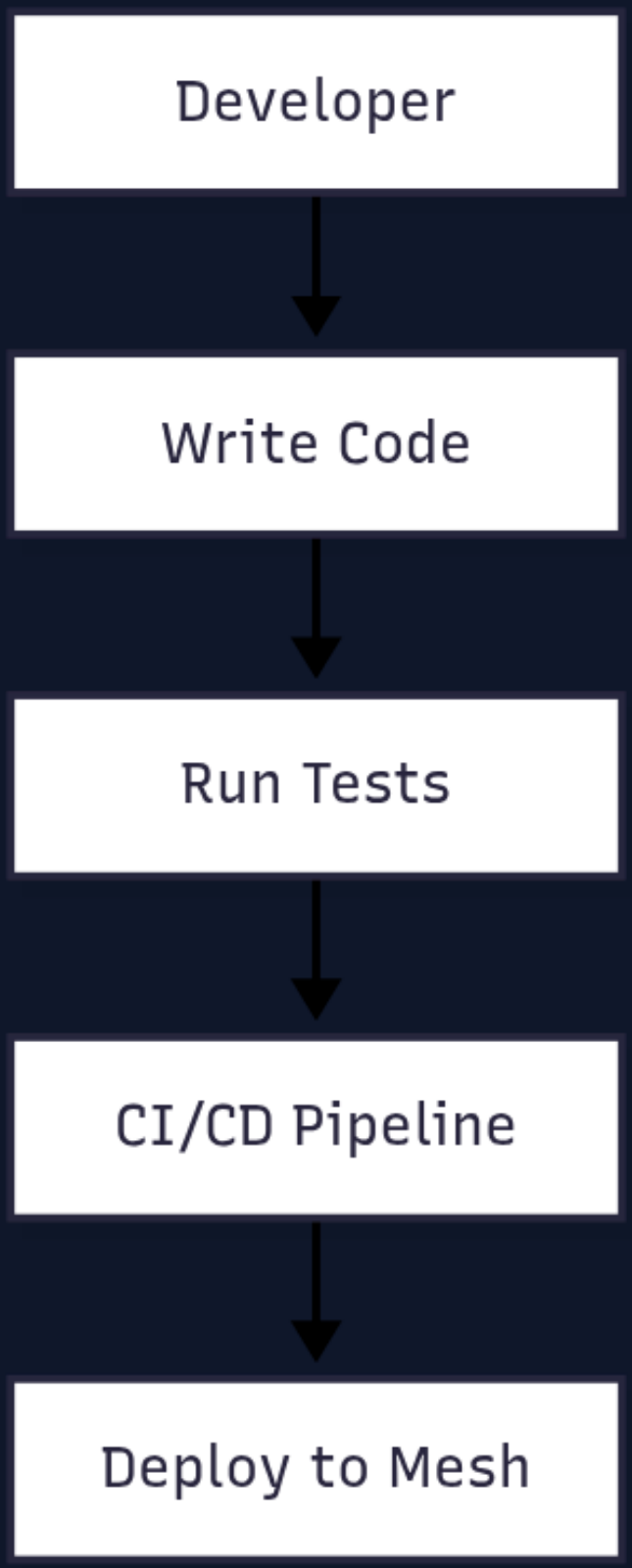


Diagram 12: developer guide

2. Hurricane Response Within 200km
3. Community Blackout
4. Rural Village Connectivity
5. Festival Mesh
6. Lessons Learned
7. Visuals: Scenario Topologies
8. Best Practices & Takeaways
9. Further Reading & Resources

---

1. Overview

This chapter presents illustrative deployment scenarios drawn from project materials. These are scenario narratives, not completed real-world rollouts. Each scenario shows how GhostWire could be deployed in a specific context, using the device hierarchy, transport options, and privacy tiers described in other chapters. The scenarios are designed to help planners understand how the system behaves under different conditions.

For context on the personas who would use these systems, see chapter 01b_who_ghostwire_serves.md. For the device hierarchy that governs role assignment, see chapter 15_device_hierarchy_and_scaling.md.

---

2. Hurricane Response Within 200km

Scenario: A hurricane has damaged infrastructure within a 200km radius. Cell towers are down, internet is unavailable, and first responders need to coordinate across a wide area.

Hierarchy and deployment: - **Phones as leaf nodes:** First responders carry phones configured as leaf nodes. They send and receive messages but do not relay for others, conserving battery. - **Laptops and Raspberry Pi as relays:** Field teams set up laptops and Raspberry Pi devices as relay nodes at staging areas. These devices forward messages between leaf nodes and extend the mesh reach. - **Battery and solar LoRa relays:** LoRa relay nodes are deployed at key locations using battery and solar power. These relays provide long-range connectivity between staging areas that are out of WiFi Direct range. - **Supernode election:** As the response matures, the team elects the most stable nodes (high uptime, stable power, good bandwidth) as supernode relay hubs. These hubs coordinate regional traffic and form the backbone of the emergency mesh.

Transport mix (aspirational — this describes the target deployment, not today's build): LoRa for long-range links between relay stations and WiFi Direct for short-range connections between nearby nodes are the intended transports for this scenario. **Neither is implemented yet** — today this case would run on stealth TCP and WebRTC over whatever internet is available, with Reticulum where amateur radio infrastructure exists. See docs/BIBLE_STATUS_SPINE.md §4.

Privacy tier: Nodes operate at L2 (laptop/tablet tier) or L1 (phone tier) depending on battery. LoRa relays with solar power maintain L2. Phones on battery drop to L1.

Lessons from this scenario: - LoRa relays are essential for bridging gaps beyond WiFi Direct range. - Solar-powered relays maintain operation longer than battery-only nodes. - Supernode election should happen early so the mesh has a stable backbone.

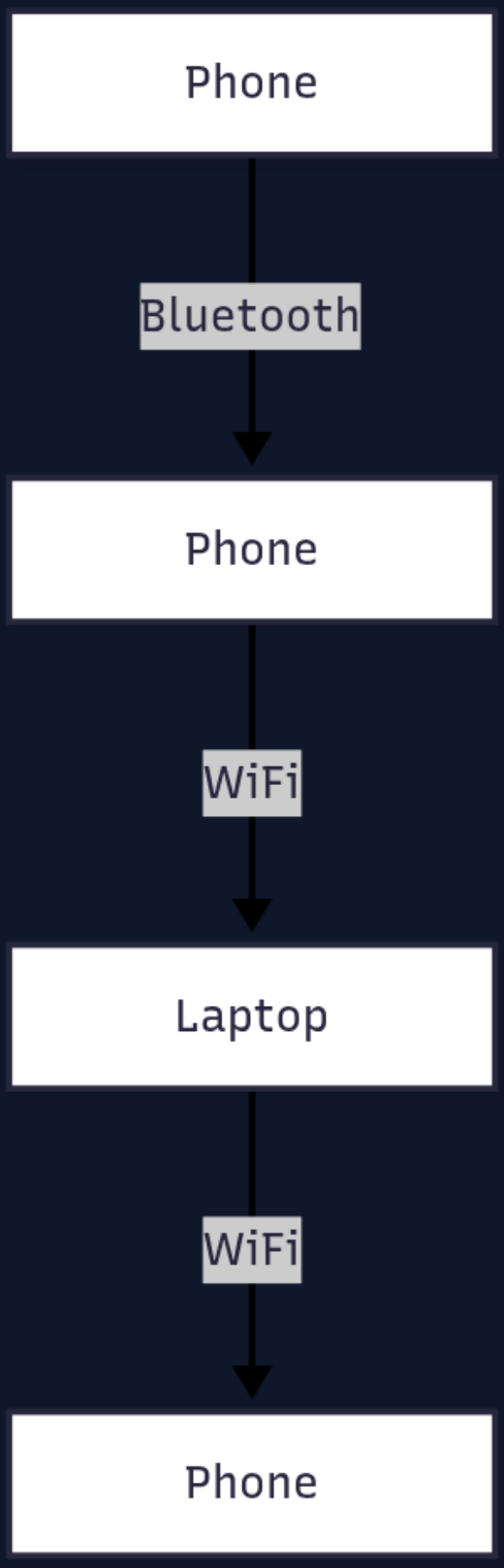


Diagram 13: case studies

3. Community Blackout

Scenario: A city experiences a prolonged power outage. Residents need to coordinate, share information, and maintain communication without internet or cellular service.

Hierarchy and deployment: - **Phones as leaf nodes:** Residents use phones as leaf nodes for messaging. - **Laptops and Raspberry Pi as relays:** Community centers and key locations deploy laptops and Raspberry Pi devices as relays. These nodes form the mesh backbone. - **Battery and solar LoRa relays:** Solar-powered LoRa relays are placed on rooftops and at community centers to provide neighborhood-wide coverage. - **Supernode election for the blackout city:** The city-wide mesh elects relay hubs based on uptime and connectivity. Nodes with stable power (e.g., community centers with generators) become supernodes. Regional sharding divides the city into zones, each with its own relay hub, connected to a city-wide backbone.

Transport mix: WiFi Direct for neighborhood-level communication, LoRa for inter-neighborhood links, WebRTC for any node that regains partial internet access.

Privacy tier: Laptops and Raspberry Pi relays operate at L2. Phones operate at L1. Nodes on battery below 50% cut cover traffic by 30%.

Lessons from this scenario: - Community centers with stable power naturally become supernodes. - Regional sharding prevents a single point of failure from collapsing the entire mesh. - Solar LoRa relays are critical for maintaining connectivity during extended outages.

---

4. Rural Village Connectivity

Scenario: A rural village with no internet access needs to share news, alerts, and educational content among homes, schools, and clinics.

Hierarchy and deployment: - **Phones as leaf nodes:** Villagers use phones to send and receive messages. - **Laptops and Raspberry Pi as relays:** A central location (school or clinic) hosts a laptop or Raspberry Pi as a relay node. This relay connects the village mesh. - **Battery and solar LoRa relays:** LoRa relays connect homes to the central relay. Solar-powered relays are placed at the edges of the village to extend coverage. - **Supernode election:** The central relay node (at the school or clinic) becomes the supernode. It maintains the DHT shard for the village and connects to any external meshes via LoRa or available internet.

Transport mix: LoRa for home-to-relay links, WiFi Direct for local connections within the central hub, Reticulum for any external mesh federation.

Privacy tier: Central relay operates at L3 (server tier) if powered continuously. Home nodes operate at L1. Solar relays maintain L2.

Lessons from this scenario: - A single central relay can serve an entire village if placed strategically. - Solar relays at the village edges extend coverage without running power lines. - The school or clinic is a natural supernode location due to reliable power and community trust.

---

5. Festival Mesh

Scenario: A large festival with overloaded cell networks. Attendees need to form a local mesh for updates, safety alerts, and coordination.

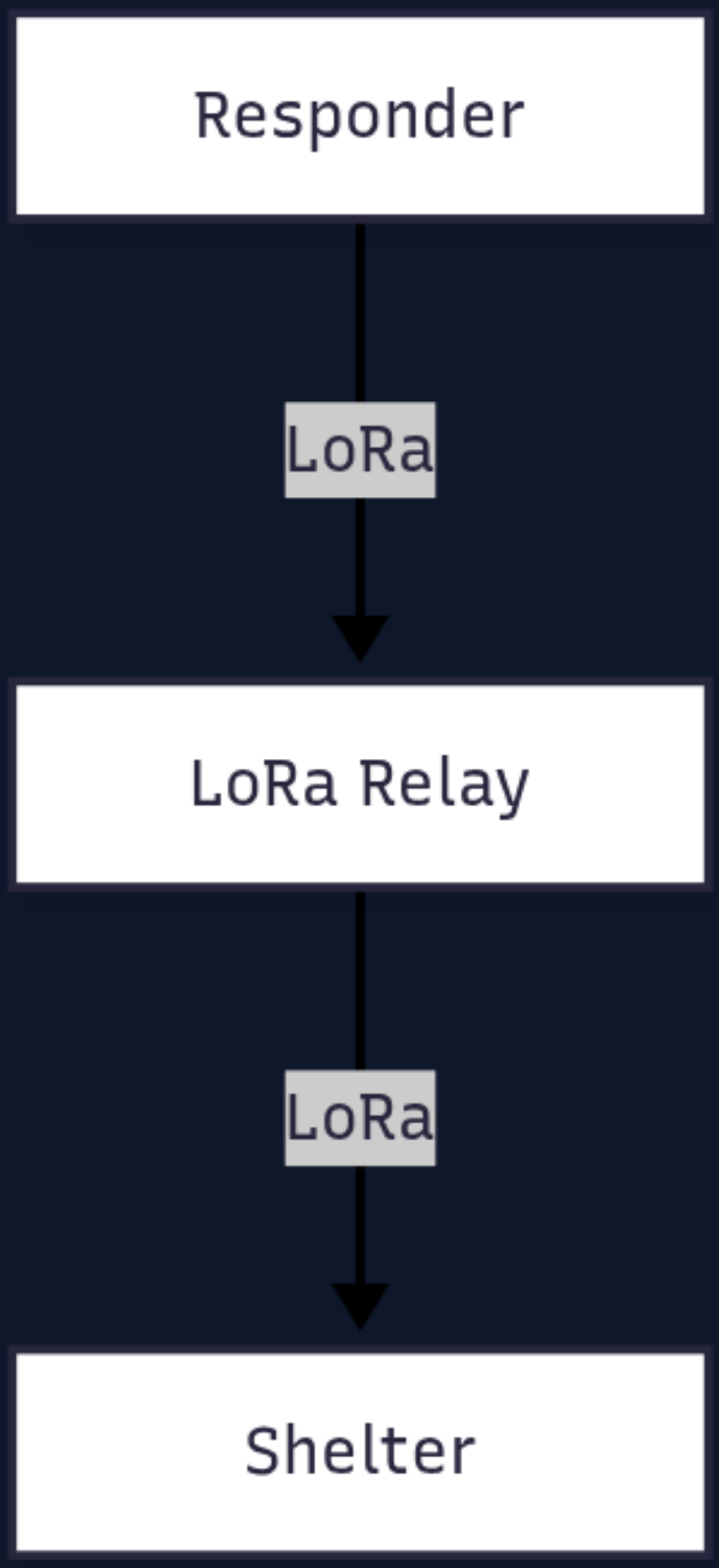


Diagram 14: case studies

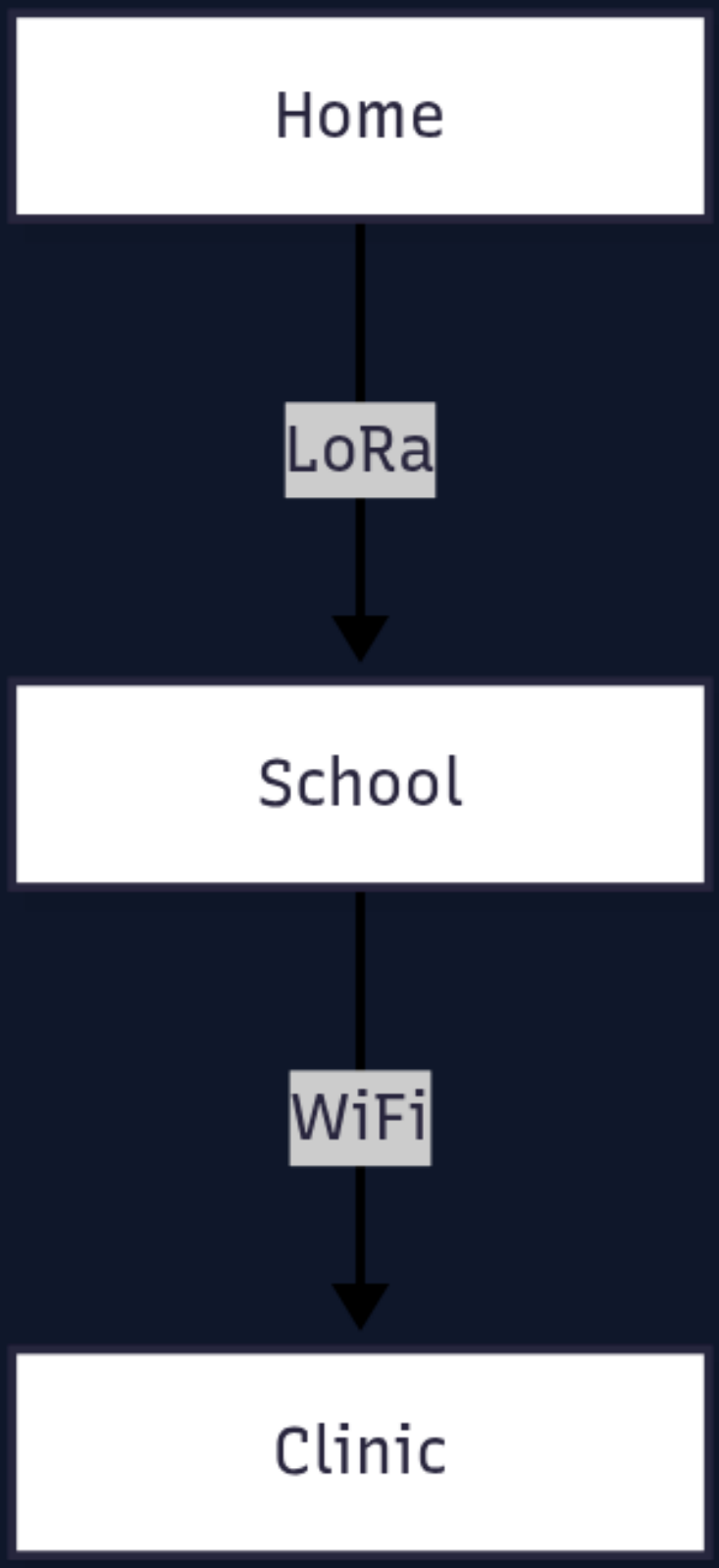


Diagram 15: case studies

Hierarchy and deployment: - **Phones as leaf nodes:** Attendees use phones as leaf nodes. - **Laptops and Raspberry Pi as relays:** Event organizers deploy laptops and Raspberry Pi devices at key points (entrance, stages, info booths) as relays. - **Battery and solar LoRa relays:** LoRa relays are placed at high points to cover the festival grounds. Battery-powered relays are used where solar is not practical. - **Supernode election:** The organizer nodes (laptops at the info booth and entrance) become supernodes. They coordinate traffic between zones and maintain the festival mesh backbone.

Transport mix: Bluetooth for very short-range peer-to-peer, WiFi Direct for local mesh connections, LoRa for ground-to-info-booth links, WebRTC disguised as video call for any internet-bound traffic.

Privacy tier: Attendee phones operate at L1. Organizer laptops operate at L2. LoRa relays operate at L2 if plugged in, L1 if battery-powered.

Lessons from this scenario: - High-density events need relays at multiple points to avoid congestion. - WebRTC traffic disguised as a video call avoids detection and blocking by event security. - Bluetooth is useful for very short-range connections between nearby attendees.

---

6. Lessons Learned

- **Resilience:** Mesh networks keep working when infrastructure fails. The tiered hierarchy ensures that even if individual nodes fail, the mesh adapts.
- **Privacy:** End-to-end encryption protects users in hostile environments. Privacy tiers automatically adjust based on battery and device capability.
- **Flexibility:** Multiple transports and adapters enable diverse deployments. No single transport is required for the mesh to function.
- **Community:** Local knowledge and training are key to success. The right people in the right roles make the difference.
- **Hierarchy matters:** Assigning roles by device capability, not just availability, produces a more stable and efficient mesh.
- **Battery awareness:** Nodes that ignore battery state degrade the entire mesh. Energy-aware transport selection and privacy tier downgrades are essential.

---

7. Visuals: Scenario Topologies

---

8. Best Practices & Takeaways

- Train users before deployment.
- Test mesh in real-world conditions.
- Use multiple transports for coverage.
- Document lessons and share with the community.
- Assign roles by device capability.
- Monitor battery state and adjust privacy tiers accordingly.
- Elect supernodes early in any deployment.
- Plan for the tiered architecture from the start.
- Cross-reference the chapter on who GhostWire serves for persona-specific deployment guidance.
- Cross-reference chapter 15_device_hierarchy_and_scaling.md for the full device hierarchy specifi-

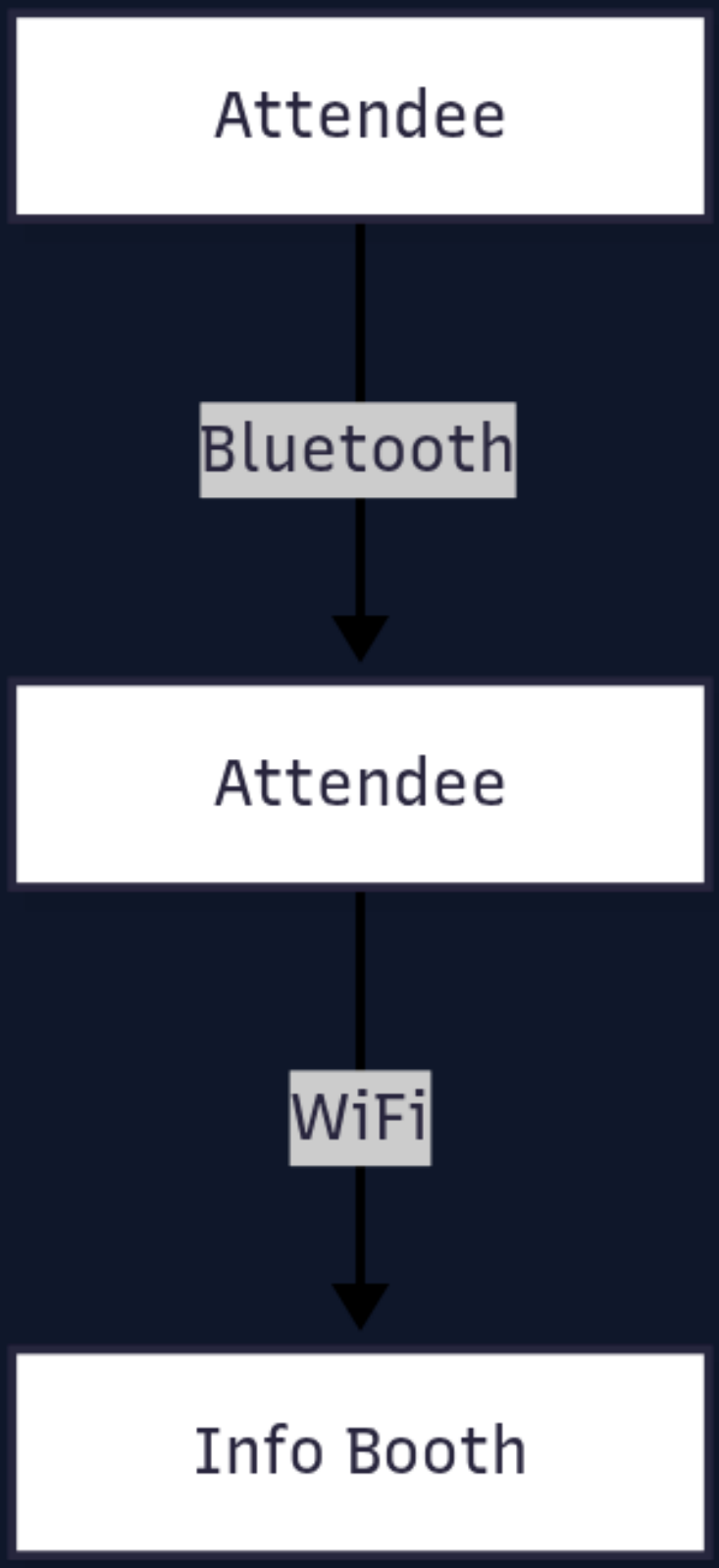


Diagram 16: case studies

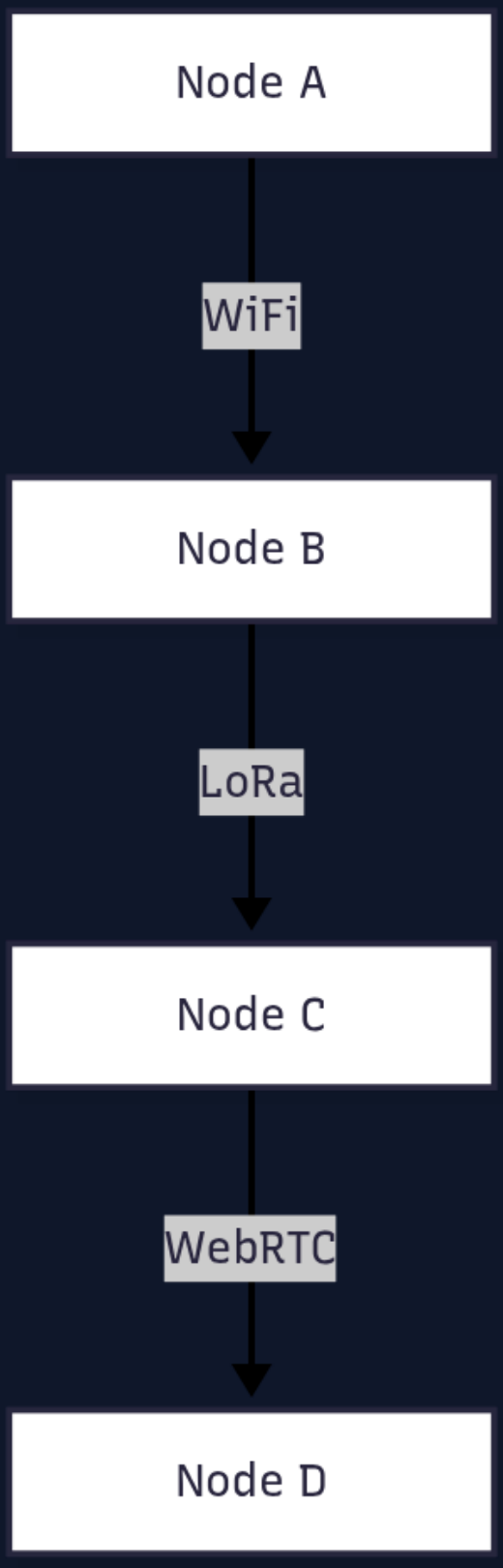


Diagram 17: case studies

cation.

---

9. Further Reading & Resources

- Mesh Networking 101
- GhostWire Developer Guide
- GhostWire Target Users
- GhostWire Device Hierarchy and Scaling

End of Chapter

Glossary & Reference

Table of Contents

1. Glossary
2. Acronyms & Definitions
3. Quick Reference: Commands & Config
4. Troubleshooting Table
5. Visuals: Reference Flows
6. Cross-References
7. Further Reading & Resources

---

1. Glossary

Term/Acronym	Meaning
Mesh Network	A network where each device (node) relays data for others, creating a resilient web of connections.
Transport	The method or technology used to send messages. Working today: stealth TCP, WebRTC, Reticulum. LoRa, WiFi Direct, and Bluetooth LE are designed but not implemented.
Protocol Adapter	Software that bridges GhostWire to other networks (Briar, Matrix, etc.).
Node	Any device running GhostWire (phone, laptop, server, etc.).
Store & Forward	Technique where messages are cached and relayed when possible.
Sybil Attack	Attack where one entity pretends to be many nodes.

Term/Acronym	Meaning
Quota	Limit on messages or actions to prevent abuse.
Federation	Connecting multiple meshes for wider reach.
Blacklist	List of nodes blocked from the mesh.
QuotaEnforcer	Module that limits message rates.
KeyManager	Module that manages encryption keys.
Supernode	A node elected dynamically based on high uptime, stable IP, and significant bandwidth to act as a relay hub for the mesh.
Relay Hub	A supernode that forwards messages between devices, carrying no trust authority over the data it relays.
Device Tier	A classification of device based on available RAM, storage, and power, determining its role and capabilities in the mesh.
Roles (Full/Relay/Leaf/Sensor/Replicator)	Device roles in GhostWire: Full nodes do everything; Relay nodes forward messages; Leaf nodes send and receive only; Sensor nodes collect data; Replicator nodes store and replicate data.
Battery State	The current charge level of a device, which determines its allowed activity level in the mesh.
Privacy Tier (L0-L3)	A classification of privacy visibility. Servers operate at L3, laptops and tablets at L2, phones and Raspberry Pis at L1. Battery drops push a device one tier down.
Delay-Tolerant Networking (DTN)	A networking approach where messages are stored and forwarded when connectivity is intermittent or unavailable.
PRoPHET	A protocol for delay-tolerant routing that uses encounter history to make forwarding decisions.
Spray-and-Wait	A routing strategy where a message is sent to multiple nodes and the first to reach the destination wins.

Term/Acronym	Meaning
LightGBM	A gradient boosting framework designed for per-message classification. Not present in the current build — no model file is in the repository and the layer is not called from routing decisions.
Gemma	A family of AI models used in GhostWire for analytical passes and network mapping, running on edge devices and relay laptops.
Graph Neural Network (GNN)	A neural network that operates on graph structures, used in GhostWire for routing decisions across the mesh.
ML-KEM-768	A post-quantum key encapsulation mechanism providing quantum-resistant key exchange, measured at about 10.5 microseconds on x86_64 development hardware.
XChaCha20-Poly1305	An authenticated encryption algorithm used for end-to-end message encryption in GhostWire.
Duress PIN	A PIN that, when entered under coercion, switches the device to a decoy identity namespace. Implemented in the identity subsystem; not yet exposed through the API or controller layer.
Dead Man's Switch	A timer that fires when the user fails to check in within a set period, notifying trusted contacts. Automatic key wiping is not yet implemented.
Shamir Secret Sharing	A method to split a secret into parts, requiring a threshold number of parts to reconstruct, used for device recovery.
Decentralized Identifier (DID)	A self-owned identifier that does not depend on a central authority, used for device and user identity in GhostWire.
KERI	A protocol for decentralized identifiers and verifiable events, planned for integration with GhostWire.
SPHINX	A routing protocol that provides unlinkable replies, ensuring that the sender of a message cannot be identified by the recipient.

Term/Acronym	Meaning
Unidirectional Reply Route (SURB)	A routing mechanism that allows a recipient to reply to a sender without revealing the sender's identity to the network.
Cover Traffic	Extra network traffic generated to obscure the patterns of real communication, making it harder to analyze who is talking to whom.
Erasure Coding	A method of encoding data into fragments with redundancy, allowing reconstruction from a subset of fragments (e.g., 3 of 5 shards).
Thompson Sampling	A bandit algorithm used for energy-aware transport selection, balancing exploration of new routes with exploitation of known good ones.
Traffic Armor	A camouflage system that disguises GhostWire traffic and auto-switches transports when one is blocked.
Adapter Registry	A system that allows custom protocol bridges to register at startup, enabling pluggable bridge support.
Knowledge Ark	A distributed storage mechanism for preserving and sharing knowledge across the mesh.
DHT	A distributed hash table used for peer-to-peer storage and lookup of model files and other data across the mesh.

2. Acronyms & Definitions

- **E2EE:** End-to-End Encryption
- **UI:** User Interface
- **CLI:** Command-Line Interface
- **API:** Application Programming Interface
- **DoS:** Denial of Service
- **PoW:** Proof of Work
- **PoS:** Proof of Stake
- **LoRa:** Long Range (radio technology)
- **P2P:** Peer-to-Peer
- **PQC:** Post-Quantum Cryptography
- **DTN:** Delay-Tolerant Networking
- **DID:** Decentralized Identifier

- **KERI:** Key Event Receipt Infrastructure
- **SURB:** Unidirectional Reply Route
- **ZKP:** Zero-Knowledge Proof

---

3. Quick Reference: Commands & Config

CLI Commands

```
ghostwire-cli start --transport wifi --mesh mymesh
ghostwire-cli send --to bob --message "Hello, Bob!"
ghostwire-cli nodes
```

Config Example

```
[transports]
bluetooth = true
wifi = true
lora = false
webrtc = true
tcpip = true
stealth_tcp = false
```

---

4. Troubleshooting Table

Problem	Solution
Can't find other nodes	Check transport settings, try another method
Messages not sending	Ensure at least one transport is active
Web UI won't load	Check backend is running, try npm install
LoRa not working	Check hardware, drivers, and permissions
Security warning	Ensure you are using the latest version

---

5. Visuals: Reference Flows

---

6. Cross-References

- Getting Started
- Architecture Deep Dive
- Transports & Protocols
- Security & Privacy
- Developer Guide
- Device Hierarchy & Power-Aware Scaling
- AI Routing & On-Device Intelligence

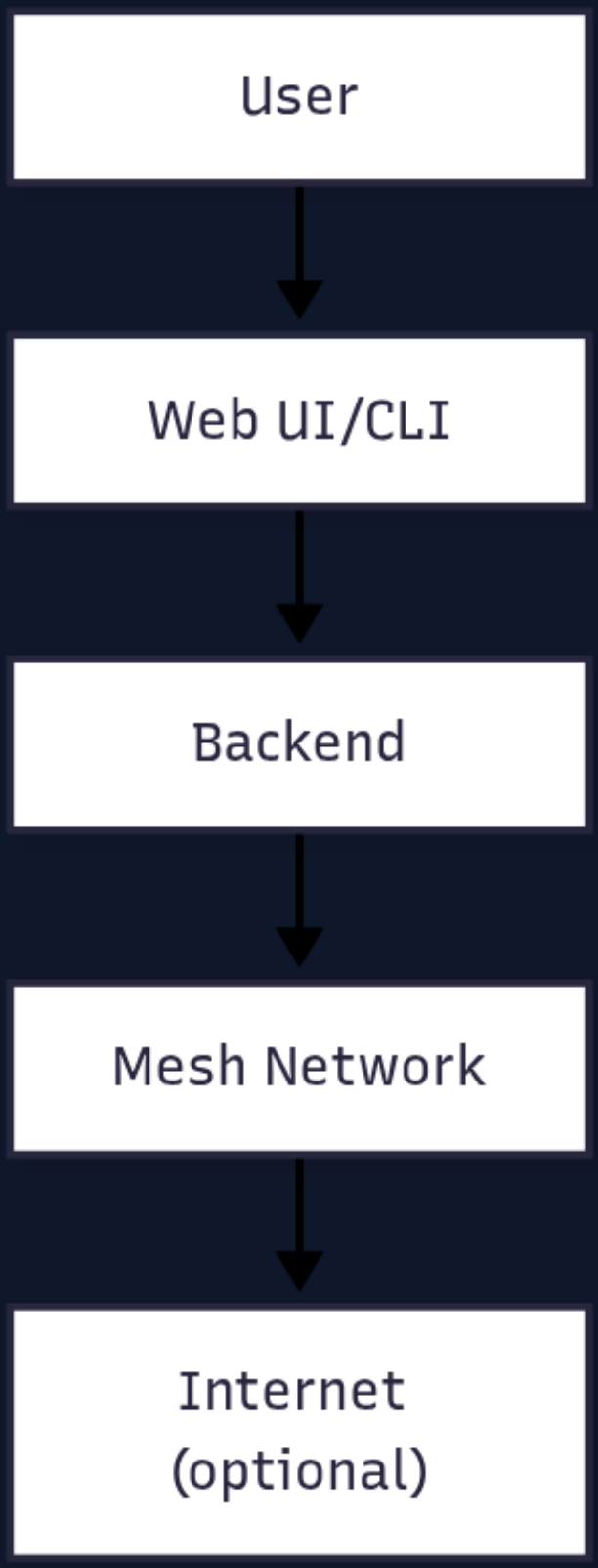


Diagram 18: glossary reference

- Who GhostWire Serves

---

7. Further Reading & Resources

- GhostWire GitHub
- Mesh Networking 101

End of Chapter

15. Device Hierarchy, Roles, and Scaling

---

1. Executive Summary

Not every device in a GhostWire mesh is the same. A phone has less memory and battery than a laptop or a server. GhostWire organises devices into classes and tiers so the network behaves predictably as it grows.

Two things are worth understanding before anything else, because both correct a common misreading of this chapter in earlier revisions:

A leaf device still relays. "Leaf" is a *default assignment*, not a hard capability limit. Every node forwards messages for others — the forwarding path in `core/mesh.rs` has no role gate at all. What changes is willingness and capacity, not permission. A phone that is the only node in range will relay because nothing else can. That is the resilient case, not an exception to be engineered around.

The hierarchy also selects AI models. Device class maps to a recommended Gemma 4 model size (`DeviceClass::recommended_gemma4_model()` in `core/ai/function_calling.rs`), so the same classification that governs relay behaviour governs which model a device can run. Details in section 9.

---

2. Why a Hierarchy Exists

Plain-Language Explanation

Think of a convoy. A motorcycle weaves between houses carrying messages for the immediate neighbourhood. A truck carries supplies between neighbourhoods. The motorcycle is quicker but carries less; the truck is slower but moves far more.

GhostWire works the same way. A phone is the motorcycle — always with you, always connected, limited in what it can carry. A laptop or nearby server is the truck. When a stronger machine is available it takes on more relay work. When a phone's battery is low it prefers to hand that work to a nearby laptop — **and if there is no nearby laptop, it keeps relaying anyway.**

Technical Depth

A flat mesh is one where every device has an identical role. As the mesh grows, every device must track every other, and the overhead outgrows the network. In a fully-connected mesh of n nodes the edge

count is *n(n-1)/2*: roughly 45 connections at 10 nodes, about 4,950 at 100, and about 499,500 at 1,000. No consumer device sustains that.

GhostWire introduces tiers so devices at the top of the hierarchy hold more connections and data, while devices at the bottom focus on sending and receiving. This lets the network grow without collapsing under its own weight.

The principle from the project's scalability guidance: *assign key nodes with good power and connectivity as relays, and use devices with more RAM/CPU for relay duties.*

---

3. Device Classes

Plain-Language Explanation

GhostWire sorts devices into five classes by available RAM, storage, and peer capacity. A phone is limited because it also runs everything else and must conserve battery. A server has no such constraint. Knowing the class tells you what a device can reasonably be asked to do.

Technical Depth

Class	RAM threshold	Max peers	Storage
Phone	≤6 GB	20	1 GB
Tablet	≤8 GB	30	2 GB
Raspberry Pi	≤4 GB	15	500 MB
Laptop	≤16 GB	50	5 GB
Server	>16 GB	100+	50 GB+

The classification is real and implemented. `DeviceClass` and `detect_device_class()` live in `core/ai/function_calling.rs:808-841`, with tests covering per-class model selection and Linux detection:

```
pub fn detect_device_class() -> DeviceClass {
    let total_ram_mb = get_total_ram_mb();
    if is_raspberry_pi() { return DeviceClass::RaspberryPi; }
    match total_ram_mb {
        0..=6144   => DeviceClass::Phone,
        6145..=8192 => DeviceClass::Tablet,
        8193..=16384 => DeviceClass::Laptop,
        _ => DeviceClass::Server,
    }
}
```

Raspberry Pi is checked before the RAM match, so a 4 GB Pi is classified as a Pi rather than a phone. The authoritative source is `docs/DEVICE_TIERS.md`.

> **One caveat.** This classification currently drives **AI model selection** (section 9). It does **not** yet drive relay-role assignment — see section 4.

---

4. Node Roles — and What "Leaf" Actually Means

Plain-Language Explanation

Five roles are described: Full, Relay, Leaf, Sensor, and Replicator. A leaf is a device that mainly sends and receives its own messages. A relay forwards for others. A full node stores the complete mesh state. A sensor publishes physical data. A replicator keeps copies for recovery.

The important correction: **a leaf is a default, not a prohibition.**

Technical Depth

Role	Forwarding	Power	Storage	Dialable
Full	Full DHT + gossip	Always on	Full node store	Yes (listeners)
Relay	Core ports only	Low-current/solar	Warm tier	Yes (relay reservation)
Leaf	Primarily local, but **relays when no better peer is available**	Sleep-heavy	Minimal (own groups)	Outbound
Sensor	Local only	Ultra-low (duty cycle)	Read-only	No
Replicator	Forwards encrypted blocks	Always on	Full (cannot decrypt)	Yes

How forwarding actually works. In `core/mesh.rs` the relay path is:

```
if !msg.is_broadcast()
    && msg.recipient != self.local_peer_id.to_string()
    && msg.hops < MAX_HOPS
{
    let mut forwarded = msg.clone();
    forwarded.hops += 1;
    // ... publish to the DM topic
}
```

The only conditions are *not for me*, *not a broadcast*, and *under the hop limit*. There is no `is_leaf`, `can_relay`, or `role ==` check anywhere in `src/`. **Every node relays.** A phone designated Leaf in a neighbourhood with laptops will typically forward little, because gossipsub and the route selector prefer better-suited peers — but that is emergent routing preference, not an enforced restriction.

This is the correct design for a resilient network. In a blackout, the assumption that a relay-class device exists is often false. If a leaf refused to relay in that situation, the mesh would partition. Better to relay on a phone for a short window than to have no path at all. Energy cost is real; connectivity loss is worse.

What is not yet implemented is the *positive* side: there is no mechanism that promotes a leaf to relay when the network needs it, nor one that demotes it when battery is low. Role assignment is currently a documented design, not runtime behaviour. The `DeviceTier` enum that does exist (`core/armor.rs:8-16` — High/Medium/Low) adjusts traffic-shaping parameters, not roles:

Tier	Devices	Packet size	Max padding	Chaff delay	Cover interval
High	Desktop, laptop on AC, server	1024	512	200 ms	5–15 s
Medium	Laptop on battery, modern phone	—	—	—	—
Low	IoT, old phone, constrained	—	—	—	fast cover

---

5. Relay Authority

Relays and supernodes carry **no trust authority**. They route messages; they cannot read, modify, or block them. End-to-end encryption means only the sender and intended recipient can read a message. A relay is a postal worker carrying a sealed letter — it cannot open it.

This holds regardless of how many relays participate, including the case where a phone is the only node relaying.

---

6. The Mailbox Pattern

When a leaf device is off or out of range, an always-on node buffers messages until it reconnects — a post-office box. The design follows the Briar project's observation that *power-constrained nodes offload storage and forwarding to a friend*.

In GhostWire the mailbox is a designated node holding messages for offline devices, which the device pulls on reconnection. This is a design target; see the status spine for implementation state.

---

7. Why a Flat Mesh Fails

See section 2 for the connection-count arithmetic. The project's own scalability guidance states the problem directly: *"A flat mesh becomes unstable beyond a certain density. We should introduce a tiered architecture."*

Staged targets from the scalability roadmap:

Phase	Target	Approach
Current	10–50 nodes	Flat mesh with DHT and gossipsub
V2.2	100–500 nodes	Protobuf and Kademlia DHT optimisation
V3.0	1,000+ nodes	Hierarchical supernodes, regional DHT sharding
V4.0	10,000+ nodes	Regional federation

---

8. Supernodes and Relay Hubs

In larger deployments, Relay Hubs connect smaller clusters. Selection favours nodes with high uptime, stable IP addresses, and sufficient bandwidth, and the election is dynamic — if a hub goes offline, another is selected.

Regional DHT sharding splits the Kademlia DHT geographically so a device in one city does not track every peer in another.

> Note: the diagram shows the *target* hierarchy. Supernode election and regional sharding are roadmap items, not current behaviour.

---

9. The Hardware Ladder — and the AI Model Ladder

Plain-Language Explanation

The hardware ladder describes what class of device can participate. The model ladder describes which AI model that device can run. Both are driven by the same device classification, so a deployment's hardware decisions automatically determine its intelligence budget.

The Hardware Ladder

Constrained microcontrollers (~$20 class). An ESP32 has on the order of 148 KB of free RAM with WiFi and BLE active — far below what rust-libp2p needs for a full node. The intended design is gateway-bridged: the device forms a short-range mini-mesh (ESP-NOW or BLE) and bridges to the wider network over a WiFi uplink, participating as a leaf or sensor rather than a full node. This follows the Meshtastic bridge pattern and microfips' `no_std` handshakes.

> **Not implemented.** There is no ESP32 firmware in the repository, and the LoRa and BLE transports that such a node would use are not implemented. Treat this as an architectural direction.

Raspberry Pi (~$100 class). More memory, runs a fuller build. Serves as a gateway for a neighbourhood or small deployment, can relay, store data, and act as a mailbox.

Server-class nodes. The heaviest loads: supernode, relay hub, replicator, full mesh state, backbone for large deployments.

The Model Ladder

This part is **implemented and tested**. `DeviceClass::recommended_gemma4_model()` maps each class to a model size — for example Laptop → `gemma4:26b`, Server → `gemma4:31b` — with unit tests asserting the mapping (`core/ai/function_calling.rs`, `test_device_model_selection`).

The reasoning is the same as the hardware ladder, applied to inference cost:

Device class	Budget	Model expectation	Relay expectation
Server	Unlimited, always on	Largest model (e.g. `gemma4:31b`)	Full node, backbone
Laptop	Ample, intermittent	Large model (e.g. `gemma4:26b`)	Relay, full store

Device class	Budget	Model expectation	Relay expectation
Tablet	Moderate	Mid-size model	Leaf or light relay
Phone	Constrained, battery	Small model, duty-cycled	Leaf by default, **relays when nothing else is available**
Raspberry Pi / constrained	Tight	Smallest viable model, or none	Relay for its segment

Two caveats, stated plainly. First, the model *selection* logic exists and is tested; the models themselves do not — no model file is present in the repository, so nothing loads at runtime (chapter 16). Second, because no role gating exists on forwarding (section 4), the "relay expectation" column describes *design intent*. In practice any device may relay; capacity and routing preference, not policy, determine how much.

---

10. Privacy Tiers by Device

Plain-Language Explanation

Privacy tiers were designed to trade metadata exposure against practicality — a plugged-in server can afford stronger cover traffic than a phone on 15% battery.

Technical Depth

> **These tiers are documentation-only.** `PrivacyParams` and `PrivacyLevel` have **zero occurrences in `src/`.** The tables below are preserved because they document the intended design and the design source exists (`docs/PRIVACY_TIERS.md`), but nothing enforces them at runtime. The mechanism that *is* implemented is the `DeviceTier` (High/Medium/Low) in section 4, which adjusts cover traffic and padding rather than a five-level ladder.

Level	Name	Latency	Cover traffic	Intended use
L0	None	0 ms	0%	Debugging, trusted LAN
L1	Low	50 ms	10%	Daily messaging
L2	Medium	150 ms	30%	Sensitive communication
L3	High	500 ms	50%	High-risk, censorship zones

Intended defaults: Server L3, Laptop L2, Tablet L2, Phone L1, Raspberry Pi L1. Intended battery rule: charging or above 50% means no change; 20–50% downgrades one level; below 20% drops to L0/L1.

The device also cannot read battery state today. So the battery-driven tier adjustment described in earlier revisions of this chapter does not happen, by either route.

---

11. A Practical Worked Example

A community centre with four devices: a phone, a laptop, a Raspberry Pi, and a server.

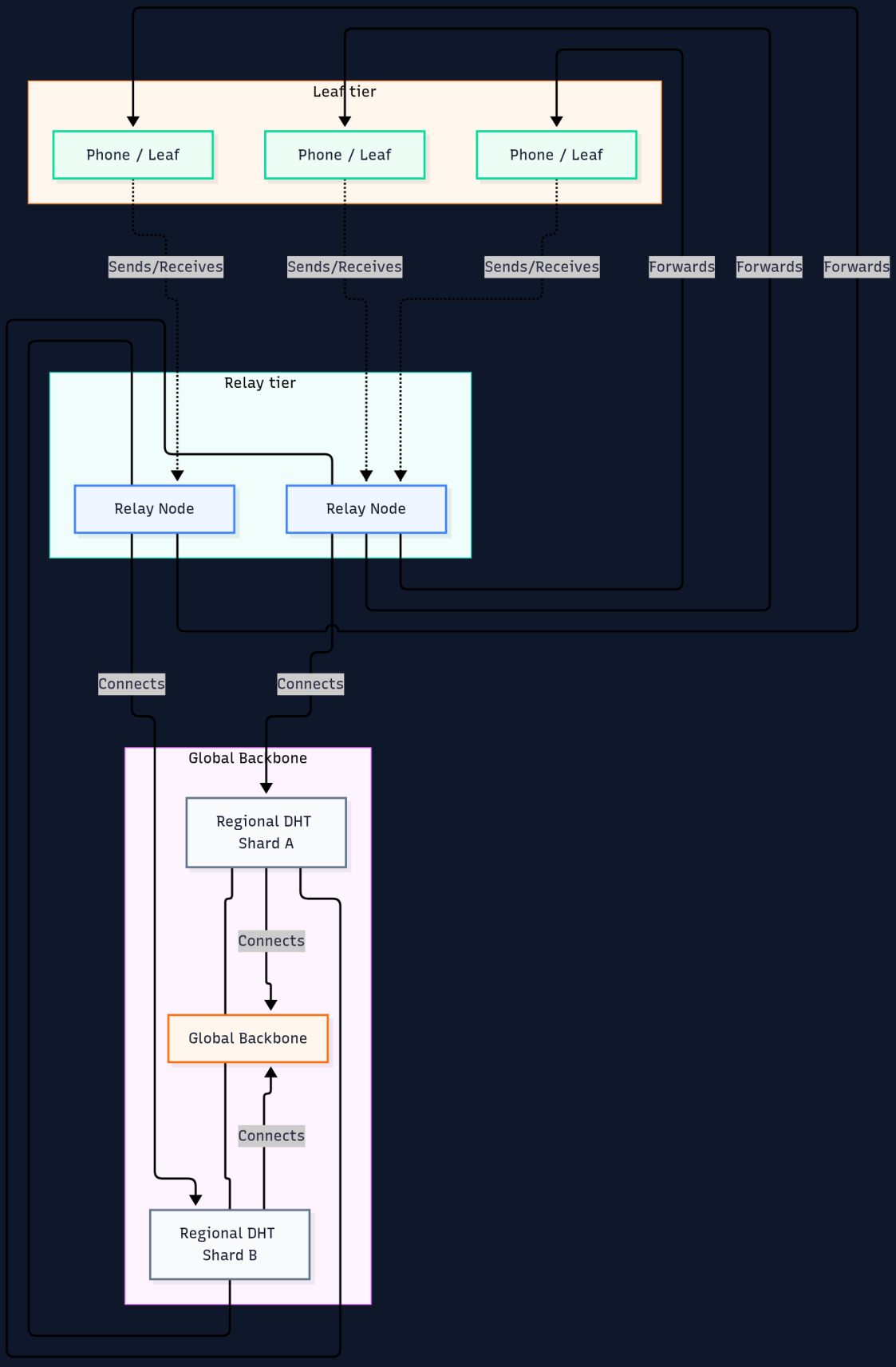


Diagram 19: device hierarchy and scaling

Normal case. The laptop relays, the Pi acts as a segment gateway, the server holds full mesh state and connects outward, and the phone is primarily an endpoint. This is the intended assignment.

Resilience case — and this is the point. The laptop goes home. The Pi's uplink drops. Now the only node in range is the phone. The phone relays, because `mesh.rs` has no role gate and nothing else can. Messages still flow between whoever is present. When the laptop returns, routing prefers it again and the phone's load drops.

Model assignment. The same classification picks models: the server would take `gemma4:31b`, the laptop `gemma4:26b`, the phone a small duty-cycled model.

Mailbox. While the phone is away, a peer buffers messages for it; it pulls them on return.

---

12. Best Practices and Anti-Patterns

- Use the tiered architecture; do not make every device do identical work.
- Assign relay roles to devices with stable power and connectivity.
- **Do not treat "leaf" as a safety guarantee.** If your only nearby node is a phone, it will relay. Design for that rather than assuming capacity you don't have.
- Size deployments against measured behaviour, not the tables in this chapter.
- Enable Android battery-optimisation exemptions on OEM devices (Samsung, Xiaomi, Transsion) — see dontkillmyapp.
- Expect no external security audit yet. Cure53, NCC Group, and Least Authority have been identified as candidates; none has been engaged.

Anti-Patterns

- Assuming a leaf device will refuse to relay, and relying on that for isolation.
- Expecting an ESP32 to perform full-node duties.
- Planning beyond current scale without measuring.
- Omitting battery-management exemptions on OEM Android devices.
- Assuming relays can read or censor message content — they cannot.
- Assuming an external audit has been performed.

---

13. Glossary

- **Device Class** — RAM-based classification: Phone, Tablet, Raspberry Pi, Laptop, Server. Implemented in `core/ai/function_calling.rs`.
- **DeviceTier** — High/Medium/Low, adjusts cover traffic and padding. Implemented in `core/armor.rs`. Distinct from the unimplemented privacy tiers.
- **Relay Hub / Supernode** — high-uptime node connecting mesh segments. Roadmap.
- **Mailbox Pattern** — an always-on peer buffers messages for an offline device.
- **Privacy Tier** — intended L0–L3 metadata ladder. **Documentation-only.**
- **Regional DHT Sharding** — geographic partitioning of the DHT for scale. Roadmap.
- **Gateway-Bridged Architecture** — constrained device joins via a short-range mini-mesh and a WiFi uplink. Design, not implemented.
- **Model Ladder** — device class to AI model size. Selection logic implemented and tested; models themselves absent.

- **Trust Authority** — ability to read, modify, or block messages. Relays have none.
- **Cover Traffic** — padding messages obscuring the timing and size of real ones.
- **Supernode Election** — dynamic selection of Relay Hubs by uptime, address stability, and bandwidth. Roadmap.

---

14. Further Reading

- Project Overview
- Performance & Scaling
- Architecture Deep Dive
- AI Routing & Intelligence
- Capability Status Spine
- Device Tiers Guide
- Privacy Tiers Guide — design source
- dontkillmyapp

End of Chapter

16. AI Routing and Intelligence

Table of Contents

1. Executive Summary
2. The Four-Layer AI Routing Stack
3. Layer 1: LightGBM Per-Message Classifier
4. Layer 2: Gemma 4 Edge Model
5. Layer 3: Graph Neural Network (GNN)
6. Layer 4: PRoPHET Delay-Tolerant Routing
7. Model Ladder per Hardware
8. Model Update Flow and Training Pipeline
9. Thompson Sampling Transport Selection
10. Privacy and Local AI Decisions
11. Knowledge Ark and Anomaly Detection
12. Worked Example: A Message's Journey
13. Future Scopes
14. End of Chapter

---

1. Executive Summary

GhostWire does not rely on a central server to decide where your messages go. Instead, every device runs a four-layer AI routing stack locally. The stack reads the mesh network, predicts the best path, and adapts in real time. A lightweight model makes the fastest decisions. A larger model provides deeper analysis when needed. A graph neural network optimizes across the whole topology. And a battle-tested

delay-tolerant protocol stands ready as the final fallback.

All of this happens on your own device. No cloud AI dependency. No external server making routing choices for you. Your data stays on your machine, and any telemetry shared upward is strictly opt-in and anonymized.

This chapter walks through each layer, the hardware each model runs on, how models get updated, how transport selection works, and what the future holds.

---

2. The Four-Layer AI Routing Stack

The AI routing design is a cascading fallback chain. Each layer is intended to handle a different kind of decision, with the next layer taking over when one is unavailable.

> **Current build status, verified 2026-09-28:** in the shipped configuration **only Layer 4 (PRoPHET) is active**. No model files are present in the repository — there are no `.onnx` artifacts anywhere in the tree — so Layers 1 and 3 both fail to load and resolve to `None` via silent `tracing::debug!` fallbacks (`core/mod.rs:781–806`) that are invisible at default log level. Layer 1 is additionally not called from routing decisions at all. Layer 2 requires a local Ollama instance.
>
> The chain below describes the **architecture being built**, not the behaviour of the current binary. Routing works today via PRoPHET and the gossipsub mesh. See docs/BIBLE_STATUS_SPINE.md §5.

Why the model layers come last

This ordering is deliberate, not unfinished work. A routing classifier is only as good as the telemetry it was trained on, and GhostWire does not yet have that telemetry: there are no deployed nodes producing sustained, representative mesh traffic. Training on synthetic or lab-scale data would produce a model that fails on the first real deployment — a worse outcome than honest PRoPHET, because it would fail silently.

The dependency runs the other way. A working mesh generates the data, and the data enables the model. So the sequence is: **get the core protocol into production, let the network grow gradually, accumulate genuine multi-node telemetry, and only then train Layer 1 and Layer 3.** PRoPHET is a principled formula rather than a learned one, which is exactly why it is the right default for a network that does not yet have the data to justify a model.

Shipping a model layer before the network is the mistake this sequencing avoids.

Diagram 20: ai routing and intelligence

The arrows above show the intended fallback direction. Layer 1 is designed as the fast path; if it fails the system drops to Layer 2, and if Layer 2 fails, Layer 3 takes over. Layer 4 requires no model file, which makes it the only layer that is genuinely always available — and, in the current build, the only one that is actually running.

---

3. Layer 1: LightGBM Per-Message Classifier

Plain-Language

Layer 1 is the fastest decision-maker in the stack. It looks at 2,387 features about the mesh network and outputs a delivery probability score in under a millisecond. Think of it as the reflex system: it reacts instantly, before you even think about the problem.

The model was trained on real data from the GuifiSants mesh network in Catalonia. That means it has seen how actual nodes behave, not just how simulated nodes behave. The training set included 7,931 samples across 63 nodes over 31 days.

Technical Depth

The model is a LightGBM gradient boosting decision tree ensemble exported to ONNX format. It takes a single 2,387-dimensional feature vector and outputs a score between 0.0 and 1.0 representing the predicted probability of successful delivery.

Property	Value
Algorithm	LightGBM (Gradient Boosting Decision Trees)
Format	ONNX (Open Neural Network Exchange)
Input dimensions	2,387 features
Output	Delivery probability score (0.0 to 1.0)
Inference time	Not yet measured — no model file is present in the repository
Model size	~2-5 MB

The feature vector is built from seven categories:

Category	Feature Count	Examples
Link quality	256	SNR, RSSI, packet loss rate, retransmission count
Latency	128	RTT, jitter, one-way delay, queue depth
Node health	64	Uptime, CPU usage, memory usage, buffer occupancy
Topology	512	Degree, betweenness centrality, clustering coefficient
Temporal	256	Hour of day, day of week, season, activity patterns
Geographic	128	Distance, elevation, line-of-sight probability
Historical	512	Rolling averages over 1h, 6h, 24h, 7d
Derived	531	Interaction terms, polynomial features, lag features

Missing features are filled with the median value from the training distribution. If more than 30% of features are missing, Layer 1 is considered unavailable and the system falls back to Layer 2.

---

4. Layer 2: Gemma 4 Edge Model

Plain-Language

When the fast path is not enough, the system turns to Gemma 4. This is a language model that actually understands mesh topology. It reads the network state, thinks about the options, and gives a reasoned recommendation. It runs every 30 seconds on a schedule, not on every single message, because it is slower and more expensive than Layer 1.

The model runs entirely on your device through Ollama or LiteRT. No data leaves your machine. The model is quantized to 4-bit precision (Q4_K_M) to keep it small enough for edge hardware.

Technical Depth

Property	Value
Model	Gemma 4 (270M parameter variant)
Context window	32,768 tokens
Runtime	Ollama or LiteRT
Quantization	Q4_K_M (4-bit, ~150 MB)
Inference time	~30 seconds
Hardware minimum	4 GB RAM

The prompt given to Gemma 4 includes the full network state: available peers, link qualities, recent delivery rates, current time, message priority, and TTL. The model returns a route as an ordered list of peer IDs, a confidence score, and a text explanation.

On devices with less than 4 GB of RAM, Layer 2 is skipped entirely and the system falls straight to Layer 3.

---

5. Layer 3: Graph Neural Network (GNN)

Plain-Language

Layer 3 looks at the mesh as a graph. Every node is a point. Every connection between nodes is a line. The neural network learns patterns across this graph structure to predict which paths will work best. It is fast and precise, running in under a hundred microseconds.

Technical Depth

Property	Value
Architecture	Graph Attention Network (GAT)
Layers	4 GAT layers

Property	Value
Attention heads	8 heads per layer
Format	ONNX
Inference time	91μs (ONNX Runtime, CPU)

The mesh network is represented as a graph G = (V, E). Nodes are active peers. Edges are active communication links. Each node has features like uptime, buffer usage, average latency, packet loss rate, trust score, geographic coordinates, and transport capacity. Each edge has features like link quality, round-trip time, bandwidth, and how recently the peers were seen.

The GAT layers compute attention-weighted messages. Each node pays attention to its neighbors, weighting their importance based on learned patterns. Eight attention heads run in parallel, and their results are concatenated. The output is a delivery probability score for each candidate route.

---

6. Layer 4: PRoPHET Delay-Tolerant Routing

Plain-Language

Layer 4 is the oldest and most reliable layer. It does not need any AI model. It does not need any model file at all. It uses a simple formula based on how often nodes have met each other. If node A has met node B many times, and node B has met node C many times, then A has a good chance of reaching C through B. This is called transitivity.

PRoPHET is always on. It is the safety net that catches every message when all the AI layers fail.

Technical Depth

Property	Value
Protocol	PRoPHET v2 (epidemic routing)
Delivery predictability update	P(a,b) = P_old + (1 - P_old) $\times$ P_init
Time decay	$\gamma = 0.98$ per hour
Transitivity weight	$\beta = 0.25$
Initial predictability	P_init = 0.75

When node A encounters node B, the predictability P(a,b) increases. Over time, if they do not meet, the predictability decays. When A meets B, A also learns about everyone B has met, and updates its predictability to those nodes through transitivity.

The routing decision is simple: forward the message to the neighbor with the highest predicted chance of reaching the destination. Ties are broken by buffer usage, recency of encounter, and finally by a deterministic hash of the bundle ID.

PRoPHET works alongside Spray-and-Wait for initial distribution. Spray-and-Wait sends up to a configurable number of copies (default 3, maximum 10) to encountered peers, then waits for one of them to reach the destination. GhostWire uses predictive spray by default, which combines Spray-and-Wait with PRoPHET predictability scores.

SyncBloom is a 1024-bit probabilistic data structure used for bundle deduplication during peer synchronization. When two peers meet, they exchange SyncBlooms to discover which bundles the other is missing, without revealing bundle contents.

---

7. Model Ladder per Hardware

Different hardware gets different sized models. A tiny phone runs a small model. A powerful server runs a large model. The ladder ensures every device uses the best model its hardware can handle.

Model Variant	Hardware	RAM/VRAM	Latency
Gemma 4 E2B	Edge devices, phones, Raspberry Pi	8 GB RAM	~100ms
Gemma 4 E4B	Relay laptops	10 GB VRAM	~50ms
Gemma 4 26B MoE	Coordinator nodes	20 GB RAM	~80ms
Gemma 4 31B	Central servers	24 GB VRAM	~120ms

The E2B variant is the smallest at 2 billion parameters. It runs on 8 GB of RAM and is ideal for Raspberry Pi mesh nodes and phones. The E4B variant at 4 billion parameters targets relay laptops with mid-range GPUs. The 26B MoE (Mixture of Experts) model activates only 4 billion parameters at a time, giving flagship quality at edge speeds. The 31B dense model runs on central servers with the most powerful GPUs.

All models run through Ollama or LiteRT on the device. LiteRT provides native on-device inference without requiring Ollama. The models are downloaded once and cached locally.

---

8. Model Update Flow and Training Pipeline

Plain-Language

Models get smarter over time, but they do not learn from your private data. Here is how it works: participating nodes (only if you opt in) send anonymized telemetry summaries upward. Those summaries go to training machines. The training machines use the data to build better models. The finished model files are then distributed back down to your device. Your raw data never leaves your machine. No gradients are ever shared. This is not federated learning.

Technical Depth

The model update pipeline follows these steps:

1. Collect telemetry from participating nodes (opt-in only)
2. Preprocess: normalize, handle missing values, feature engineering
3. Train LightGBM on labeled data (delivery success or failure)
4. Export to ONNX format
5. Train GNN on graph-structured data
6. Export to ONNX format
7. Publish model hashes to the DHT
8. Distribute via peer-to-peer sync

Model verification is strict. Before loading a new model, the system computes a BLAKE3 hash of the model file and compares it against the expected hash published by the GhostWire maintainers. If the hash does not match, the model is rejected and a warning is logged.

The key distinction: this is not federated learning. No gradients are aggregated across nodes. Raw training data never leaves a node. Only an opt-in telemetry summary goes up, and only finished model files come down. The model files are distributed through DHT hash publication, where peers verify against known-good hashes, plus peer-to-peer sync. In-app updates check for new models weekly.

---

9. Thompson Sampling Transport Selection

Plain-Language

Once a route is chosen, the system still needs to pick the physical transport. This is done with Thompson Sampling, a method that balances exploration (trying new transports) with exploitation (sticking with what works). Each transport has an energy efficiency weight that reflects how much battery it drains. The weights cover transports that are not all implemented — the selector can rank an unavailable transport, so a weight is not evidence of support. See the status spine §4 for which transports actually exist.

Technical Depth

The transport selector uses a `TransportMetrics` struct with fields for signal strength, throughput, packet loss, latency, energy efficiency, congestion, and reliability. Each transport type has a weight profile that encodes design intent. The energy efficiency weights are:

Transport	Energy Efficiency Weight
BLE	0.30
LoRa	0.35
Reticulum	0.25
WiFi	0.05
TCP/QUIC	0.0

BLE and LoRa score highest on energy efficiency because they are designed for low-power communication. WiFi uses moderate energy. TCP/QUIC scores zero because it assumes a powered, connected device.

The scoring blends a weighted sum of normalized metrics (70%) with Thompson Sampling exploration (30%). A Beta distribution posterior tracks each transport's success rate in each context (battery tier, signal tier, congestion tier). Hysteresis prevents rapid switching: a new transport must beat the current one by at least 0.1 score points, and a 30-second cooldown applies between switches.

Black-hole detection monitors each transport's success rate. If a transport fails too many times in a sliding window, it is blocked until a success resets the detector.

---

10. Privacy and Local AI Decisions

Plain-Language

Every AI decision in GhostWire happens on your device. The models run locally. The routing choices are made locally. No cloud server sees your network state, your message content, or your routing decisions.

Telemetry is strictly opt-in. If you choose to participate, only anonymized summary statistics leave your device. Node IDs are hashed before export. No personally identifiable information is included in any training data.

There is no cloud AI dependency for core routing. The system works entirely offline. If you are in a remote area with no internet connection, the AI routing still functions because every layer runs on your own hardware.

---

11. Knowledge Ark and Anomaly Detection

Plain-Language

Beyond routing, GhostWire has a knowledge layer called the Knowledge Ark. Every node can store notes, articles, code snippets, and references. These entries replicate across trusted peers through the DHT. The Knowledge Ark is content-addressed, meaning every entry has a unique hash that verifies its integrity.

Anomaly detection runs alongside the routing stack. An Isolation Forest model monitors mesh network traffic and flags unusual patterns. This is the early warning system: if a node is degrading or behaving strangely, the anomaly detector catches it before it causes routing problems.

Technical Depth

The Knowledge Ark (`knowledge_ark.rs`) manages collections of entries. Each entry has a title, body, kind (note, article, code, reference, dataset, media clip, question, or answer), access level (public, trusted, shared, or private), and content-addressed ID computed as a BLAKE3 hash. Entries are stored in a local sled tree for search and retrieval. Full-text search indexes words across titles and bodies.

The Isolation Forest model (`anomaly_detector.rs`) is an ensemble of decision trees. Each tree partitions the feature space randomly. Anomalies have shorter average path lengths through the trees. The model loads from a JSON file and computes an anomaly score for each feature vector. A positive score means normal; a negative score means anomalous. The model uses 200 trees across 39 features.

---

12. Worked Example: A Message's Journey

Plain-Language

Imagine you are on your phone, sending a message to a friend who is two neighborhoods away. Your phone has a nearby laptop acting as a relay. Here is what happens, step by step, as your message passes through all four AI layers.

Technical Depth

Step 1: Layer 1 (LightGBM) — illustrative, not current behaviour

The trace below illustrates the intended design. In the current build there is no model file, so Layer 1 does not execute and the walkthrough does not occur — see the status note in section 2.

Your phone extracts 2,387 features from the mesh telemetry. The feature vector includes your phone's RSSI (-45 dBm), the laptop relay's latency (18 ms), packet loss (2%), and the trust score of the relay peer (0.9). A trained LightGBM ONNX model would process this vector and output a delivery probability. Layer 1 would then succeed, giving the system a candidate route through the laptop relay. Inference cost has not yet been benchmarked.

Step 2: Layer 2 (Gemma 4, ~30 seconds)

On a 30-second schedule, the Gemma 4 model runs a deeper analysis. The prompt includes the full network state: your phone, the laptop relay, three other nearby peers, their link qualities, and the destination's last-seen timestamp. Gemma 4 returns a route through the laptop relay with a confidence score of 0.91 and a text explanation noting that the relay has the strongest link quality and lowest latency to the destination.

Step 3: Layer 3 (GNN, 91μs)

The Graph Attention Network constructs the mesh as a graph. Your phone is a node. The laptop relay is a connected node. The destination is a distant node. The GAT layers compute attention weights across the graph. The GNN outputs a delivery probability of 0.93 for the route through the laptop relay, confirming Layer 1 and Layer 2.

Step 4: Layer 4 (PRoPHET, <0.01ms)

Even though Layers 1 through 3 all agreed, Layer 4 runs its PRoPHET formula as a safety check. The delivery predictability from your phone to the laptop relay is 0.88 (based on recent encounter history). The predictability from the laptop relay to the destination is 0.82. The transitivity boost gives a combined predictability of 0.85. Layer 4 confirms the route.

Transport Selection

The transport selector runs Thompson Sampling. Your phone has BLE and WiFi available. The energy efficiency weights favor BLE (0.30) over WiFi (0.05). The Thompson Sampling posterior for BLE in the current context (high battery, strong signal, low congestion) returns a score of 0.82. WiFi returns 0.71. BLE wins. The message is sent over Bluetooth Low Energy to the nearby laptop relay.

The Relay Forward

The laptop relay receives the bundle over BLE. It runs its own four-layer stack. Its Gemma 4 E4B model (4 billion parameters, 10 GB VRAM) analyzes the mesh. Its GNN confirms the next hop toward the destination. It forwards the bundle over WiFi or LoRa, depending on the transport selector's decision at that moment.

Bundle Details

The bundle contains the encrypted payload (XChaCha20-Poly1305), a 32-byte BLAKE3 bundle ID, a TTL of 12 hours (Normal priority), a SyncBloom for deduplication, and the previous hop list for PRoPHET state updates. If the laptop relay loses connectivity, the bundle sits in its 50 MB buffer until the next encounter.

---

13. Future Scopes

Plain-Language

GhostWire has four major directions for the future. Each one pushes the system further toward a truly resilient, private, and accessible mesh network.

Technical Depth

Nucleus ESP32: The goal is to shrink GhostWire to run on $1.00 microcontrollers like the ESP32 and STM32. The current system needs about 50 MB of RAM. The Nucleus proposal strips `libp2p` and `tokio` in favor of `no_std` alternatives like `embassy` and `smol`. A minimal Gossip Light protocol would run on low-power radio links. A bridge would connect Nucleus nodes (sensors, IoT devices) to Core nodes (hubs, servers). This would make GhostWire the global standard for $10 off-grid mesh devices.

Sovereign Persona ZKP: Identity would move beyond simple keys to rich, decentralized profiles. Decentralized Identifiers (DIDs) would follow W3C standards. Zero-Knowledge Proofs would allow attribute verification without revealing identity, such as proving you are a verified rescue worker without disclosing your name. Social recovery mechanisms would let a quorum of trusted peers help restore a lost identity.

Recursive DHT: The DHT would scale from 100 nodes to 100,000+ nodes. S/Kademlia hardening would protect against Sybil and Eclipse attacks. Hierarchical sharding would divide the global mesh into regional shards to reduce routing table bloat on mobile devices. Supernode orchestration would dynamically promote high-uptime, high-bandwidth nodes to act as regional gateways.

PQC Armor: Post-quantum cryptography would protect GhostWire traffic against future quantum computer decryption. Hybrid ML-KEM (Kyber, NIST standard) would integrate alongside X25519. PQ-Signatures would transition identity signing from Ed25519 to Dilithium (ML-DSA). Algorithm agility would let nodes step down to legacy crypto only if the peer does not support PQC.

---

14. End of Chapter

End of Chapter